

目录

目录

Chapter 1: 梯度下降算法	3
1.1 线性代数基础	3
1.1.1 实对称阵的特征值一定是实数	3
1.1.2 实对称阵一定存在相合（合同）对角化: $Q^T A Q = \Lambda$	4
1.1.3 从多元泰勒展开到凸函数二阶条件的完整推导（超详细版）	6
1.2 凸的礼物	10
1.2.1 相关定义	10
1.2.2 凸性判定的条件	11
1.2.3 强凸性	13
1.2.4 凸优化的核心定理	13
1.3 梯度下降的收敛性	13
1.3.1 基本假设与记号	13
1.3.2 关键引理	14
1.3.3 主定理: $O(1/T)$ 收敛率	14
1.3.4 强凸函数的线性收敛率	15
1.3.5 具体例子: $f(x) = kx^2$ 的梯度下降	16
1.3.6 为什么凸性保证梯度下降找到最优解?	17
1.4 具体实现过程	18
1.4.1 模拟数据生成	18
1.4.2 基础梯度下降算法实现	18
1.4.3 基础模型训练与结果查看	19
1.4.4 核心原理回顾	21
1.4.5 学习率对梯度下降的影响	21
1.4.6 Adam 优化器实战	24
1.4.7 梯度下降的三种核心变体	26
Chapter 2: 从函数到 Transformer 与新闻分类实战	27
2.1 从函数到 Transformer	27
2.1.1 现实世界与数学符号的映射	27
2.1.2 线性回归: 最基础的模型规则	27
2.1.3 激活函数: 赋予模型非线性的灵魂	28
2.1.4 复合函数: 构建多层嵌套的智能	28
2.1.5 从神经元到人工神经网络	28
2.1.6 前向传播与深度网络架构	29
2.1.7 预测偏差与损失函数	29
2.1.8 梯度下降与反向传播	30
2.1.9 过拟合与正则化技术	30
2.1.10 CNN: 卷积神经网络的崛起	31
2.1.11 RNN: 让网络拥有记忆	32
2.1.12 词嵌入: 语言的数学表达	33
2.1.13 注意力机制与 Transformer 的诞生	34
2.1.14 多头注意力与 Transformer 架构全貌	35
2.2 实战: Transformer 的基础代码与分析模块	38
2.2.1 第一步: 标准与高质量数据集的协同提取	38
2.2.2 第二步: 文本数字化（建立词典与填充）及包装为数据集	39
2.2.3 第三步: 搭建 Transformer 核心分类网络架构	40

- 2.2.4 第四步：参数与模型本质的训练循环（主步骤） 41
- 2.3 基础模型的训练优化方向 43
 - 2.3.1 从线性代数到数据归一化 43
 - 2.3.2 SGD 的概率论基础 55
 - 2.3.3 SGD 理论拆解 58
- 2.4 SGD 调参理论的具体实验 68
 - 2.4.1 实验环境与基础设置 68
 - 2.4.2 实验一：稳态方差与学习率的关系 70
 - 2.4.3 实验二：稳态方差与 Batch Size 的关系 71
 - 2.4.4 实验三：学习率与 Batch Size 的线性缩放法则 72
 - 2.4.5 实验四：学习率衰减（LR Decay）对偏差-方差的阶段性影响 73
- 2.5 Regret 理论：对学习全过程进行监控（未完待续） 74
- 2.6 Warmup: 训练前期的学习率优化方案 74
- 2.7 NKT、 μ P 理论：大规模算力下的优化策略 74

大模型学习笔记

2026 Spring

Chapter 1: 梯度下降算法

理论基础与实现

Author: Cabbage

ECNU

本讲摘要

本讲详细介绍了梯度下降算法的精确定义、核心定理及其证明，并提供了可直接运行的 Python 代码实现。梯度下降是最基础且广泛应用的优化算法，通过沿着负梯度方向迭代更新参数，逐步逼近目标函数的最小值。

1.1 线性代数基础

本文覆盖实对称矩阵特征值性质、实对称矩阵合同对角化、从多元泰勒展开到凸函数二阶条件的完整推导三大知识点，所有推导从基础定义出发，先明确定义再陈述定理、给出证明，符号体系统一为： $\mathbb{R}^n$ 表示 n 维实向量空间， $\mathbb{R}^{n \times n}$ 表示 n 阶实方阵， $D \subset \mathbb{R}^n$ 表示 n 元函数的定义域。

1.1.1 实对称阵的特征值一定是实数

相关名词定义

(1) 实对称矩阵

设 $A \in \mathbb{R}^{n \times n}$ ，若 A 的转置等于自身，即 $A^T = A$ ，则称 A 为实对称矩阵（注：若无特殊说明，“对称阵”均指实对称矩阵，复对称阵无此性质，复域中对应埃尔米特矩阵）。

(2) 特征值与特征向量

设 $A \in \mathbb{R}^{n \times n}$ ，若存在复数 λ 和非零复向量 $x \in \mathbb{C}^n$ ，使得 $Ax = \lambda x$ ，则称 λ 为 A 的特征值，非零向量 x 为 A 对应于 λ 的特征向量（特征向量必非零，这是定义核心）。

(3) 复数的共轭与复向量的共轭转置

对复数 $a + bi$ ，其共轭记为 $\overline{a + bi} = a - bi$ ；对复向量 $x = (x_1, x_2, \dots, x_n)^T \in \mathbb{C}^n$ ，其共轭为 $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)^T$ 。

复向量的共轭转置记为 $x^H = (\bar{x})^T$ ，实向量的共轭转置等价于普通转置（ $x^H = x^T$ ）。

(4) 复向量的内积

对 $x, y \in \mathbb{C}^n$ ，内积定义为 $\langle x, y \rangle = x^H y$ ，若 $x \neq 0$ ，则 $\langle x, x \rangle = \sum_{i=1}^n |x_i|^2 > 0$ （模的平方和，恒正）。

定理准确陈述

定理 1.1. 设 $A \in \mathbb{R}^{n \times n}$ 为实对称矩阵，则 A 的所有特征值均为实数。

严格证明（从定义出发）

设 λ 为实对称阵 A 的任意一个特征值， $x \in \mathbb{C}^n$ 是对应于 λ 的非零特征向量，则由特征值定义有：

$$Ax = \lambda x \quad (1.1)$$

步骤 1: 对等式 (1) 两边取共轭

因 A 是实矩阵, 其元素均为实数, 故 $\overline{A} = A$; 对等式两边共轭得:

$$\overline{Ax} = \overline{\lambda x} \implies \overline{Ax} = \overline{\lambda} \overline{x} \implies A\overline{x} = \overline{\lambda} \overline{x} \quad (1.2)$$

步骤 2: 对等式 (1) 两边取共轭转置

共轭转置满足 $(AB)^H = B^H A^H$, 且 A 是实对称阵, 故 $A^H = (\overline{A})^T = A^T = A$, 对 (1) 取共轭转置得:

$$(Ax)^H = (\lambda x)^H \implies x^H A^H = \overline{\lambda} x^H \implies x^H A = \overline{\lambda} x^H \quad (1.3)$$

步骤 3: 对等式 (3) 两边右乘 x , 结合特征值定义化简

$$x^H Ax = \overline{\lambda} x^H x \quad (1.4)$$

由 (1) 知 $Ax = \lambda x$, 代入 (4) 左边得:

$$x^H \cdot \lambda x = \overline{\lambda} x^H x \implies \lambda x^H x = \overline{\lambda} x^H x \quad (1.5)$$

步骤 4: 利用复向量内积的正定性推导 $\lambda = \overline{\lambda}$

因 x 是特征向量, 故 $x \neq 0$, 由复向量内积定义得 $x^H x = \sum_{i=1}^n |x_i|^2 > 0$, 对 (5) 两边同时除以 $x^H x$ 得:

$$\lambda = \overline{\lambda}$$

一个复数等于其共轭, 当且仅当该复数为实数, 故实对称阵的任意特征值均为实数。

推论 1.2. 实对称阵的特征向量均可取为实向量 (因特征值为实数, 特征方程 $(A - \lambda E)x = 0$ 是实系数线性方程组, 必有实非零解)。

1.1.2 实对称阵一定存在相合 (合同) 对角化: $Q^T A Q = \Lambda$

相关名词定义

(1) 矩阵的合同 (相合)

设 $A, B \in \mathbb{R}^{n \times n}$, 若存在 n 阶实可逆矩阵 C , 使得 $C^T A C = B$, 则称矩阵 A 与 B 合同 (相合), 称 C 为合同变换矩阵; 若 C 为正交矩阵, 则称 A 与 B 正交合同。

(2) 可逆矩阵

设 $A \in \mathbb{R}^{n \times n}$, 若存在 $B \in \mathbb{R}^{n \times n}$, 使得 $AB = BA = E$ (E 为 n 阶单位阵), 则称 A 可逆, B 为 A 的逆矩阵, 记为 A^{-1} ; 方阵可逆的充要条件是 $\det(A) \neq 0$ (行列式非零)。

(3) 正交矩阵

设 $Q \in \mathbb{R}^{n \times n}$, 若 $Q^T Q = Q Q^T = E$, 则称 Q 为正交矩阵; 正交矩阵的核心性质: $Q^T = Q^{-1}$ (逆等于转置), 且 $\det(Q) = \pm 1$ (必可逆)。

(4) 对角矩阵

设 $\Lambda \in \mathbb{R}^{n \times n}$, 若 Λ 的非对角线元素均为 0, 即

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n) = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{pmatrix}$$

，则称 Λ 为对角矩阵。

(5) 矩阵的相似

设 $A, B \in \mathbb{R}^{n \times n}$ ，若存在 n 阶实可逆矩阵 P ，使得 $P^{-1}AP = B$ ，则称 A 与 B 相似；若 P 为正交矩阵，则称 A 与 B 正交相似。

定理准确陈述

定理 1.3. 设 $A \in \mathbb{R}^{n \times n}$ 为实对称矩阵，则存在 n 阶正交矩阵 Q ，使得 $Q^T A Q = \Lambda$ ，其中 $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ ， $\lambda_1, \lambda_2, \dots, \lambda_n$ 为 A 的全部特征值。

推论 1.4. 实对称阵必可正交合同对角化，且合同变换矩阵可取正交矩阵（正交合同是合同的特殊情况，故实对称阵必可普通合同对角化）。

严格证明（从定义出发，分两步）

实对称阵的合同对角化基于其正交相似对角化，而正交相似对角化又基于实对称阵的特征向量正交性，证明分两步：先证实对称阵的不同特征值的特征向量正交，再构造正交矩阵完成对角化。

步骤 1：证明实对称阵不同特征值的特征向量相互正交

设 λ_1, λ_2 为实对称阵 A 的两个不同特征值， x_1, x_2 为对应的实特征向量（由第一部分推论，特征向量可取实向量），则：

$$Ax_1 = \lambda_1 x_1, \quad Ax_2 = \lambda_2 x_2$$

对 $Ax_1 = \lambda_1 x_1$ 两边取转置得：

$$x_1^T A^T = \lambda_1 x_1^T$$

因 $A^T = A$ ，故 $x_1^T A = \lambda_1 x_1^T$ 。

两边右乘 x_2 得：

$$x_1^T A x_2 = \lambda_1 x_1^T x_2$$

代入 $Ax_2 = \lambda_2 x_2$ 得：

$$\lambda_2 x_1^T x_2 = \lambda_1 x_1^T x_2 \implies (\lambda_1 - \lambda_2) x_1^T x_2 = 0$$

因 $\lambda_1 \neq \lambda_2$ ，故 $x_1^T x_2 = 0$ ，即实向量 x_1 与 x_2 正交（实向量内积为 0 即正交）。

步骤 2：构造正交矩阵 Q ，证明 $Q^T A Q = \Lambda$

设实对称阵 A 的 n 个特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$ （重根按重数计），对应 n 个实特征向量：

若特征值为单根：取其单位化的特征向量（ $e_i = x_i / \|x_i\|$ ， $\|x_i\| = \sqrt{x_i^T x_i}$ 为向量的模）；

若特征值为 k 重根：实对称阵的 k 重特征值必有 k 个线性无关的实特征向量，对其做施密特正交化再单位化，得到 k 个正交的单位特征向量。

由步骤 1，不同特征值的特征向量已正交，因此最终可得到 n 个两两正交的单位实特征向量： $e_1, e_2, \dots, e_n$ 。

将这 n 个特征向量按列构成矩阵： $Q = (e_1, e_2, \dots, e_n)$ ，则：

Q 是正交矩阵：因列向量两两正交且单位化，故 $Q^T Q = (e_i^T e_j)_{n \times n} = E$ （ $e_i^T e_j = 1$ 当 $i = j$ ，0 当 $i \neq j$ ）；

由特征值定义， $A(e_1, e_2, \dots, e_n) = (\lambda_1 e_1, \lambda_2 e_2, \dots, \lambda_n e_n)$ ，即：

$$AQ = Q\Lambda$$

其中 $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ 。

因 Q 是正交矩阵, 故 $Q^T = Q^{-1}$, 对 $AQ = Q\Lambda$ 两边左乘 Q^T 得:

$$Q^T AQ = Q^T Q\Lambda = E\Lambda = \Lambda$$

即实对称阵可由正交矩阵合同对角化, 证毕。

1.1.3 从多元泰勒展开到凸函数二阶条件的完整推导 (超详细版)

3.1 核心基础定义 (从底层逻辑出发)

3.1.1 n 元函数与凸集

n 元实值函数: 设 $D \subset \mathbb{R}^n$ 为非空集合, 若对任意 $x = (x_1, x_2, \dots, x_n)^T \in D$, 存在唯一实数 $f(x)$ 与之对应, 则称映射 $f: D \rightarrow \mathbb{R}$ 为 **n 元实值函数**, 记为 $f(x) = f(x_1, x_2, \dots, x_n)$ 。

凸集: 若对任意 $x_1, x_2 \in D$, 以及任意 $\theta \in [0, 1]$, 都有 $\theta x_1 + (1 - \theta)x_2 \in D$, 则称 D 为凸集。直观理解: 集合中任意两点的连线完全包含在集合内 (如 $\mathbb{R}^n$ 、超平面、球体均为凸集)。

3.1.2 一阶偏导数与梯度

一阶偏导数: 设 f 在 $x_0 = (x_{01}, x_{02}, \dots, x_{0n})^T$ 的某邻域内有定义, 若极限

$$\frac{\partial f}{\partial x_i}(x_0) = \lim_{\Delta x_i \rightarrow 0} \frac{f(x_{01}, \dots, x_{0i} + \Delta x_i, \dots, x_{0n}) - f(x_0)}{\Delta x_i}$$

存在, 则称该极限为 f 在 x_0 处对 x_i 的一阶偏导数, 描述函数沿 x_i 方向的变化率。

梯度: 若 f 在 x_0 处所有一阶偏导数均存在, 则梯度为 **n** 维列向量:

$$\nabla f(x_0) = \left(\frac{\partial f}{\partial x_1}(x_0), \frac{\partial f}{\partial x_2}(x_0), \dots, \frac{\partial f}{\partial x_n}(x_0) \right)^T$$

梯度方向是函数在 x_0 处增长最快的方向, 负梯度方向是下降最快的方向。

3.1.3 二阶偏导数与黑塞矩阵 (含对称性证明)

二阶偏导数: 若 $\frac{\partial f}{\partial x_i}$ 在 x_0 处对 x_j 的偏导数存在, 则称

$$\frac{\partial^2 f}{\partial x_j \partial x_i}(x_0) = \lim_{\Delta x_j \rightarrow 0} \frac{\frac{\partial f}{\partial x_i}(x_0 + \Delta x_j e_j) - \frac{\partial f}{\partial x_i}(x_0)}{\Delta x_j}$$

为 f 在 x_0 处的二阶混合偏导数, 其中 e_j 是第 j 个标准基向量 (仅第 j 个分量为 1, 其余为 0)。

黑塞矩阵 (Hessian Matrix): 若 f 在 x_0 处二阶连续可微 (所有二阶偏导数存在且连续), 则黑塞矩阵为 $n \times n$ 方阵:

$$H_f(x_0) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2}(x_0) & \frac{\partial^2 f}{\partial x_1 \partial x_2}(x_0) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(x_0) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(x_0) & \frac{\partial^2 f}{\partial x_2^2}(x_0) & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n}(x_0) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(x_0) & \frac{\partial^2 f}{\partial x_n \partial x_2}(x_0) & \cdots & \frac{\partial^2 f}{\partial x_n^2}(x_0) \end{pmatrix}$$

第 i 行第 j 列元素为 $\frac{\partial^2 f}{\partial x_j \partial x_i}(x_0)$ 。

黑塞矩阵的对称性证明 (克莱罗定理): 若 f 在 x_0 处二阶连续可微, 则混合偏导数与求导顺序无关, 即:

$$\frac{\partial^2 f}{\partial x_j \partial x_i}(x_0) = \frac{\partial^2 f}{\partial x_i \partial x_j}(x_0)$$

因此 $H_f(x_0)^T = H_f(x_0)$, 即黑塞矩阵是实对称矩阵。

备注

注：只有二阶偏导数连续时，黑塞矩阵才一定对称；若混合偏导不连续，对称性可能失效。

3.2 多元函数泰勒展开：从线性到高阶近似

泰勒展开的核心思想：用多项式函数近似复杂函数，阶数越高，近似精度越高，是连接「函数局部变化」与「导数信息」的桥梁。

3.2.1 一阶泰勒展开（线性近似）

前提： f 在 x_0 处可微（可微 $\Rightarrow$ 一阶偏导数存在且连续）。

佩亚诺余项形式（局部分析用）：

$$f(x) = f(x_0) + \nabla f(x_0)^T(x - x_0) + o(\|x - x_0\|)$$

含义：在 x_0 附近， $f(x)$ 可近似为「函数值 + 梯度方向的线性变化」，误差 $o(\|x - x_0\|)$ 是 $\|x - x_0\|$ 的高阶无穷小（当 $x \rightarrow x_0$ 时，误差比 $\|x - x_0\|$ 更快趋近于 0）。

拉格朗日余项形式（全局分析用）：

$$f(x) = f(x_0) + \nabla f(\xi)^T(x - x_0)$$

其中 ξ 是 x_0 与 x 连线上的点（因 D 是凸集，故 $\xi \in D$ ），描述了全局范围内的线性近似误差。

3.2.2 二阶泰勒展开（核心，二次近似）

前提： f 在 x_0 处二阶连续可微。

佩亚诺余项形式（局部分析用）：

$$f(x) = f(x_0) + \nabla f(x_0)^T(x - x_0) + \frac{1}{2}(x - x_0)^T H_f(x_0)(x - x_0) + o(\|x - x_0\|^2)$$

含义：在一阶近似基础上，加入二次型项 $\frac{1}{2}(x - x_0)^T H_f(x_0)(x - x_0)$ ，描述函数在 x_0 处的「弯曲程度」，误差 $o(\|x - x_0\|^2)$ 是 $\|x - x_0\|^2$ 的高阶无穷小。

拉格朗日余项形式（全局证明用，核心）：

$$f(x) = f(x_0) + \nabla f(x_0)^T(x - x_0) + \frac{1}{2}(x - x_0)^T H_f(\xi)(x - x_0)$$

其中 ξ 是 x_0 与 x 连线上的点（凸集保证 $\xi \in D$ ），将局部的二阶导数信息扩展到全局。

泰勒展开的作用：将 $f(x) - f(x_0)$ 拆解为「线性变化项 + 二次弯曲项」，把抽象的函数差转化为可代数分析的形式，为后续利用黑塞矩阵正定性推导凸性奠定基础。

3.2.3 k 阶泰勒展开（高阶近似）

前提： f 在 x_0 处 k 阶连续可微。

$$f(x) = \sum_{m=0}^k \frac{1}{m!} D^m f(x_0)(x - x_0)^m + o(\|x - x_0\|^k)$$

其中：

- $D^0 f(x_0) = f(x_0)$ （0 阶导数，函数本身）
- $D^1 f(x_0)(v) = \nabla f(x_0)^T v$ （一阶导数，线性形式）
- $D^2 f(x_0)(v) = v^T H_f(x_0)v$ （二阶导数，二次型形式）
- $D^m f(x_0)(v)$ 是 m 阶多线性形式，对应 m 阶偏导数的张量乘积，描述更高阶的函数变化。

3.3 黑塞矩阵的谱分解：二次型的特征值视角

因 $H_f(x)$ 是实对称矩阵，可利用实对称矩阵的正交对角化性质，将二次型转化为特征值加权的平方和。

3.3.1 实对称矩阵的正交对角化定理

定理 1.5. 设 $A \in \mathbb{R}^{n \times n}$ 为实对称矩阵，则存在正交矩阵 Q （满足 $Q^T Q = I$ ，即 $Q^T = Q^{-1}$ ），使得：

$$Q^T A Q = \Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$$

其中 $\lambda_1, \lambda_2, \dots, \lambda_n$ 是 A 的特征值（全为实数）， Q 的列是 A 对应特征值的单位正交特征向量。

3.3.2 二阶泰勒二次型的谱分解

设 $\Delta x = x - x_0$ ，二阶泰勒展开中的二次型项为：

$$\Delta x^T H_f(x_0) \Delta x$$

将 $H_f(x_0) = Q \Lambda Q^T$ （由正交对角化定理， $H = Q \Lambda Q^T$ ）代入：

$$\Delta x^T H_f(x_0) \Delta x = \Delta x^T (Q \Lambda Q^T) \Delta x = (Q^T \Delta x)^T \Lambda (Q^T \Delta x)$$

令 $z = Q^T \Delta x$ （正交变换不改变向量长度，故 $\|z\| = \|\Delta x\|$ ），则：

$$\Delta x^T H \Delta x = z^T \Lambda z = \sum_{i=1}^n \lambda_i z_i^2$$

几何意义：二次型的值等于各特征值乘以对应方向分量的平方和：

- 若所有 $\lambda_i \geq 0$ ：二次型 ≥ 0 （半正定），函数在所有方向上「向上弯曲」或平坦；
- 若所有 $\lambda_i > 0$ ：二次型 > 0 （正定），函数在所有方向上严格「向上弯曲」；
- 若存在 $\lambda_i < 0$ ：二次型可正可负（不定），函数在某些方向「向下弯曲」。

3.4 凸函数的定义与一阶等价条件（证明铺垫）

3.4.1 凸函数的原始定义

设 $D \subset \mathbb{R}^n$ 为凸集，若对任意 $x, y \in D$ ，以及任意 $\theta \in [0, 1]$ ，都有：

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

则称 f 是 D 上的凸函数；若不等号严格成立（ $\theta \in (0, 1)$ 且 $x \neq y$ ），则称 f 为严格凸函数。

3.4.2 可微凸函数的一阶等价条件

定理 1.6. 设 D 为开凸集， $f: D \rightarrow \mathbb{R}$ 可微，则 f 是凸函数当且仅当：

$$\forall x, y \in D, \quad f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

直观理解：函数图像永远在其任意一点的切平面上方，切平面是函数的「全局下界」。

作用：将凸性的原始定义（两点加权平均）转化为更易分析的「函数差与梯度线性项的不等式」，是二阶条件证明的核心桥梁。

3.5 凸函数二阶条件的完整证明（核心：突出泰勒展开的作用）

定理 1.7. 设 $D \subset \mathbb{R}^n$ 为开凸集, $f: D \rightarrow \mathbb{R}$ 在 D 上二阶连续可微, 则:

$$f \text{ 是凸函数} \iff \forall x \in D, H_f(x) \succeq 0 \text{ (黑塞矩阵半正定)}$$

且 $\forall x \in D, H_f(x) \succ 0 \implies f$ 是严格凸函数。

3.5.1 充分性证明: $\forall x \in D, H_f(x) \succeq 0 \implies f$ 是凸函数

目标: 证明满足可微凸函数的一阶等价条件 $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ 。

步骤 1: 应用二阶泰勒展开 (拉格朗日余项)

任取 $x, y \in D$, 对 $f(y)$ 在 x 处做二阶泰勒展开 (拉格朗日余项形式):

$$f(y) = f(x) + \nabla f(x)^T(y - x) + \frac{1}{2}(y - x)^T H_f(\xi)(y - x)$$

其中 ξ 是 x 与 y 连线上的点 (凸集保证 $\xi \in D$)。

泰勒展开的作用: 将 $f(y) - f(x)$ 拆解为「梯度线性项 + 黑塞二次型项」, 把抽象的凸性不等式转化为可直接分析的代数形式, 为后续利用黑塞矩阵半正定性做准备。

步骤 2: 利用黑塞矩阵半正定条件

由题设 $H_f(\xi) \succeq 0$ (半正定), 根据半正定矩阵定义: 对任意非零向量 $v \in \mathbb{R}^n$, 有 $v^T H_f(\xi) v \geq 0$ 。令 $v = y - x$, 则:

$$(y - x)^T H_f(\xi)(y - x) \geq 0$$

步骤 3: 推导凸函数一阶等价条件

将二次型项的非负性代入泰勒展开式:

$$f(y) = f(x) + \nabla f(x)^T(y - x) + \underbrace{\frac{1}{2}(y - x)^T H_f(\xi)(y - x)}_{\geq 0}$$

因此:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

满足可微凸函数的一阶等价条件, 故 f 是凸函数。

3.5.2 必要性证明: f 是凸函数 $\implies \forall x \in D, H_f(x) \succeq 0$

目标: 证明对任意 $x_0 \in D$, 任意非零向量 $v \in \mathbb{R}^n$, 有 $v^T H_f(x_0) v \geq 0$ 。

步骤 1: 构造一元函数并利用凸性

任取 $x_0 \in D$, 任取非零向量 $v \in \mathbb{R}^n$, 构造一元函数:

$$g(t) = f(x_0 + tv), \quad t \in (-\delta, \delta)$$

其中 $\delta > 0$ 足够小, 使得 $x_0 + tv \in D$ (开凸集保证存在这样的 δ)。因 f 是凸函数, 故 $g(t)$ 是一元凸函数 (凸函数与仿射变换的复合仍为凸函数)。

步骤 2: 对 $g(t)$ 做二阶泰勒展开 (拉格朗日余项)

对 $g(t)$ 在 $t = 0$ 处做二阶泰勒展开:

$$g(t) = g(0) + g'(0)t + \frac{1}{2}g''(\xi)t^2$$

其中 $\xi \in (0, t)$ 或 $(t, 0)$ ($t \neq 0$)。由复合函数求导法则:

- 一阶导数: $g'(t) = \nabla f(x_0 + tv)^T v \implies g'(0) = \nabla f(x_0)^T v$

• 二阶导数: $g''(t) = v^T H_f(x_0 + tv)v \implies g''(\xi) = v^T H_f(x_0 + \xi v)v$

泰勒展开的作用: 将一元函数 $g(t)$ 的变化分解为「线性项 + 二次项」, 通过凸性的一阶条件约束二次项符号, 进而将 n 元函数的凸性转化为黑塞矩阵的局部正定性。

步骤 3: 利用一元凸函数的一阶条件

一元凸函数的一阶等价条件为: 对任意 $t \in (-\delta, \delta)$,

$$g(t) \geq g(0) + g'(0)t$$

将二阶泰勒展开代入得:

$$g(0) + g'(0)t + \frac{1}{2}g''(\xi)t^2 \geq g(0) + g'(0)t$$

两边消去 $g(0)$ 和 $g'(0)t$, 得:

$$\frac{1}{2}g''(\xi)t^2 \geq 0$$

因 $t \neq 0$ 时 $t^2 > 0$, 故 $g''(\xi) \geq 0$, 即:

$$v^T H_f(x_0 + \xi v)v \geq 0$$

步骤 4: 取极限得到黑塞矩阵半正定

令 $t \rightarrow 0$, 则 $\xi \rightarrow 0$, 由 f 二阶连续可微, $H_f(x_0 + \xi v) \rightarrow H_f(x_0)$, 故:

$$v^T H_f(x_0)v \geq 0$$

对任意非零 v 成立, 因此 $H_f(x_0) \succeq 0$ 。

3.5.3 严格凸函数的推论

若 $\forall x \in D, H_f(x) \succ 0$ (正定), 则在充分性证明中, 当 $y \neq x$ 时, 二次型项 $(y-x)^T H_f(\xi)(y-x) > 0$, 故:

$$f(y) > f(x) + \nabla f(x)^T (y - x)$$

满足严格凸函数的一阶等价条件, 因此 f 是严格凸函数。

3.6 泰勒展开在证明中的核心作用总结

- **分解函数变化:** 将 $f(y) - f(x)$ 拆解为「梯度线性项 + 黑塞二次型项」, 把抽象的凸性不等式转化为可代数分析的形式, 是连接「函数全局凸性」与「局部导数信息」的关键桥梁。
- **扩展局部信息到全局:** 拉格朗日余项形式将局部的二阶导数 (黑塞矩阵) 信息扩展到全局, 使得我们可以用局部点 ξ 的黑塞正定性推导全局凸性。
- **直观解释凸性几何:** 二次型项直接对应函数的「弯曲方向」, 通过谱分解将二次型与特征值联系起来, 直观说明「黑塞半正定 $\Leftrightarrow$ 函数向上弯 $\Leftrightarrow$ 凸函数」的逻辑。

1.2 凸的礼物

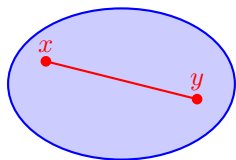
凸性是优化理论中最核心的概念之一, 它为优化问题提供了强大的理论保证。本节将系统介绍凸集、凸函数的定义及其判定条件, 并深入探讨凸优化问题的核心性质。

1.2.1 相关定义

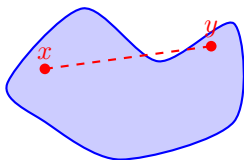
定义

定义 1.8 (凸集). 集合 $C \subseteq \mathbb{R}^n$ 是**凸集**, 当且仅当: 对任意 $x, y \in C$ 和任意 $\theta \in [0, 1]$, 有 $\theta x + (1 - \theta)y \in C$ 。

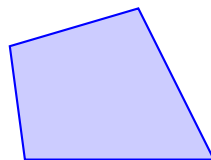
直觉: 集合内任意两点的连线段都完全在集合内。



凸集 ✓



非凸集 ×



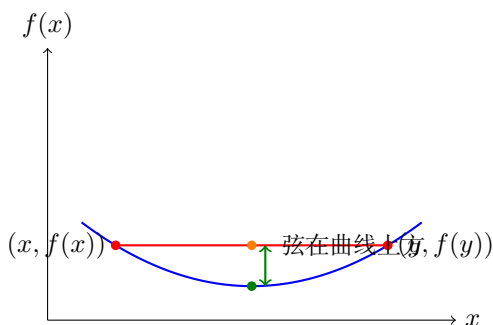
多面体 (凸)

定义

定义 1.9 (凸函数). 函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 是**凸函数**, 当且仅当: 对任意 x, y 和任意 $\theta \in [0, 1]$:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

直觉: 函数图像上任意两点连线在函数图像的上方。

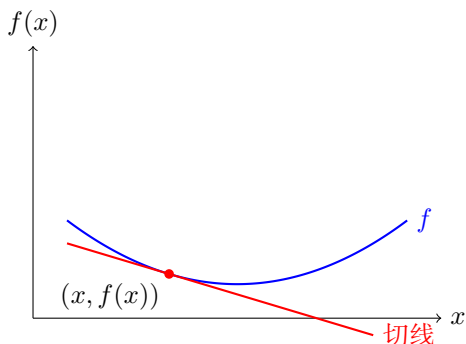


1.2.2 凸性判定的条件

定理 1.10 (凸性的一阶条件). 设 f 可微. f 是凸函数当且仅当对所有 x, y :

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

几何意义: 函数图像始终在其切平面的上方。



推论 1.11. 如果 $\nabla f(x^*) = 0$, 则 x^* 是全局最小值。

分析

一阶条件的证明:

证明 ($\Rightarrow$): 设 f 凸, 要证 $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ 。

由凸性定义, 对 $\theta \in (0, 1)$:

$$f(x + \theta(y - x)) \leq (1 - \theta)f(x) + \theta f(y)$$

移项:

$$f(x + \theta(y - x)) - f(x) \leq \theta(f(y) - f(x))$$

除以 θ :

$$\frac{f(x + \theta(y - x)) - f(x)}{\theta} \leq f(y) - f(x)$$

令 $\theta \rightarrow 0^+$, 左边趋于方向导数 $\nabla f(x)^T(y - x)$:

$$\nabla f(x)^T(y - x) \leq f(y) - f(x)$$

即 $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ 。

定理 1.12 (凸性的二阶条件). 设 f 二次可微. f 是凸函数当且仅当对所有 x :

$$\nabla^2 f(x) \succeq 0$$

即 *Hessian* 矩阵处处半正定。

回忆: 矩阵 H 半正定 $\Leftrightarrow$ 所有特征值 $\geq 0 \Leftrightarrow v^T H v \geq 0$ 对所有 v 。

直觉: 二阶导数处处非负 $\Rightarrow$ 函数「向上弯曲」。

分析

二阶条件的证明:

证明 ($\Rightarrow$): 设 f 凸, 要证 $\nabla^2 f(x) \succeq 0$ 。

由一阶条件, 对任意 y :

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

同时, 交换 x, y 的角色:

$$f(x) \geq f(y) + \nabla f(y)^T(x - y)$$

两式相加:

$$0 \geq (\nabla f(x) - \nabla f(y))^T(y - x) = -(\nabla f(y) - \nabla f(x))^T(y - x)$$

即 $(\nabla f(y) - \nabla f(x))^T(y - x) \geq 0$ (梯度是单调的)。

设 $y = x + tv$, $t > 0$:

$$(\nabla f(x + tv) - \nabla f(x))^T v \geq 0$$

除以 t , 令 $t \rightarrow 0$:

$$v^T \nabla^2 f(x) v \geq 0$$

对所有 v 成立, 故 $\nabla^2 f(x) \succeq 0$ 。

1.2.3 强凸性

定义

定义 1.13 (强凸函数). 函数 f 是 m -强凸的, 如果对所有 x :

$$\nabla^2 f(x) \succeq mI, \quad m > 0$$

等价定义: $f(x) - \frac{m}{2}\|x\|^2$ 是凸函数。

备注

强凸性的性质:

- 强凸函数有唯一的全局最小值
- 梯度下降收敛更快 (线性收敛 vs 次线性收敛)

1.2.4 凸优化的核心定理

定理 1.14 (局部最优 = 全局最优). 对于凸优化问题:

$$\min_x f(x) \quad \text{s.t.} \quad x \in C$$

其中 f 是凸函数, C 是凸集。

如果 x^* 是局部最优解, 则 x^* 是全局最优解。

证明. 反证法. 假设存在 $y \in C$ 使得 $f(y) < f(x^*)$ 。

由凸性, 对任意 $\theta \in (0, 1)$:

$$f(\theta y + (1 - \theta)x^*) \leq \theta f(y) + (1 - \theta)f(x^*) < f(x^*)$$

当 $\theta \rightarrow 0$ 时, $\theta y + (1 - \theta)x^* \rightarrow x^*$, 矛盾于 x^* 是局部最优。 □

1.3 梯度下降的收敛性

本节深入分析梯度下降算法的理论收敛性质, 证明其在凸优化问题下的收敛率, 并探讨强凸性带来的加速效果。

1.3.1 基本假设与记号

设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 是可微函数, $x^* = \arg \min_x f(x)$ 是全局最优解, $f^* = f(x^*)$ 。

梯度下降迭代 (步长 $\alpha > 0$):

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

本节使用以下两个核心假设:

1. **凸性**: f 是凸函数, 即对所有 x, y :

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

2. **L -光滑性** (梯度 Lipschitz 连续): 存在常数 $L > 0$, 使得对所有 x, y :

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2$$

等价地, 对所有 x, y :

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|^2 \quad (\text{下降引理})$$

备注

直觉: L -光滑意味着函数不会“弯曲得太快”, 梯度信息在一步之内仍然可靠。

1.3.2 关键引理

引理 1.15 (充分下降引理). 设 f 是 L -光滑函数, 步长 $\alpha = 1/L$, 则梯度下降每步保证:

$$f(x_{t+1}) \leq f(x_t) - \frac{1}{2L}\|\nabla f(x_t)\|^2$$

证明. 将 $y = x_{t+1} = x_t - \frac{1}{L}\nabla f(x_t)$ 代入下降引理:

$$f(x_{t+1}) \leq f(x_t) + \nabla f(x_t)^T\left(-\frac{1}{L}\nabla f(x_t)\right) + \frac{L}{2}\left\|\frac{1}{L}\nabla f(x_t)\right\|^2 = f(x_t) - \frac{1}{2L}\|\nabla f(x_t)\|^2$$

□

推论 1.16. $f(x_t)$ 单调不减, 即梯度下降在凸 L -光滑函数上永远不会变差。

1.3.3 主定理: $O(1/T)$ 收敛率

定理 1.17 (凸函数梯度下降收敛率). 设 f 是凸函数且 L -光滑, 步长 $\alpha = 1/L$. 则对任意 $T \geq 1$:

$$f(x_T) - f^* \leq \frac{L\|x_0 - x^*\|^2}{2T}$$

证明. **第一步:** 建立距离递推关系. 令 $r_t = \|x_t - x^*\|^2$, 展开:

$$r_{t+1} = \|x_t - \frac{1}{L}\nabla f(x_t) - x^*\|^2 = r_t - \frac{2}{L}\nabla f(x_t)^T(x_t - x^*) + \frac{1}{L^2}\|\nabla f(x_t)\|^2$$

第二步: 利用凸性下界. 由凸性一阶条件 (取 $y = x^*$):

$$f(x_t) - f^* \leq \nabla f(x_t)^T(x_t - x^*)$$

令 $\delta_t = f(x_t) - f^*$, 代入第一步:

$$r_{t+1} \leq r_t - \frac{2}{L}\delta_t + \frac{1}{L^2}\|\nabla f(x_t)\|^2$$

第三步: 利用充分下降引理消去梯度范数. 由引理知 $\|\nabla f(x_t)\|^2 \leq 2L(\delta_t - \delta_{t+1})$, 故:

$$r_{t+1} \leq r_t - \frac{2}{L}\delta_t + \frac{2}{L}(\delta_t - \delta_{t+1}) = r_t - \frac{2}{L}\delta_{t+1}$$

整理得：

$$\frac{2}{L}\delta_{t+1} \leq r_t - r_{t+1}$$

第四步：对 $t = 0, 1, \dots, T-1$ 求和（望远镜求和）：

$$\frac{2}{L} \sum_{t=1}^T \delta_t \leq r_0 - r_T \leq r_0 = \|x_0 - x^*\|^2$$

第五步：由充分下降引理知 δ_t 单调不增，故 $T \cdot \delta_T \leq \sum_{t=1}^T \delta_t$ ，因此：

$$\frac{2T}{L} \delta_T \leq \|x_0 - x^*\|^2 \implies f(x_T) - f^* \leq \frac{L \|x_0 - x^*\|^2}{2T}$$

□

备注

[$O(1/T)$ 收敛率的含义]

- 要使误差 $\leq \varepsilon$ ，需要迭代次数 $T = O(L\|x_0 - x^*\|^2/\varepsilon)$ ，即 $O(1/\varepsilon)$ 步。
- 每步计算量为 $O(n)$ （只算一次梯度），总计算量 $O(n/\varepsilon)$ 。
- 这被称为**次线性收敛**（sublinear convergence）：误差以 $1/T$ 的速度衰减。
- 此速率对一般凸函数是**最优的**（信息论下界），即无法在相同假设下做得更好。

1.3.4 强凸函数的线性收敛率

若进一步假设 f 是 μ -强凸 ($\mu > 0$)，即对所有 x, y ：

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2}\|y - x\|^2$$

定理 1.18 (强凸函数线性收敛率). 设 f 是 μ -强凸且 L -光滑，步长 $\alpha = 1/L$ ，则：

$$\|x_T - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2$$

证明. 由强凸性（令 $y = x^*$ ）：

$$f(x_t) - f^* \geq \frac{\mu}{2}\|x_t - x^*\|^2 = \frac{\mu}{2}r_t$$

从第一步和强凸性下界出发：

$$r_{t+1} \leq r_t - \frac{2}{L}\delta_t + \frac{1}{L^2}\|\nabla f(x_t)\|^2$$

更简洁地，利用强凸性的等价形式（强单调性）：

首先展开距离的平方：

$$r_{t+1} = \left\|x_t - \frac{1}{L}\nabla f(x_t) - x^*\right\|^2 = r_t - \frac{2}{L}\nabla f(x_t)^T(x_t - x^*) + \frac{1}{L^2}\|\nabla f(x_t)\|^2$$

利用强凸性的下界（强单调性）：

$$\nabla f(x_t)^T(x_t - x^*) \geq \frac{\mu}{2}r_t + \frac{1}{2L} \|\nabla f(x_t)\|^2$$

代入得：

$$r_{t+1} \leq r_t - \frac{2}{L} \left(\frac{\mu}{2}r_t + \frac{1}{2L} \|\nabla f(x_t)\|^2 \right) + \frac{1}{L^2} \|\nabla f(x_t)\|^2$$

简化：

$$r_{t+1} \leq r_t - \frac{\mu}{L}r_t = \left(1 - \frac{\mu}{L}\right) r_t$$

递归展开得：

$$\|x_T - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2$$

□

备注

[线性收敛率的含义]

- 定义**条件数** $\kappa = L/\mu \geq 1$ ，收缩因子为 $1 - 1/\kappa$ 。
- 要使 $\|x_T - x^*\|^2 \leq \varepsilon$ ，需要 $T = O(\kappa \log(1/\varepsilon))$ 步，即 $O(\log(1/\varepsilon))$ 步——远优于 $O(1/\varepsilon)$ ！
- 这是**线性收敛**（linear convergence）：误差的对数随迭代线性下降，即“每次迭代误差减少一个固定百分比”。
- κ 越大（病态程度越高），收敛越慢； $\kappa = 1$ 时（即 $\mu = L$ ）一步到达最优。

假设	收敛率	步数达到精度 ε	名称
凸 + L -光滑	$O(1/T)$	$O(1/\varepsilon)$	次线性收敛
强凸 + L -光滑	$O((1 - \mu/L)^T)$	$O(\kappa \log 1/\varepsilon)$	线性收敛

1.3.5 具体例子： $f(x) = kx^2$ 的梯度下降

Example

取 $f(x) = kx^2$ ， $k > 0$ ，最优解显然是 $x^* = 0$ ， $f^* = 0$ 。

第一步：确定 L 和 μ

$f''(x) = 2k$ （常数），故 f 同时是 $L = 2k$ -光滑的和 $\mu = 2k$ -强凸的。条件数 $\kappa = L/\mu = 1$ ，这是最优情况。

第二步：写出迭代格式

第二步：写出迭代格式

$\nabla f(x) = 2kx$ ，梯度下降：

$$x_{t+1} = x_t - \alpha \cdot 2kx_t = (1 - 2k\alpha) x_t$$

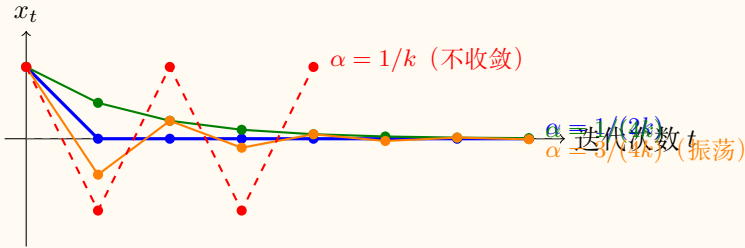
记收缩因子 $\rho = 1 - 2k\alpha$ ，递归展开得**闭合公式**：

$$x_t = \rho^t x_0 = (1 - 2k\alpha)^t x_0$$

第三步：分析不同步长的行为

步长 α	$\rho = 1 - 2k\alpha$	$ \rho $	行为
$\frac{1}{2k}$ ($= 1/L$, 最优)	0	0	一步收敛! $x_1 = 0$
$\frac{1}{4k}$	$\frac{1}{2}$	0.5	单调收敛, 每步误差减半
$\frac{3}{4k}$	$-\frac{1}{2}$	0.5	振荡收敛 (正负交替, 幅度减半)
$\frac{1}{k}$ ($= 2/L$, 临界)	-1	1	永远振荡, 不收敛
$> \frac{1}{k}$	$ \rho > 1$	> 1	发散 (步长太大)

直觉图 ($k = 1, x_0 = 1$, 不同步长的轨迹):



第四步：与一般理论对照

- 最优步长 $\alpha = 1/L = 1/(2k)$, 此时 $\rho = 0$ —— $\kappa = 1$ 的情形理论预测 $(1 - \mu/L)^T = 0^T = 0$, 即一步精确到达, 与计算一致。
- 步长约束 $0 < \alpha < 1/k = 2/L$ 与充分下降引理的要求 $\alpha \leq 1/L$ 对应:
 - $\alpha \leq 1/L$ 保证单调下降
 - $1/L < \alpha < 2/L$ 振荡但仍收敛
 - $\alpha \geq 2/L$ 发散
- 误差公式 $|x_t - x^*| = |\rho|^t |x_0|$ 正是线性收敛 $O((1 - \mu/L)^T)$ 的具体体现。

1.3.6 为什么凸性保证梯度下降找到最优解？

备注

[核心逻辑链]

1. 凸性 $\Rightarrow$ 局部 = 全局：不存在”陷阱”式局部最优。
 2. L -光滑 $\Rightarrow$ 每步必下降（充分下降引理）：梯度下降永远在改善。
 3. 两者合并 $\Rightarrow$ 收敛到全局最优：既然每步都在改善，且改善终点就是全局最优，梯度下降必然收敛至全局最优。
- 对比非凸情形：满足下降引理，但可能收敛到局部最优——有下降保证，却无全局保证。

1.4 具体实现过程

本节通过房价预测实例，完整展示梯度下降算法的实现过程，包括数据生成、基础梯度下降、学习率影响分析、Adam 优化器等核心内容。

1.4.1 模拟数据生成

我们生成 100 套房屋数据，真实规律为：

房价 = $2.5 \times \text{面积(平米)} + 30$ （等价于 $250 \times \text{面积(百平米)} + 30$ ）

为稳定梯度下降，将面积单位从平米转换为百平米（范围 0.9-1.5）。

Python 代码实现

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 解决中文显示问题
5 plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu Sans']
6 plt.rcParams['axes.unicode_minus'] = False
7
8 # 固定随机种子，保证结果可复现
9 np.random.seed(42)
10 n_samples = 100
11
12 # 生成随机面积（90-150 平米），转换为百平米
13 areas_m2 = np.random.uniform(90, 150, n_samples)
14 areas = areas_m2 / 100
15
16 # 生成房价数据（加入随机噪声模拟真实场景）
17 prices = 250 * areas + 30 + np.random.randn(n_samples) * 15
18
19 # 打印数据基本信息
20 print(f"房屋数量: {len(areas)} 套")
21 print(f"面积范围: {areas_m2.min():.1f} - {areas_m2.max():.1f} 平米")
22 print(f"价格范围: {prices.min():.1f} - {prices.max():.1f} 万")
```

备注

[运行结果]

房屋数量: 100 套

面积范围: 90.3 - 149.2 平米 (0.90 - 1.49 百平米)

价格范围: 248.0 - 414.8 万

真实规律: 房价 = $2.5 \times \text{面积(平米)} + 30$

1.4.2 基础梯度下降算法实现

基于均方误差（MSE）损失函数，实现梯度下降来拟合线性回归参数 w 和 b 。

Python 代码实现

```
1 def gradient_descent(X, y, lr=0.1, epochs=200):
2     # 初始化参数 w (权重) 和 b (偏置)
3     w, b = 0.0, 0.0
4     n = len(X) # 样本数量
5
6     # 初始化历史记录: 保存每一轮的 w、b 和损失值
7     history = {'w': [w], 'b': [b],
8               'loss': [np.mean((w*X + b - y)**2)]}
```

```

9
10 # 迭代训练
11 for epoch in range(epochs):
12     # 1. 计算当前模型预测值
13     y_pred = w * X + b
14
15     # 2. 计算损失函数对 w 和 b 的梯度
16     dw = (2/n) * np.sum((y_pred - y) * X) # 对 w 的偏导
17     db = (2/n) * np.sum(y_pred - y) # 对 b 的偏导
18
19     # 3. 沿梯度反方向更新参数
20     w -= lr * dw
21     b -= lr * db
22
23     # 4. 记录当前轮的参数和损失
24     history['w'].append(w)
25     history['b'].append(b)
26     history['loss'].append(np.mean((y_pred - y)**2))
27
28     # 5. 每隔 1% 训练轮数，打印训练进度
29     if epoch % (epochs/100) == 0:
30         print(f"迭代 {epoch:3d}: w={w:.2f}, b={b:.2f}, "
31               f"loss={history['loss'][-1]:.2f}")
32
33     return history

```

1.4.3 基础模型训练与结果查看

调用梯度下降函数进行训练，并输出最终拟合结果。

Python 代码实现

```

1 # 执行梯度下降训练
2 history = gradient_descent(areas, prices, lr=0.01, epochs=100000)
3
4 # 输出最终拟合结果 (将 w 转换回平米单位)
5 print(f"\n最终结果：房价 = {history['w'][-1]/100:.3f} "
6       f"x 面积(平米) + {history['b'][-1]:.2f}")

```

备注

[训练过程输出（部分）]

```

迭代 0: w=7.85, b=6.51, loss=107933.06
迭代 1000: w=179.58, b=114.15, loss=291.90
迭代 5000: w=217.77, b=68.40, loss=195.15
迭代 10000: w=232.89, b=50.30, loss=182.49
迭代 20000: w=238.09, b=44.07, loss=181.49
迭代 50000: w=238.51, b=43.57, loss=181.48

```

最终结果：房价 = 2.385 × 面积(平米) + 43.57

训练过程可视化

Python 代码实现

```

1  # 可视化训练过程
2  fig, axes = plt.subplots(1, 3, figsize=(15, 5))
3
4  # 左图：拟合效果
5  axes[0].scatter(areas_m2, prices, alpha=0.6, s=60,
6                  edgecolors='black', linewidth=0.5, label='房屋数据')
7  x_fit = np.linspace(areas_m2.min(), areas_m2.max(), 100)
8  y_fit = history['w'][-1]/100 * x_fit + history['b'][-1]
9  axes[0].plot(x_fit, y_fit, 'r-', linewidth=2.5,
10              label=f'拟合结果: y = {history["w"][-1]/100:.2f}x + {history["b"][-1]:.2f}')
11 axes[0].set_xlabel('面积 (平米)', fontsize=12)
12 axes[0].set_ylabel('房价 (万)', fontsize=12)
13 axes[0].set_title('房价预测拟合效果', fontsize=13, fontweight='bold')
14 axes[0].legend(fontsize=10)
15 axes[0].grid(True, alpha=0.3)
16
17 # 中图：参数收敛过程
18 axes[1].plot(history['w'][100:], 'b-', linewidth=2, label='w (权重)')
19 axes[1].plot(history['b'][100:], 'g-', linewidth=2, label='b (偏置)')
20 axes[1].axhline(y=250, color='r', linestyle='--', alpha=0.5, label='真实 w=250')
21 axes[1].axhline(y=30, color='orange', linestyle='--', alpha=0.5, label='真实 b=30')
22 axes[1].set_xlabel('迭代次数', fontsize=12)
23 axes[1].set_ylabel('参数值', fontsize=12)
24 axes[1].set_title('参数收敛过程', fontsize=13, fontweight='bold')
25 axes[1].legend(fontsize=10)
26 axes[1].grid(True, alpha=0.3)
27
28 # 右图：损失函数下降过程
29 axes[2].plot(history['loss'], 'purple', linewidth=2)
30 axes[2].set_xlabel('迭代次数', fontsize=12)
31 axes[2].set_ylabel('MSE 损失', fontsize=12)
32 axes[2].set_title('损失函数下降过程', fontsize=13, fontweight='bold')
33 axes[2].grid(True, alpha=0.3)
34 axes[2].set_yscale('log') # 使用对数坐标更清晰
35
36 plt.tight_layout()
37 plt.show()

```

备注

[可视化说明] 生成了三幅图来展示训练过程：

1. **拟合效果图**：展示模型对房价数据的拟合效果，红线为预测结果
2. **参数收敛图**：展示权重 w 和偏置 b 随迭代次数的收敛过程，虚线为真实值
3. **损失下降图**：展示 MSE 损失随迭代次数的下降过程，使用对数坐标

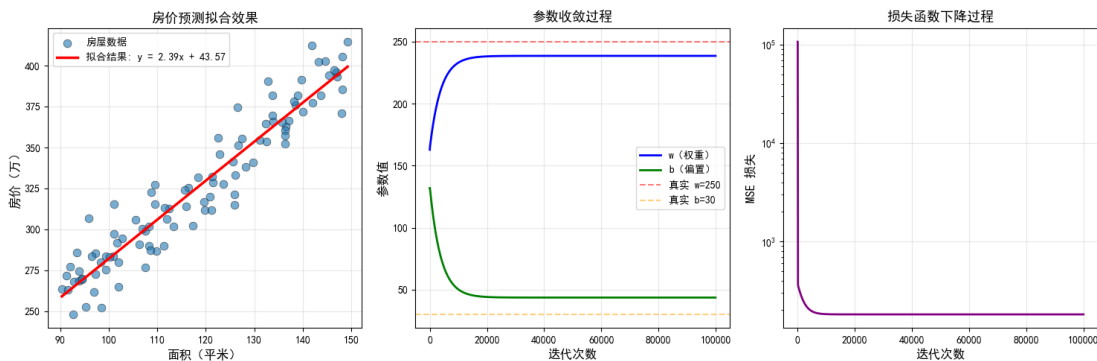


图 1: 基础梯度下降训练过程可视化

1.4.4 核心原理回顾

1. 损失函数 (MSE)

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (wx_i + b - y_i)^2$$

训练目标是最小化 $L(w, b)$ 。

2. 梯度计算

对 w 求偏导:

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_{i=1}^n (wx_i + b - y_i) \cdot x_i$$

对 b 求偏导:

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_{i=1}^n (wx_i + b - y_i)$$

3. 参数更新公式

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$

$$b \leftarrow b - \eta \cdot \frac{\partial L}{\partial b}$$

其中 η 为学习率, 控制参数更新的步长。

1.4.5 学习率对梯度下降的影响

学习率核心作用

学习率直接决定参数更新步长, 是梯度下降的关键超参数:

- 太小: 收敛极慢, 蜗牛爬坡
- 合适: 快速稳定收敛
- 偏大: 剧烈震荡, 无法收敛
- 过大: 梯度爆炸, 参数发散

Python 代码实现

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 核心逻辑
5 def gradient_descent(start_x, lr, n_steps=15):
6     """在  $f(x)=x^2$  上做梯度下降, 返回每一步的  $x$  值"""
7     trajectory = [start_x]
8     x = start_x
9     for _ in range(n_steps):
10         grad = 2 * x #  $f'(x) = 2x$ 
11         x = x - lr * grad # 参数更新
12         trajectory.append(x)
13         if abs(x) > 500: # 已经爆炸, 提前停
14             break
15     return trajectory
16
17 # 四种学习率对比
18 x0 = 4.0
19 configs = [
20     (0.05, '太小: 蜗牛爬坡', 'steelblue'),
21     (0.3, '合适: 稳步收敛', 'seagreen'),
22     (0.9, '偏大: 剧烈震荡', 'orange'),
23     (1.1, '过大: 梯度爆炸', 'crimson'),
24 ]
25
26 # 参数值随迭代变化图
27 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
28 for ax, (lr, title, color) in zip(axes.flatten(), configs):
29     traj = gradient_descent(x0, lr)
30     ax.plot(range(len(traj)), traj, 'o-', color=color, markersize=5)
31     ax.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
32     ax.set_title(f' $lr={lr}$  — {title}', fontsize=12)
33     ax.set_xlabel('迭代次数')
34     ax.set_ylabel('参数  $x$ ')
35     ax.grid(True, alpha=0.3)
36
37 plt.suptitle('学习率对梯度下降的影响 (参数变化)', fontsize=14)
38 plt.tight_layout()
39 plt.show()

```

可视化结果展示

学习率对梯度下降的影响（参数变化）

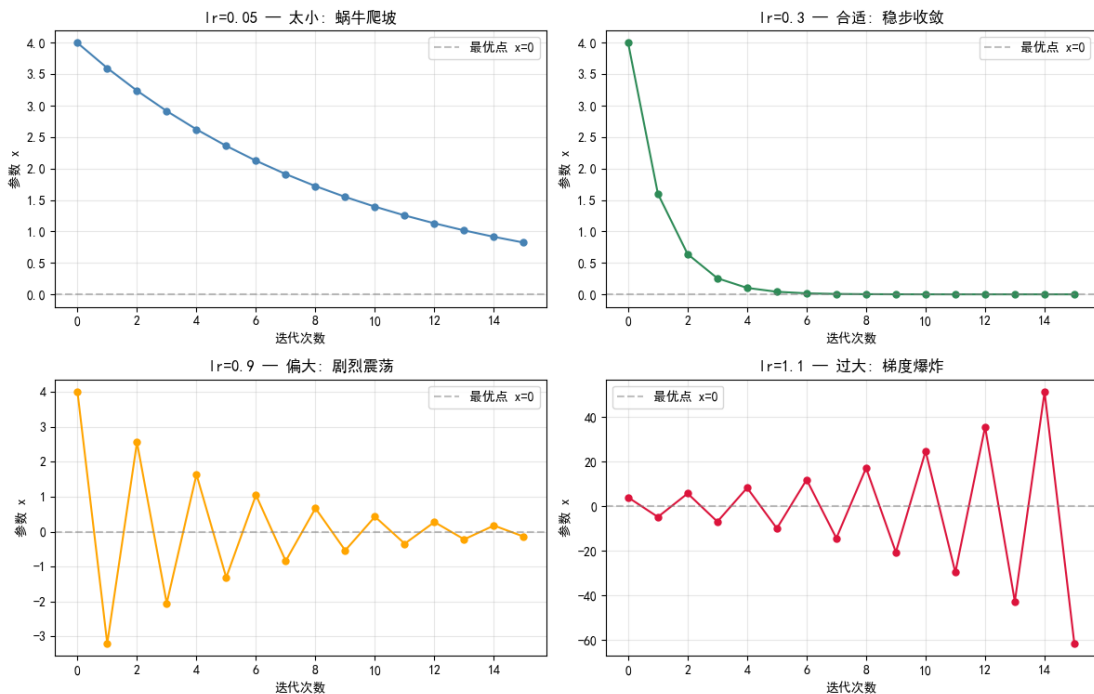


图 2: 学习率对梯度下降的影响（参数变化）

学习率对梯度下降的影响（损失曲面轨迹）

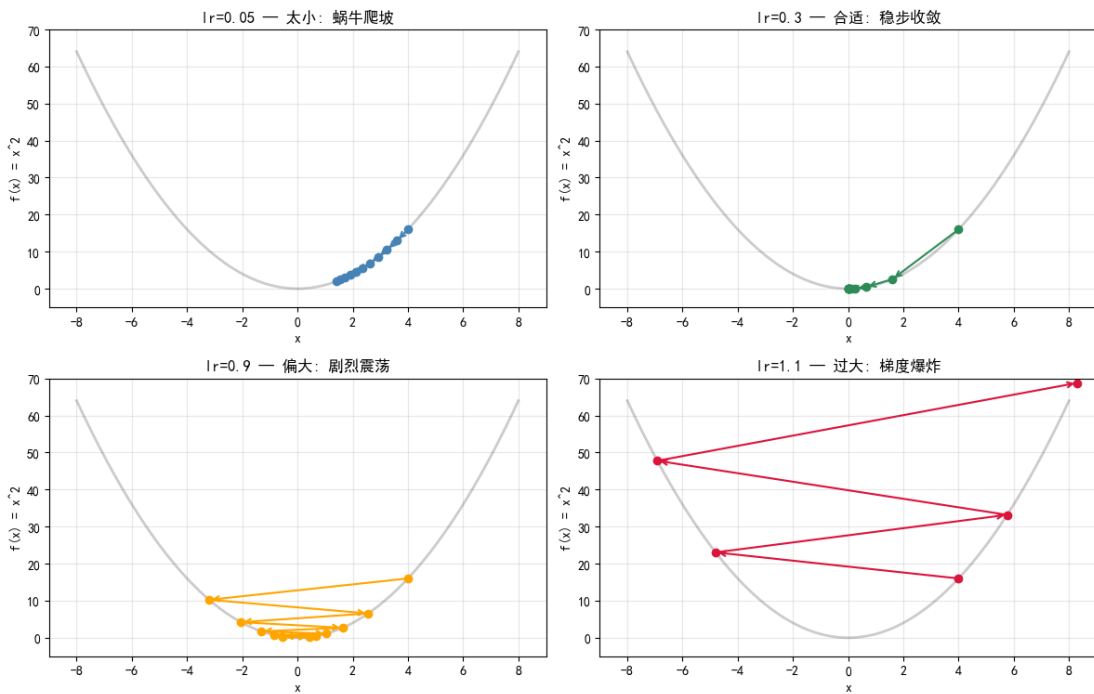


图 3: 学习率对梯度下降的影响（损失曲面轨迹）

备注

[可视化结果分析] 生成了两幅图：

1. **参数变化图**：展示四种学习率下参数随迭代次数的变化轨迹
2. **损失曲面轨迹图**：在 $f(x) = x^2$ 曲线上展示梯度下降路径

从图中可以清晰看到：

- 学习率 0.05：收敛缓慢，需要更多迭代
- 学习率 0.3：最优，稳定快速收敛
- 学习率 0.9：震荡但最终收敛
- 学习率 1.1：发散，无法收敛

1.4.6 Adam 优化器实战

核心原理

Adam = 动量法 (Momentum) + RMSProp，自适应学习率，收敛速度远超基础梯度下降。

Python 代码实现

```

1 def adam_optimizer(X, y, lr=0.1, epochs=200,
2                   beta1=0.9, beta2=0.999, epsilon=1e-8):
3     # 初始化参数
4     w, b = 0.0, 0.0
5     n = len(X)
6
7     # Adam 特有的状态变量初始化
8     m_w, v_w = 0.0, 0.0 # w 的一阶矩和二阶矩
9     m_b, v_b = 0.0, 0.0 # b 的一阶矩和二阶矩
10
11     # 初始化历史记录
12     history = {'w': [w], 'b': [b],
13              'Loss': [np.mean((w*X + b - y)**2)]}
14
15     # 进行迭代训练
16     for epoch in range(epochs):
17         t = epoch + 1 # Adam 的时间步 t 从 1 开始
18
19         # 1. 计算当前预测值
20         y_pred = w * X + b
21
22         # 2. 计算当前梯度
23         dw = (2/n) * np.sum((y_pred - y) * X)
24         db = (2/n) * np.sum(y_pred - y)
25
26         # 3. 更新一阶矩估计 (动量)
27         m_w = beta1 * m_w + (1 - beta1) * dw
28         m_b = beta1 * m_b + (1 - beta1) * db
29
30         # 4. 更新二阶矩估计 (RMSProp)
31         v_w = beta2 * v_w + (1 - beta2) * (dw**2)
32         v_b = beta2 * v_b + (1 - beta2) * (db**2)
33
34         # 5. 偏差校正
35         m_w_hat = m_w / (1 - beta1**t)
36         m_b_hat = m_b / (1 - beta1**t)
37         v_w_hat = v_w / (1 - beta2**t)
38         v_b_hat = v_b / (1 - beta2**t)
39
40         # 6. 使用 Adam 公式更新参数
41         w -= lr * m_w_hat / (np.sqrt(v_w_hat) + epsilon)

```



```

42     b -= lr * m_b_hat / (np.sqrt(v_b_hat) + epsilon)
43
44     # 计算当前 loss
45     current_loss = np.mean((w*X + b - y)**2)
46
47     # 记录参数和损失
48     history['w'].append(w)
49     history['b'].append(b)
50     history['loss'].append(current_loss)
51
52     return history

```

备注

[Adam 优化器优势] Adam 优化器相比基础梯度下降的优势：

1. 自适应学习率：每个参数都有独立的学习率
2. 动量加速：累积历史梯度方向，加速收敛
3. 稳定性强：通过二阶矩调整，适应不同梯度尺度
4. 收敛更快：通常比基础梯度下降快 10-100 倍

Adam 优化器训练过程可视化

Python 代码实现

```

1  # 可视化 Adam 优化器训练过程
2  fig, axes = plt.subplots(1, 3, figsize=(15, 5))
3
4  # 左图：拟合效果
5  axes[0].scatter(areas_m2, prices, alpha=0.6, s=60,
6                  edgecolors='black', linewidth=0.5, label='房屋数据')
7  x_fit = np.linspace(areas_m2.min(), areas_m2.max(), 100)
8  y_fit = history['w'][-1]/100 * x_fit + history['b'][-1]
9  axes[0].plot(x_fit, y_fit, 'r-', linewidth=2.5,
10              label=f'Adam拟合: y = {history["w"][-1]/100:.2f}x + {history["b"][-1]:.2f}')
11 axes[0].set_xlabel('面积 (平方米)', fontsize=12)
12 axes[0].set_ylabel('房价 (万)', fontsize=12)
13 axes[0].set_title('Adam优化器拟合效果', fontsize=13, fontweight='bold')
14 axes[0].legend(fontsize=10)
15 axes[0].grid(True, alpha=0.3)
16
17 # 中图：参数收敛过程
18 axes[1].plot(history['w'], 'b-', linewidth=2, label='w (权重)')
19 axes[1].plot(history['b'], 'g-', linewidth=2, label='b (偏置)')
20 axes[1].axhline(y=250, color='r', linestyle='--', alpha=0.5, label='真实 w=250')
21 axes[1].axhline(y=30, color='orange', linestyle='--', alpha=0.5, label='真实 b=30')
22 axes[1].set_xlabel('迭代次数', fontsize=12)
23 axes[1].set_ylabel('参数值', fontsize=12)
24 axes[1].set_title('Adam参数收敛过程', fontsize=13, fontweight='bold')
25 axes[1].legend(fontsize=10)
26 axes[1].grid(True, alpha=0.3)
27
28 # 右图：损失函数下降过程
29 axes[2].plot(history['loss'], 'purple', linewidth=2)
30 axes[2].set_xlabel('迭代次数', fontsize=12)
31 axes[2].set_ylabel('MSE 损失', fontsize=12)
32 axes[2].set_title('Adam损失下降过程', fontsize=13, fontweight='bold')
33 axes[2].grid(True, alpha=0.3)
34 axes[2].set_yscale('log') # 使用对数坐标
35

```

```
36 plt.tight_layout()
37 plt.show()
```

备注

[Adam 可视化说明] 通过可视化可以观察到：

1. **拟合效果**：Adam 优化器成功拟合房价数据，结果与基础梯度下降一致
2. **收敛速度**：Adam 的参数收敛更快，震荡更小
3. **损失下降**：Adam 的损失下降更平滑，收敛更稳定

对比基础梯度下降，Adam 优化器展现出更优的收敛性能。

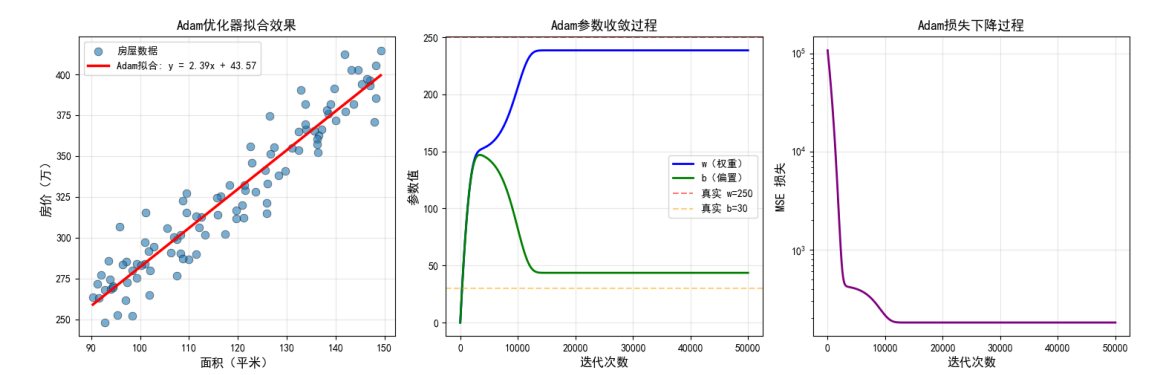


图 4: Adam 优化器训练过程可视化

1.4.7 梯度下降的三种核心变体

算法名称	全称	数据使用量	核心特点
BGD	全量梯度下降	全部训练数据	梯度精准，速度极慢，不适合大数据
SGD	随机梯度下降	单条数据	速度快，噪声大，震荡剧烈
Mini-batch GD	小批量梯度下降	一小批数据 (32/64)	业界标准，兼顾速度与稳定性

备注

[三种变体的选择建议]

- **小数据集** ($n < 1000$)：使用 BGD，梯度最准确
- **中等数据集** ($1000 < n < 10000$)：使用 Mini-batch GD，batch size=32 或 64
- **大数据集** ($n > 10000$)：使用 SGD 或 Mini-batch GD，配合 Adam 优化器

大模型学习起始篇

2026 Spring

Chapter 2: 从函数到 Transformer 与实战

大模型基础架构与实战落地

Author: Cabbage

ECNU

本讲摘要

本章从最基础的数学函数和线性映射出发，带你一步步推演深度学习的发展脉络。从单层网络到深度学习，从 CNN、RNN 再到当下主流的 Transformer 架构，我们将揭开大模型底层的原理面纱。随后，本章还会探讨基础模型在训练和扩展时所面临的优化问题。

2.1 从函数到 Transformer

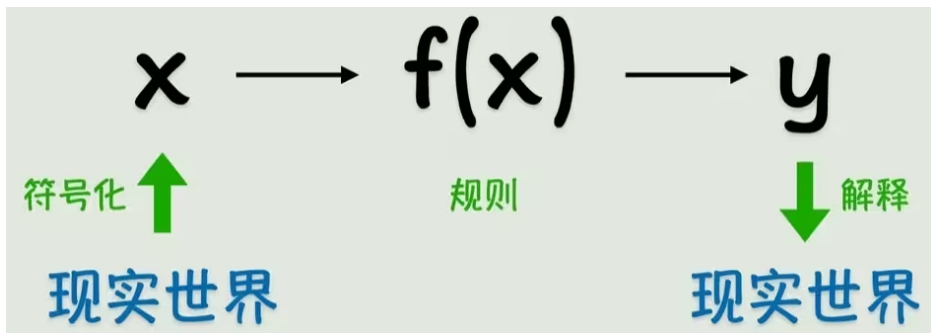
深度学习看似复杂，但其核心逻辑源自于最基础的数学概念。本节我们将通过通俗易懂的语言、严谨的公式以及形象的图表，带您完成一次从简单函数到前沿架构的认知飞跃。

2.1.1 现实世界与数学符号的映射

人类认识世界的过程，本质上是将现实事物**符号化**，通过确定的**规则**（函数 $f(x)$ ）对其进行处理与转换，再对输出结果进行**解释**以指导现实。

在机器学习中，我们的核心任务便是寻找这样一个最优映射函数 $f(x)$ 。例如，将输入的图片像素矩阵映射到代表“猫”或“狗”的类别标签上：

$$y = f(x)$$



2.1.2 线性回归：最基础的模型规则

在各种规则中，线性模型是最简单、最基础的。例如一元线性方程：

$$y = wx + b$$

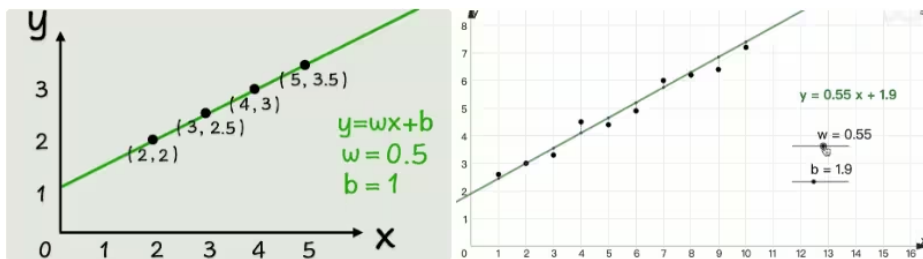
其中：

- x 代表输入的特征 (Feature)；
- w 是权重 (Weight)，决定了特征对结果的贡献程度；
- b 是偏置 (Bias)，代表了模型的基准平移。

分析

直观理解参数作用：

权重 w 调整直线的倾斜角度，而偏置 b 决定了直线在坐标轴上的截距。通过不断优化这两个参数，我们可以让直线尽可能地拟合数据分布。

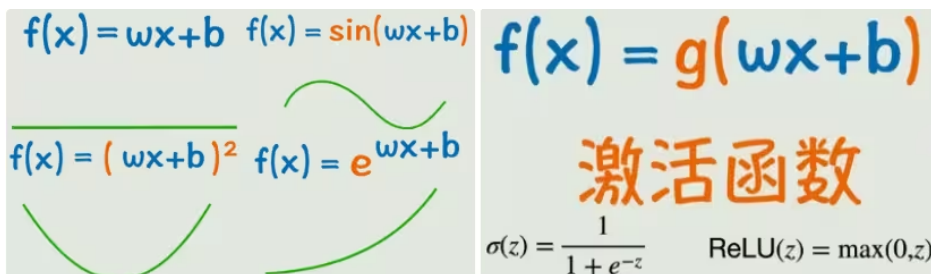


2.1.3 激活函数：赋予模型非线性的灵魂

现实世界的分布极少是完美线性的。如果只使用线性操作（即使叠加了无数层： $f_2(f_1(x)) = w_2(w_1x + b_1) + b_2$ ），其本质仍是线性映射。

为了打破这种限制，我们在每层线性计算后引入**激活函数 (Activation Function)** $g(x)$ 。常见的激活函数包括：

- **Sigmoid:** $\sigma(x) = \frac{1}{1 + e^{-x}}$ ，常用于将输出压缩到 $(0, 1)$ 区间。
- **ReLU:** $\text{ReLU}(x) = \max(0, x)$ ，由于其计算极简且能有效缓解梯度消失，被广泛应用于深层网络。



2.1.4 复合函数：构建多层嵌套的智能

加入了非线性激活函数后，我们将简单的函数进行复合与嵌套，形成深层次的映射。例如：

$$f(x) = g(w_3 \cdot g(w_1x_1 + w_2x_2 + b_1) + b_2)$$

这其实正是多层神经网络的底层逻辑。每一层将前一层的输出作为输入，从而提取出越来越抽象的特征。

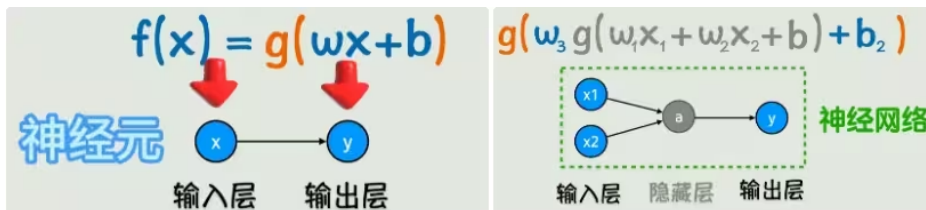
$$g(w_3 g(w_1x_1 + w_2x_2 + b) + b_2)$$

2.1.5 从神经元到人工神经网络

为了更加直观地表示复杂的复合函数，我们将其图结构化。我们将“输入、加权求和、激活”这一系列操作合并为一个节点，称之为**神经元**。

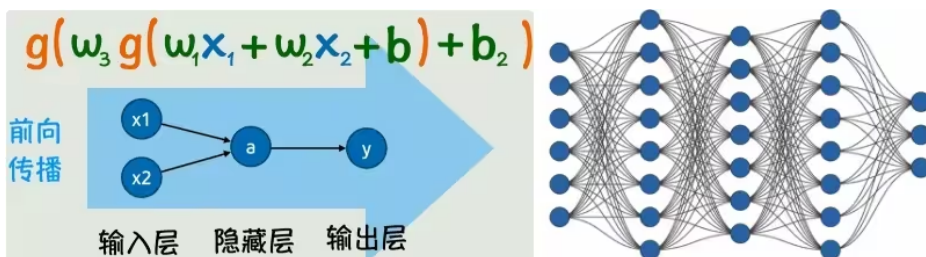
多个神经元组合起来，便形成了**人工神经网络 (Artificial Neural Network, ANN)**。主要包含三层结构：

1. 输入层 (Input Layer): 接收原始数据。
2. 隐藏层 (Hidden Layer): 进行特征提取与转换。
3. 输出层 (Output Layer): 得出最终预测结果。



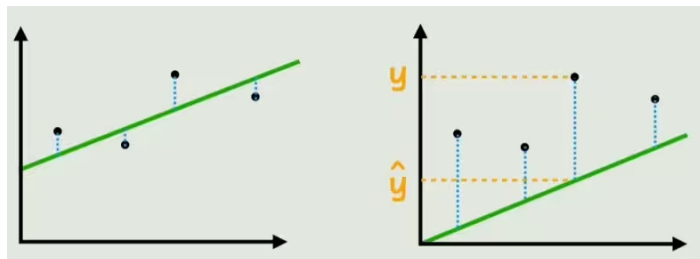
2.1.6 前向传播与深度网络架构

数据从输入层出发，经过隐藏层的层层计算到达输出层的过程被称为前向传播 (Forward Propagation)。增加隐藏层的层数和节点数后，网络变得极为“深邃”，这就构成了深度神经网络 (Deep Neural Network, DNN)。



2.1.7 预测偏差与损失函数

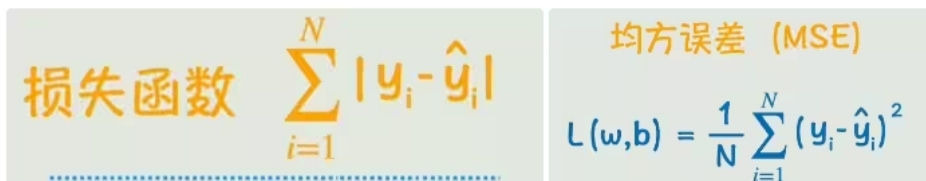
在模型初始化时，其内部参数是随机的，导致预测结果 $\hat{y}$ 与真实结果 y 之间存在巨大的差异。



为了量化这种差距，我们定义了损失函数 (Loss Function)。例如对于回归任务，最常用的是均方误差 (MSE)：

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

对于分类任务，最常用的是交叉熵损失。模型训练的唯一目标，就是通过调整参数，使得损失函数 L 达到最小。

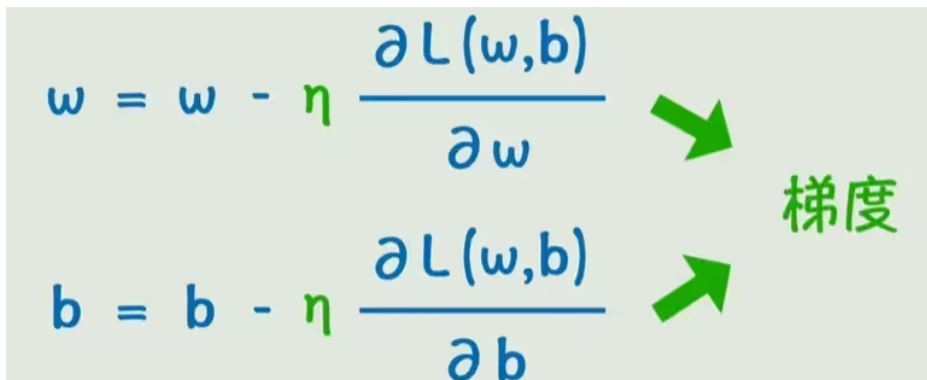


2.1.8 梯度下降与反向传播

为了找到使得损失最小的参数 w 和 b ，我们采用**梯度下降 (Gradient Descent)** 算法：

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w_t}$$

其中， η 为学习率 (Learning Rate)，控制每次更新的步长。

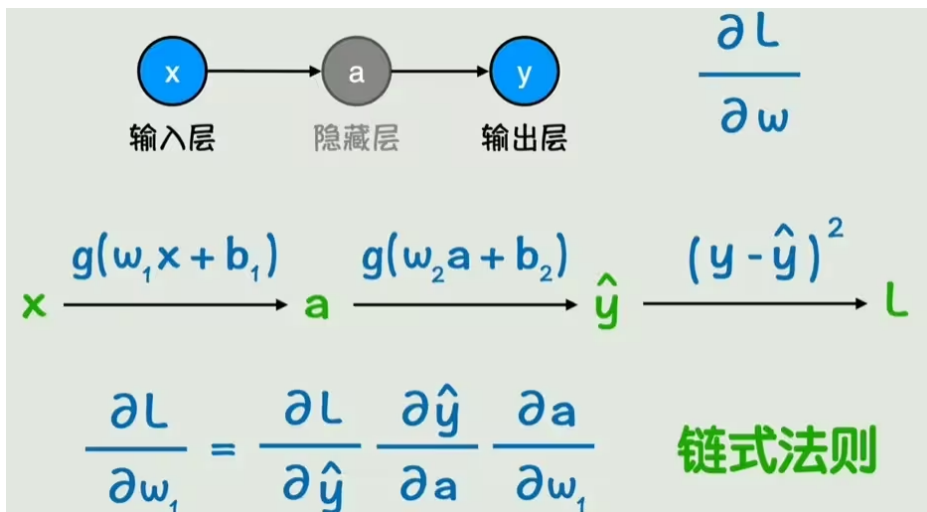


$$w = w - \eta \frac{\partial L(w,b)}{\partial w}$$

$$b = b - \eta \frac{\partial L(w,b)}{\partial b}$$

梯度

在具有数以亿计参数的深层网络中，高效计算梯度的核心技术是**反向传播 (Backpropagation)** 算法。它利用微积分中的**链式法则**，将输出层的误差反向一层层传递给隐藏层，从而计算出每个参数的梯度。



$\textcircled{x}$
输入层

 $\textcircled{a}$
隐藏层

 $\textcircled{y}$
输出层

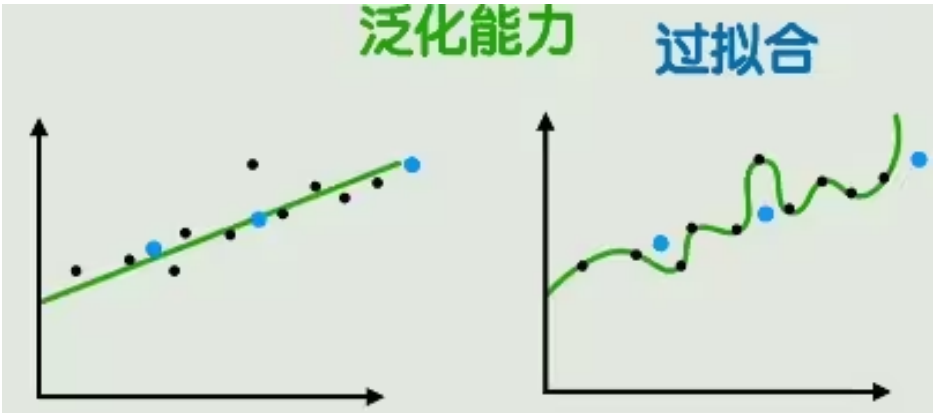
$$x \xrightarrow{g(w_1x + b_1)} a \xrightarrow{g(w_2a + b_2)} \hat{y} \xrightarrow{(y - \hat{y})^2} L$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a} \frac{\partial a}{\partial w_1}$$

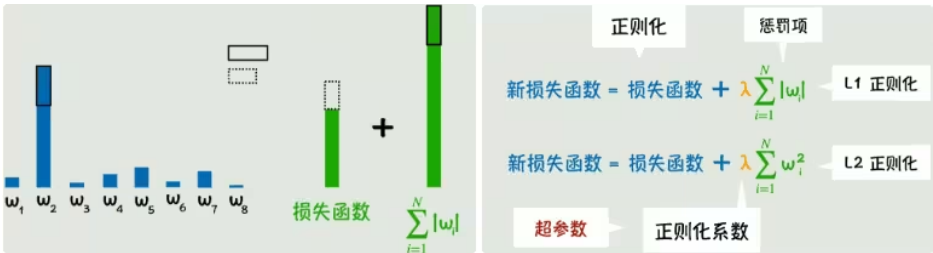
链式法则

2.1.9 过拟合与正则化技术

如果模型结构过于复杂或训练过度，模型可能会将数据中的“噪声”也一并记住。这种现象称为**过拟合 (Overfitting)**，表现为训练集上效果极好，但测试集上表现糟糕。



为了缓解过拟合，常引入正则化技术。特别是 **Dropout**，它会在每次训练中随机使得部分神经元“失活”（输出置 0），迫使网络不依赖特定的局部特征，从而增强泛化能力。

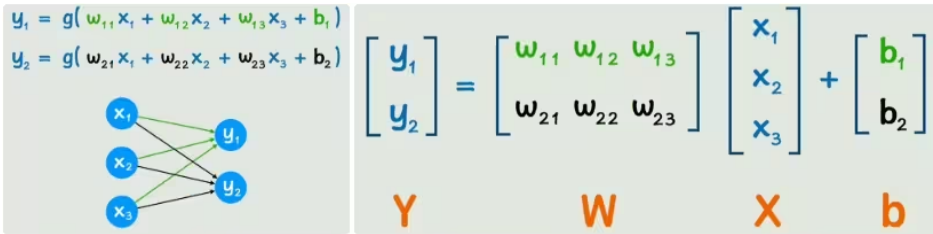


2.1.10 CNN：卷积神经网络的崛起

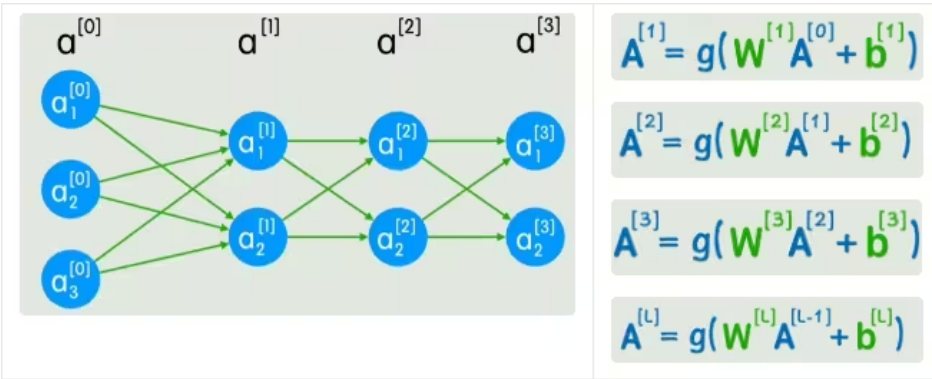
在处理图像时，图片表示为多维像素张量。如果直接使用全连接层，参数量将面临指数级爆炸。

梯度消失	神经网络越深，梯度反向传播时越来越小，参数更新困难	用残差网络来防止深层网络的梯度衰减
梯度爆炸	梯度数值越来越大，参数的调整幅度失去控制	用梯度裁剪来防止梯度的更新过大 用合理的权重初始化和输入数据归一化来让梯度分布更加平滑
收敛速度慢	可能陷入局部最优或来回震荡	用动量法、RMSProp、Adam等自适应优化器来加速收敛，减少震荡
计算开销大	数据规模太大，完整的前向传播和反向传播都耗时巨大	用mini-batch把巨量数据分割成小批次，来降低单词的计算开销

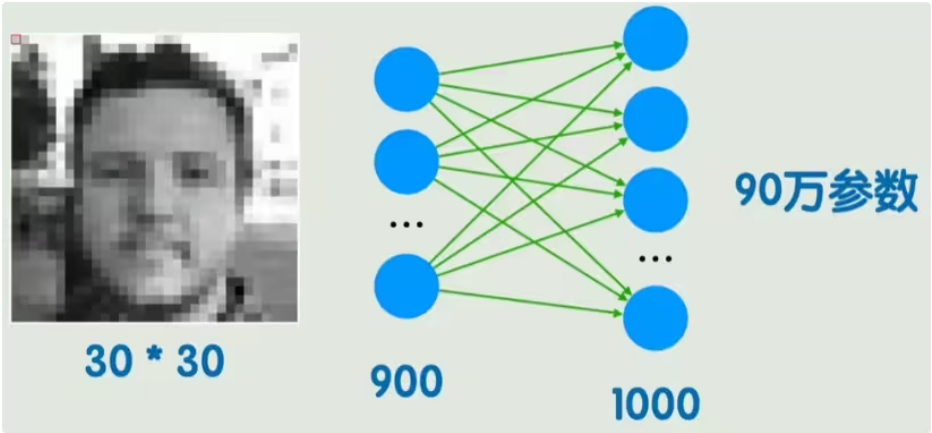
卷积神经网络 (CNN) 引入了**卷积核 (Filter/Kernel)**。卷积核在图像矩阵上滑动，通过局部乘加运算提取特定特征。



不同的卷积核可以提取不同的特征图 (Feature Map)。



在卷积层后通常接入**池化层 (Pooling Layer)** (如最大池化)，它在局部区域取最大值，在实现降维的同时赋予了网络“平移不变性”。



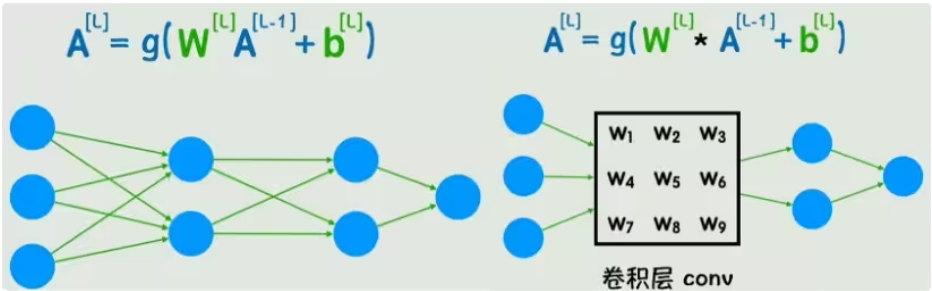
由卷积层、池化层和全连接层组成的 CNN 架构，曾经统治了计算机视觉领域多年。



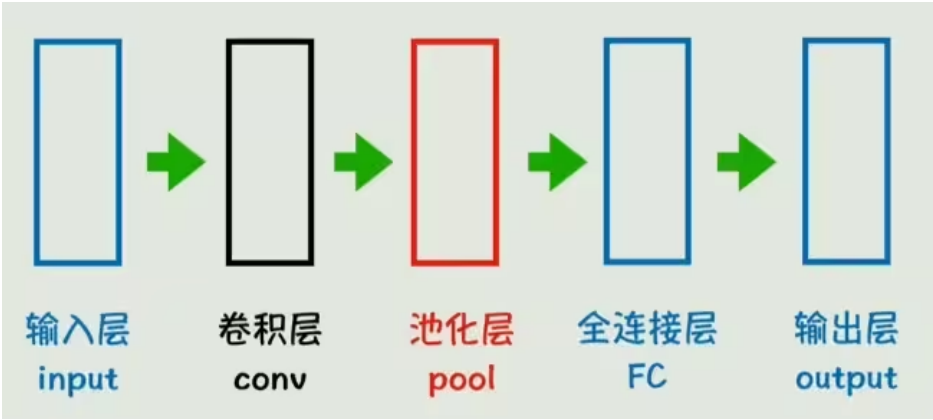
2.1.11 RNN：让网络拥有记忆

面对自然语言等**序列数据**时，CNN 显得束手无策，因为这类数据的理解高度依赖于时序信息。

循环神经网络 (RNN) 引入了时间维度的隐藏状态。在时间步 t ，它接收当前的输入 x_t 与上一时刻传来的记忆 h_{t-1} 。

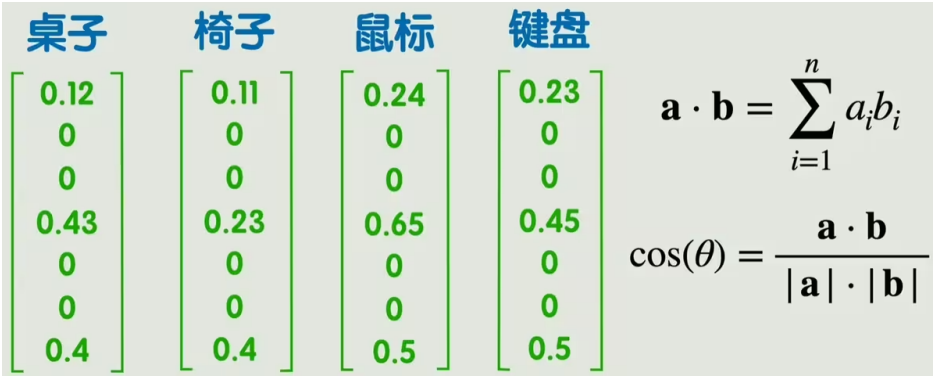


将 RNN 按时间展开，信息在时序上不断传递。

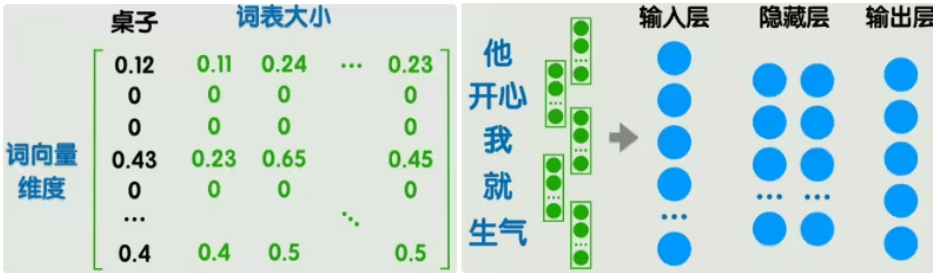


2.1.12 词嵌入：语言的数学表达

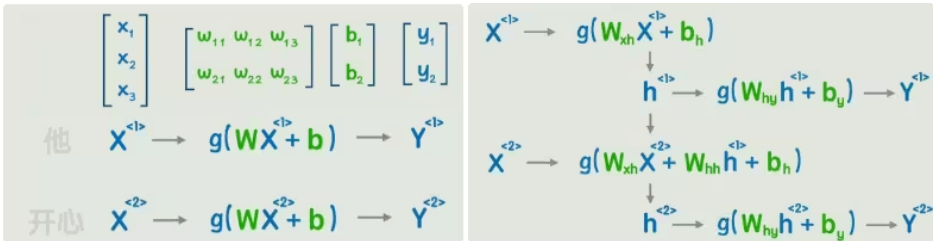
自然语言是以离散词汇的形式存在的，模型无法直接处理。将词汇转换为连续的实数向量的技术被称为词嵌入 (Word Embedding)。



在词嵌入的高维空间中，语义相近的词距离更近，并隐含了奇妙的数学关系。

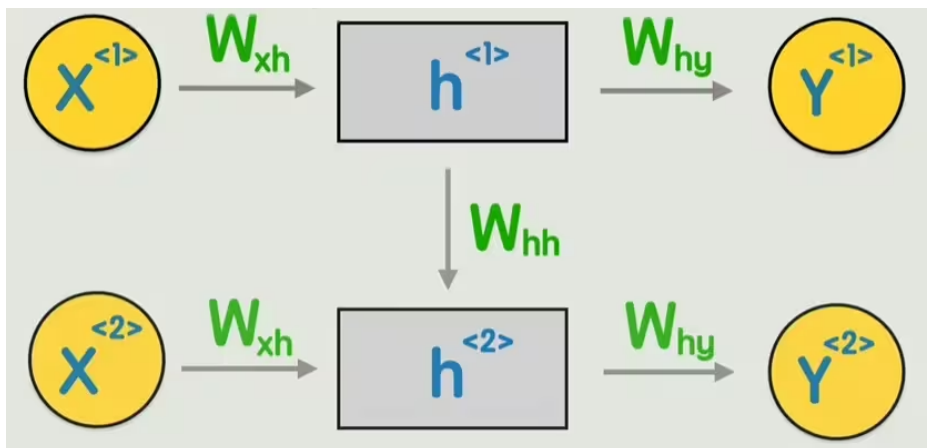


通过词嵌入，文本转化为稠密的词向量矩阵，输入到深度网络中。



2.1.13 注意力机制与 Transformer 的诞生

早期的 RNN 在处理长序列时常面临“长距离依赖遗忘”和无法并行的痛点。**注意力机制 (Attention)** 允许模型在解码当前词时，自动“聚焦”于源序列中最相关的特征。



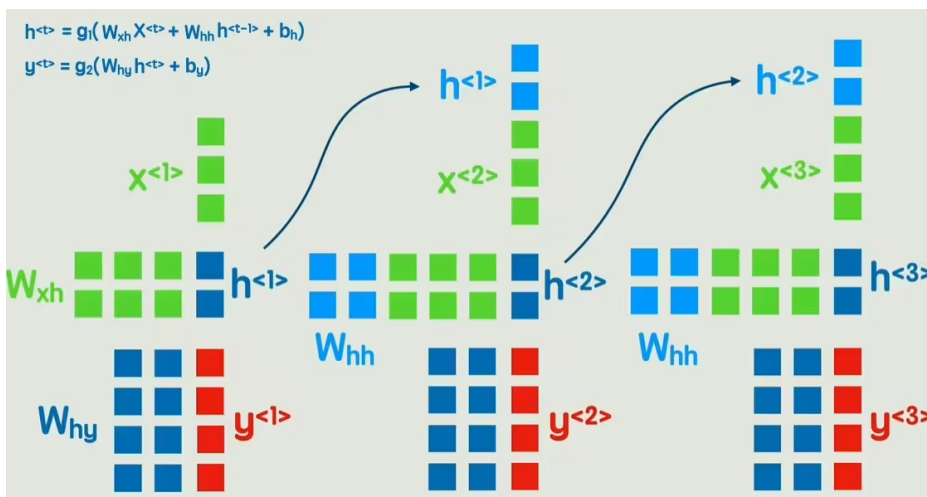
2017 年，**Transformer** 彻底抛弃了循环结构，完全依赖**自注意力机制 (Self-Attention)** 建立词语之间的全局依赖。

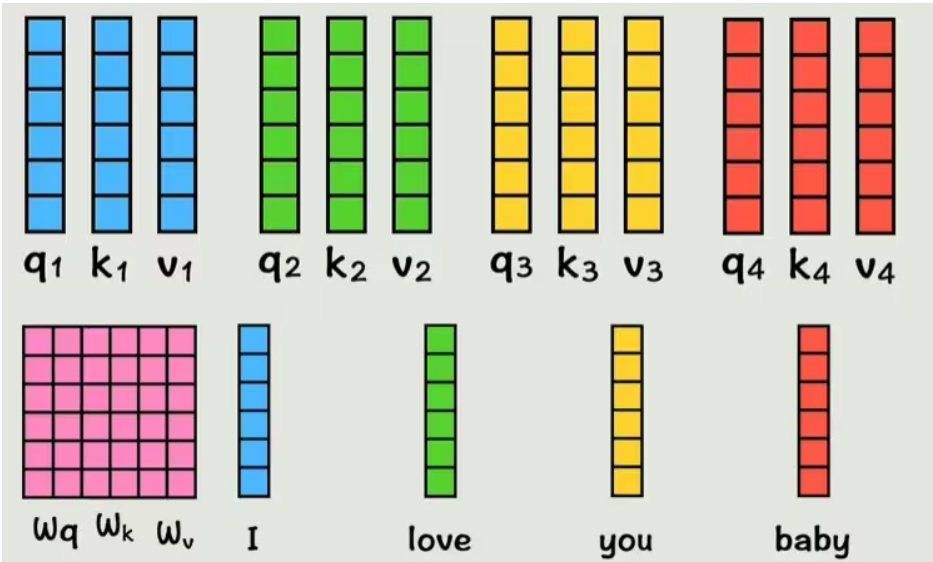
本讲摘要

Self-Attention 的核心计算：

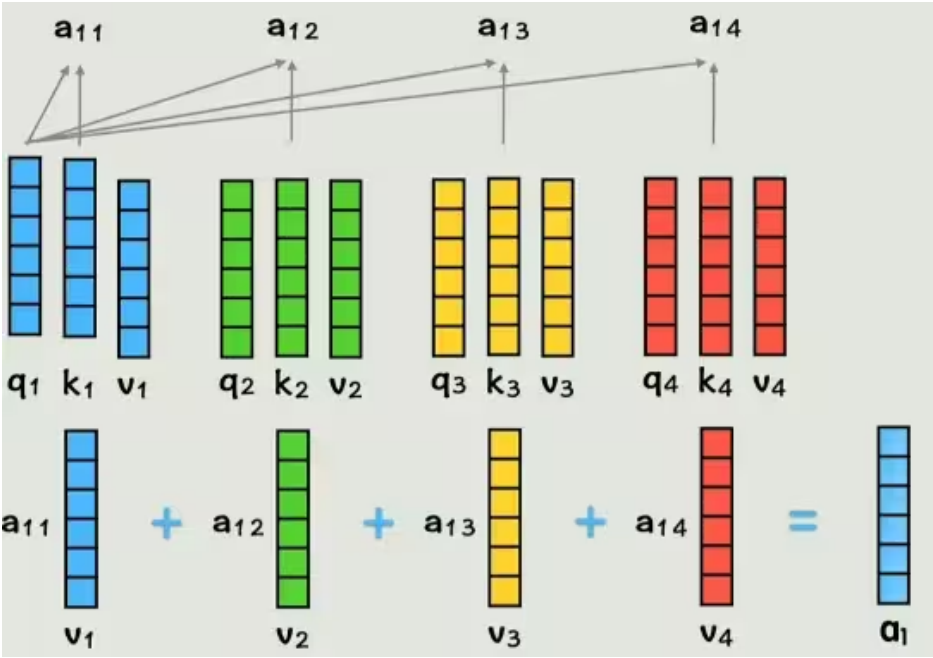
1. 每一个词生成三个向量：查询 Q 、键 K 、值 V 。
2. 利用 Q 和其他词的 K 进行点积，计算“相关度分数”。
3. 经过 Softmax 后，作为权重对所有 V 向量加权求和，得到融合上下文的新表示：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



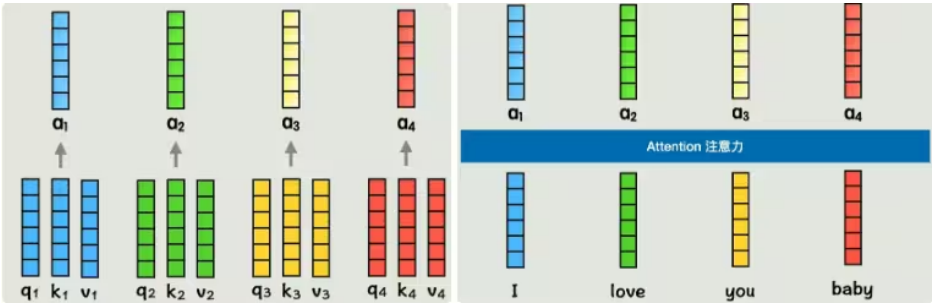


在生成文本时，为防止模型“偷看”未来的词，采用了掩码自注意力机制 (Masked Attention)。

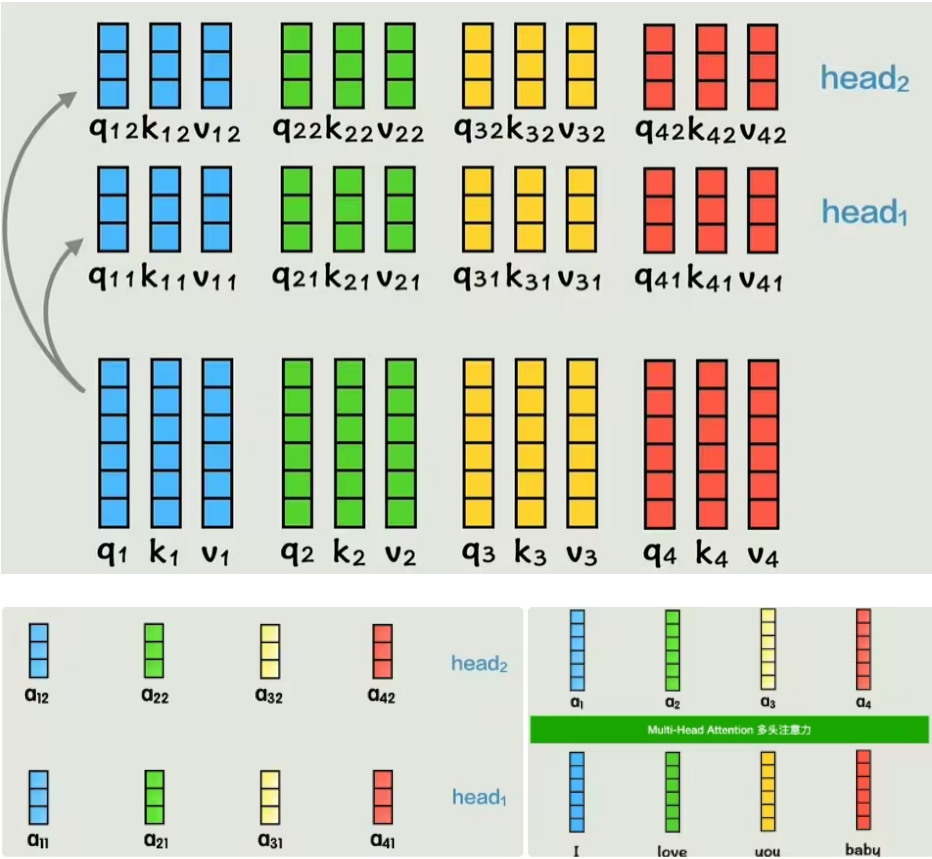


2.1.14 多头注意力与 Transformer 架构全貌

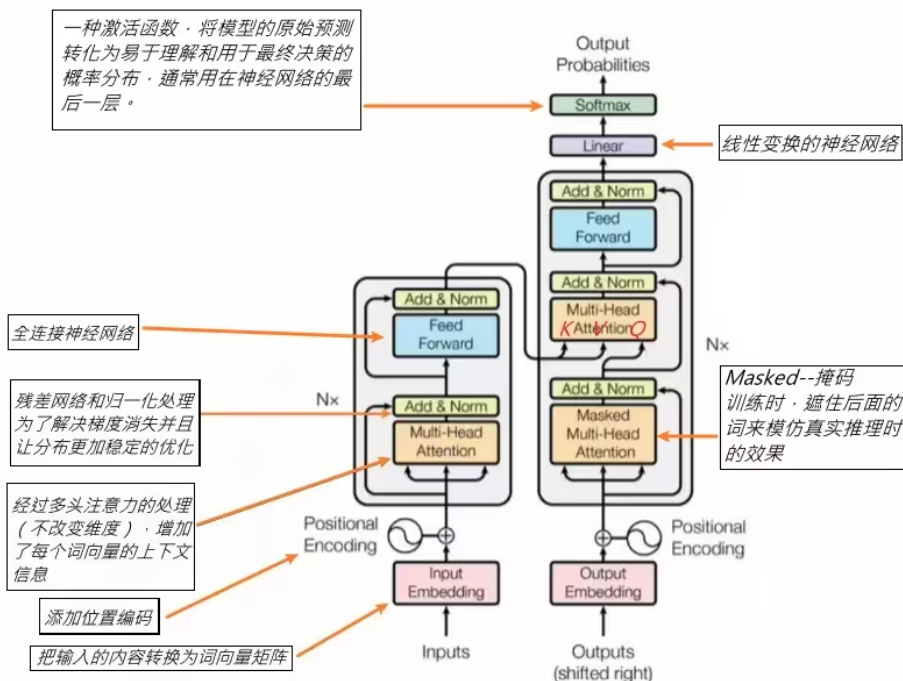
多头注意力 (Multi-Head Attention) 将特征空间切分，使不同的“注意力头”独立关注序列中不同维度的信息（语法、语态等），最后拼接。



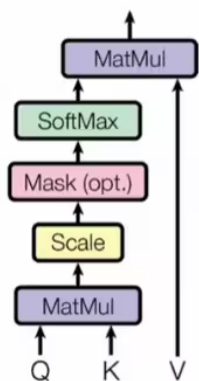
Transformer 由**编码器 (Encoder)** 和**解码器 (Decoder)** 组成：



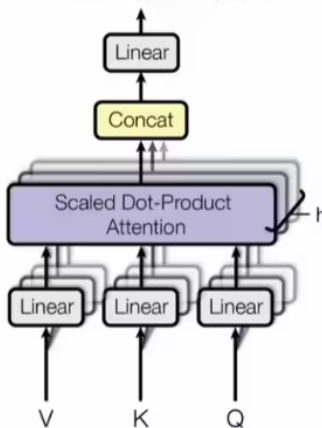
配合上残差连接与层归一化，Transformer 拉开了大模型时代的序幕。



Scaled Dot-Product Attention



Multi-Head Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

2.2 实战：Transformer 的基础代码与分析模块

在这一节中，我们从代码层面解析目前深度学习领域内 Transformer 架构的发展与基础。本节主要通过实战代码组，演示如何从零实现一个词表、词嵌入、注意力机制提取以及构建一个类似于工业界底层测试用的 Transformer Encoder 分类模型。

在真实实战模型中，我们将全方位向模型传递代码的四个基本核心流水线步骤：**数据集与数据清洗、文本高维数字化、模型核心架构构建以及模型训练优化与循环**。在这些核心中，每一行代码我们都加入了详尽的学习逻辑和细致的代码中文注释，请大家充分理解代码背景与数学原理是如何在实战中交织的。

2.2.1 第一步：标准与高质量数据集的协同提取

深度学习模型依赖海量数据。我们的数据常存放在 `news_data.csv` 文件中。我们首先要编写代码，安全地读取该文件，并将新闻内容和对应的类目标签一同提取出来。

在实战数据收集常常良莠不齐，假设某些 CSV 文件存在缺失，没有 `content`（有些新闻没有内容或标签，也就是可能存在缺失值）。为此，使用典型的防御性编程（Defensive Programming）思想，通过判断 `content` 是否为空，动态读取文本，如果缺失时直接使用文件占位符作为备用标签。这种代码范式能极大程度地保证程序健壮性。

Python 代码实现

```

1  # ===== 1. 导入必要的 Python 工具包（库） =====
2  # 语法小课堂：import 表示引入别人写好的代码包。as 是给它起个简短的别名，方便后面写代码。
3  import os                # os 用于操作文件路径和文件夹
4  import pandas as pd      # pandas 专门用来处理表格，我们叫它 pd
5  import torch             # 深度学习老大哥 PyTorch
6  import torch.nn as nn    # nn 是 Neural Network(神经网络)的缩写，里面全是网络层
7  import torch.optim as optim # optim 是优化器模块，负责做梯度下降
8  # 语法小课堂：from ... import ... 表示只从一个大包里拿出我们需要的小工具
9  from torch.utils.data import Dataset, DataLoader
10
11 # ===== 2. 读取新闻数据集 =====
12 data_dir = 'news/news-articles-classification-dataset-for-nlp-and-ml' # 字符串变量，存放路径
13
14 # 语法小课堂：列表推导式 [结果 for 变量 in 集合 if 条件]
15 # 这行代码的意思是：把 data_dir 文件夹里所有的文件名（f）拿出来，如果名字是以 '.csv'
16 # 结尾的，就留下装进 csv_files 列表(箱子)里。
17 csv_files = [f for f in os.listdir(data_dir) if f.endswith('.csv')]
18
19 # 语法小课堂：[] 代表列表(List)；{} 代表字典(Dict)。
20 all_data = []           # 空列表，一会儿用来装所有的（新闻文字，类别编号）
21 labels_map = {}         # 空字典，一会儿用来记录 "business"->0 这样的对应关系
22
23 for file in csv_files:
24     file_path = os.path.join(data_dir, file)
25     df = pd.read_csv(file_path)
26
27     # 寻找装有文字的那一列的名字，在我们的数据集中是 'content' 列
28     text_col = 'content' if 'content' in df.columns else df.columns[0]
29     # 寻找装有类别的那一列的名字，在我们的数据集中是 'category' 列
30     label_col = 'category' if 'category' in df.columns else None
31
32     for idx, row in df.iterrows():
33         # 获取文本和类别
34         text = str(row[text_col])
35         # 如果有 category 列就用，没有就回退到使用文件名
36         label_name = str(row[label_col]) if label_col else file.replace('_data.csv', '')
37
38         # 给遇到的新类别分配一个唯一的编号
39         if label_name not in labels_map:
40             labels_map[label_name] = len(labels_map)
41
42         # 跳过纯空白数据
43         if text.strip() and text != 'nan':

```



```

43         all_data.append((text, labels_map[label_name]))
44
45 # 语法小课堂: f"..." 是格式化字符串, 花括号 {} 里的变量会自动填进去
46 print(f"总共读取了 {len(all_data)} 条新闻数据。")
47 print(f"类别映射 (字典长这样): {labels_map}")

```

执行输出:

```

1 总共读取了 10000 条新闻数据。
2 类别映射 (字典长这样): {'business': 0, 'education': 1, 'entertainment': 2, 'sports': 3,
   'technology': 4}

```

2.2.2 第二步：文本数字化（建立词典与填充）及包装为数据集

深度学习网络单元只执行数学计算，完全无法理解汉字或英文字符。因此，需将每个词“映射”为整数（词的编号）。

词表 (Vocabulary) 建立：我们通过 **Counter** 统计所有词汇的出现频率，保留最常见的前 10000 个词 (**VOCAB_SIZE**)。按频率由高到低赋予固定编号，例如 **apple** → 55。为对应那些内部未记录或从未在训练数据出现过的生僻词，我们预留了 **<UNK>**（编号为 1），作为统一的未知词占位符。

长度填充 (Padding)：GPU 并行加速运算要求输入数据的矩阵形状一致。因此，我们强制规定每篇新闻长度必须为 **MAX_LEN=128**。内容太长的直接在尾部截断；太短的，用编号为 0 的 **<PAD>** 字符在末尾进行补齐。

接着我们将这些处理打包成 PyTorch 官方标准的 **Dataset** 类中，并装载到 **DataLoader** 上，形成一个可迭代的底层批量数据 (**Batch**) 进行异步流水化。

Python 代码实现

```

1  from collections import Counter
2
3  MAX_LEN = 128      # 规定每篇新闻一律截断或补齐到 128 个单词
4  VOCAB_SIZE = 10000 # 词表最高容量: 1万个常见词
5
6  # 语法小课堂: def 用于定义一个函数(相当于一个加工厂), 接收输入 text, 加工后用 return 返回结果
7  def simple_tokenize(text):
8      # .lower() 变小写, .split() 默认按空格把句子切成一个个单词的列表
9      # 语法小课堂: 切片 [:MAX_LEN] 表示从第 0 个单词一直取到第 MAX_LEN(128)
10     # 个单词, 后面的直接扔掉
11     return str(text).lower().split()[:MAX_LEN]
12
13 word_counts = Counter() # Counter 是一种特殊的字典, 专门用来数数
14 for text, _ in all_data: # 遍历刚才收集的所有新闻, 下划线 _ 表示我们不关心第二个元素(类别标签)
15     word_counts.update(simple_tokenize(text)) # 把切碎的单词扔进去统计词频
16
17 # 手动初始化密码本(词表), 预留两个特殊号码 (这里的0用作填充符, 这里的1用作生僻词 (不在词表))
18 vocab = {'<PAD>': 0, '<UNK>': 1}
19
20 # .most_common 取出最常出现的前 9998 个词, 返回的是 [(词1, 次数), (词2, 次数)...]
21 for word, _ in word_counts.most_common(VOCAB_SIZE - 2):
22     # len(vocab) 算出有字典有多长。比如刚才里面有两个词, len 就是 2, 新进来的词编号就是 2。
23     vocab[word] = len(vocab)
24
25 # 把一段纯文本翻译成一串数字密码的加工厂函数 (把新闻转换为词表编码列表)
26 def text_to_indices(text):
27     tokens = simple_tokenize(text)
28     # 语法小课堂: 字典的 .get(单词, 默认值)
29     # 方法。如果单词在字典里, 返回它的编号; 如果不在, 返回 '<UNK>' 的编号(1)
30     indices = [vocab.get(w, vocab['<UNK>']) for w in tokens]

```

```

30     if len(indices) < MAX_LEN:
31         # 语法小课堂：在 Python 里，列表乘以一个数字，表示把列表复制多少遍。比如 [0] * 3 变成
           [0, 0, 0]
32         # 这里如果长度不够 128，就用 0 补齐。
33         indices += [vocab['<PAD>']] * (MAX_LEN - len(indices))
34     return indices # 返回长度固定为 128 的数字列表
35
36 # 语法小课堂：面向对象编程之 Class（类）。
37 # 类就像一张图纸。继承（Dataset）表示我们的图纸是在 Dataset 官方图纸基础上改的。
38 class NewsDataset(Dataset):
39     # __init__
           是“构造函数”。当我们根据图纸造出一个实实在在的管理员对象时，第一步就会自动运行这里。
40     # self 代表“自己”。self.data = data 就是把外面传进来的数据，存在自己肚子里。
41     def __init__(self, data):
42         self.data = data
43
44     # __len__ 定义了别人问你“有多长”时你怎么回答
45     def __len__(self):
46         return len(self.data)
47
48     # __getitem__ 定义了别人找你要“第 idx 个东西”时，你怎么拿给他
49     def __getitem__(self, idx):
50         text, label = self.data[idx] # 从自己肚子里的列表拿出对应的（文字，标签）
51
52         # 调用上面的翻译函数，然后套一层 torch.tensor，把普通的 Python 列表变成 PyTorch
           支持的“张量”（高级数组）
53         text_tensor = torch.tensor(text_to_indices(text), dtype=torch.long)
54         label_tensor = torch.tensor(label, dtype=torch.long)
55
56         return text_tensor, label_tensor
57
58 # 实例化：按图纸造一个仓库管理员 dataset
59 dataset = NewsDataset(all_data)
60
61 # 雇佣搬运工 dataloader，告诉他去 dataset 仓库拿货，每次拿 32 篇（batch_size），打乱顺序拿
           (shuffle)
62 dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

```

2.2.3 第三步：搭建 Transformer 核心分类网络架构

这部分我们使用面向对象（OOP）的编程方式，包装我们整个模型。在 `__init__` 构造函数中，我们建立了核心的“子工厂”：

1. **Embedding 嵌入层**：将之前转换的整数编号进一步转化为机器可读的、带有语义关系的连续实数空间向量（例如一个词扩展为 128 维的浮点数组）。
2. **Transformer Encoder 层**：调用多头自注意力机制（Self-Attention），使每一个词都可以看到整个上下文的历史输入，提取出整句话的语境信息。
3. **均值池化（Pooling）和分类**：使用 `mean(dim=1)` 强制将 128 个特征的真实信息浓缩为 1 个总结句特征。然后接入全连接的 Linear 线性层输出 5 个类别的得分矩阵。

Python 代码实现

```

1 import math
2
3 # 继承神经网络老大哥 nn.Module
4 class TransformerClassifier(nn.Module):
5     # 初始化函数：准备机器零件。参数是我们要告诉图纸的一些配置（比如词表多大，要几层等）
6     def __init__(self, vocab_size, embed_dim, num_heads, num_layers, num_classes, dropout=0.1):
7         super(TransformerClassifier, self).__init__() # 固定套路，先帮父类做初始化
8
9         # 机器1：词嵌入（把数字密码 变成 长长的小数向量）
10        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

```



```

11
12     # 机器2：配置单层 Transformer
13     encoder_layer = nn.TransformerEncoderLayer(
14         d_model=embed_dim,          # 输入向量的宽度
15         nhead=num_heads,            # 注意力探照灯(多头)的个数
16         dim_feedforward=embed_dim * 4, # 内部全连接层放大的倍数
17         dropout=dropout,
18         batch_first=True             # 声明我们的数据排布是 (批次, 句子长度, 词特征维度)
19     )
20     # 机器3：把刚才那一层叠 num_layers 次 (比如叠2层) 变成大编码器
21     self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
22
23     # 机器4：随机抛弃器 (防止过拟合的 Dropout)
24     self.dropout = nn.Dropout(dropout)
25
26     # 机器5：全连接线性分类器
27     # (把前面提取的高深莫测的向量特征, 强行拍碎映射到具体的5个分类上)
28     self.classifier = nn.Linear(embed_dim, num_classes)
29
30     # forward 是流水线, 规定了从输入 x 开始, 每台机器是怎么一步步起作用的
31     def forward(self, x):
32         # x 的长相: [32, 128] 代表 32 篇新闻, 每篇 128 个数字
33
34         # 第一步: 查字典映射
35         x = self.embedding(x) * math.sqrt(self.embedding.embedding_dim)
36         # 现在 x 变胖了, 长相: [32, 128, 128] (最后那个128是我们设置的 embed_dim 词向量维度)
37
38         # 第二步: 送进 Transformer, 词与词之间开始互相“眉目传情”(自注意力计算)
39         x = self.transformer_encoder(x)
40         # x 长相没变: [32, 128, 128], 但里面的小数已经揉杂了上下文的语义
41
42         # 第三步: 降维池化。
43         # 我们有 128 个词, 怎么代表一篇文章? 求个平均值吧。dim=1
44         # 意味着沿着“句子长度”那个维度压扁。
45         x = x.mean(dim=1)
46         # 现在 x 长相: [32, 128]。32篇文章, 每篇文章浓缩成了一个 128 维的终极特征。
47
48         # 第四步: 丢 Dropout
49         x = self.dropout(x)
50
51         # 第五步: 得出分类打分
52         logits = self.classifier(x)
53         # 最终长相: [32, 5] (假设有5个类)。这代表这 32篇新闻在各个类别的得分(未经过概率转化)。
54
55         return logits
56
57     # 获取真实新闻类别的个数
58     num_classes = len(labels_map)
59
60     # 根据图纸, 把我们的加工厂(模型)正式实例化造出来!
61     model = TransformerClassifier(
62         vocab_size=VOCAB_SIZE,
63         embed_dim=128,
64         num_heads=4,
65         num_layers=2,
66         num_classes=num_classes
67     )

```

2.2.4 第四步：参数与模型本质的训练循环（主步骤）

模型刚刚建立时，其内部的所有参数都是随机的。模型训练，本质上就是一个计算误差然后爬山寻找谷底的过程。这四步是 PyTorch 甚至绝大多数框架内标准的闭环：

- **清零 (zero_grad)**：抹去老师（优化器）之前残留的旧有梯度痕迹。
- **前向 (forward)**：让学生（模型）根据当前知识水平对考卷（数据）做一次题，得出预测答案。
- **求失 (loss)**：拿标准答案（真实标签）和学生的作答对比，评判到底偏差有多大（Loss）。

- **反向 (backward)**：运用链式求导法则逆向反推，找出为了减小该误差，每一个神经元参数应该怎么向负方向微调。
- **步进 (step)**：优化器（老师）全面下发调整命令，沿着梯度的方向走出一小步（Learning Rate）。

Python 代码实现

```

1  # 语法小课堂：if ... else ...
2  # 判断电脑里有没有 Nvidia GPU 加速卡 (cuda)。有就用，没有就老老实实用 CPU 算。
3  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
4
5  print(device)
6  model.to(device) # 把刚才造好的模型搬运到 GPU 显存里去
7
8  # 考官：多分类交叉熵损失函数。它专治分类错题。
9  criterion = nn.CrossEntropyLoss()
10
11 # 老师：Adam 优化器。
12 # model.parameters() 是一句指令，把工厂里所有能拧的旋钮（权重和偏置参数）都交给 Adam 老师控制。
13 # lr=1e-3 表示每次拧旋钮的幅度（学习率）。
14 optimizer = optim.Adam(model.parameters(), lr=1e-3)
15
16 epochs = 3 # 设定复习几遍题库
17
18 print("=== 开始激动人心的基础训练啦 ===")
19 for epoch in range(epochs): # 外层循环：第几遍刷题
20
21     model.train() # 拨下工厂的“训练”开关。这会让 Dropout 开始随机断电，增加训练难度。
22
23     total_loss = 0
24     correct = 0
25     total = 0
26
27     # 内层循环：每次拉来 32 篇新闻和答案。
28     # 语法小课堂：enumerate(data_loader) 会把批次编号赋给 batch_idx，把数据打包赋给后面。
29     # (inputs, labels) 是“解包”语法，因为 data_loader 返回的是元组，我们直接用两个变量接住它。
30     for batch_idx, (inputs, labels) in enumerate(data_loader):
31         inputs, labels = inputs.to(device), labels.to(device) # 数据也得送进 GPU
32
33         # 【核心 5 步法，深度学习永恒的铁律】
34
35         # 1. 老师说：清空上次的错题记录！
36         optimizer.zero_grad()
37
38         # 2. 学生做题：前向传播，通过前文写的流水线，得到分数矩阵。
39         outputs = model(inputs)
40
41         # 3. 考官判卷：算算预测分数和标准答案的差距（损失 Loss）。
42         loss = criterion(outputs, labels)
43
44         # 4. 反思找原因：反向传播！PyTorch
45             的魔法机制，自动根据微积分链式法则，算出每个旋钮该怎么扭才能让 Loss 变小。
46         loss.backward()
47
48         # 5. 执行修改：老师动手拧旋钮！参数被更新，模型变得更聪明了一点点。
49         optimizer.step()
50
51         # ---- 下面是汇报进度的代码，不影响训练 ----
52         total_loss += loss.item()
53
54         # outputs.max(1) 沿着类别的那个维度找最大值。
55         # 返回的 predicted 是网络认为概率最高的类别数字。
56         _, predicted = outputs.max(1)
57         total += labels.size(0)
58
59         # 看预测对不对。对了就加 1。
60         correct += predicted.eq(labels).sum().item()
61
62         # 每隔 100 车，汇报一次当下的成绩
63         if (batch_idx + 1) % 100 == 0:

```

```

63     acc = 100. * correct / total
64     print(f"大循环(Epoch) [{epoch+1}/{epochs}], 小批次 [{batch_idx+1}],
        错题大小(Loss): {loss.item():.4f}, 准确率(Acc): {acc:.2f}%")
65
66     print(" 基础版模型训练完成！跑通这个过程，你已经跨入深度学习的大门了！")

```

执行输出：

```

1  cuda
2  === 开始激动人心的基础训练啦 ===
3  大循环(Epoch) [1/3], 小批次 [100], 错题大小(Loss): 0.4038, 准确率(Acc): 66.03%
4  大循环(Epoch) [1/3], 小批次 [200], 错题大小(Loss): 0.3216, 准确率(Acc): 76.83%
5  大循环(Epoch) [1/3], 小批次 [300], 错题大小(Loss): 0.2272, 准确率(Acc): 81.15%
6  大循环(Epoch) [2/3], 小批次 [100], 错题大小(Loss): 0.3127, 准确率(Acc): 94.34%
7  大循环(Epoch) [2/3], 小批次 [200], 错题大小(Loss): 0.1082, 准确率(Acc): 94.33%
8  大循环(Epoch) [2/3], 小批次 [300], 错题大小(Loss): 0.0345, 准确率(Acc): 94.51%
9  大循环(Epoch) [3/3], 小批次 [100], 错题大小(Loss): 0.0233, 准确率(Acc): 97.00%
10 大循环(Epoch) [3/3], 小批次 [200], 错题大小(Loss): 0.0150, 准确率(Acc): 96.69%
11大循环(Epoch) [3/3], 小批次 [300], 错题大小(Loss): 0.0191, 准确率(Acc): 96.59%
12 基础版模型训练完成！跑通这个过程，你已经跨入深度学习的大门了！

```

本讲摘要

训练与模型优化小结

你可以看到控制台打印的 Loss 越来越小，Accuracy（准确率）越来越高，这说明模型底层的参数空间正在被我们提供的高质量真实数据不断重塑。即便是目前拥有千亿参数的大模型，其核心的训练逻辑与这个极简的循环步进代码本质也是一致无二的。

2.3 基础模型的训练优化方向

2.3.1 从线性代数到数据归一化

备注

[笔记说明] 本笔记以实对称矩阵的特征值分解为核心起点，逐步推导梯度下降算法的收敛条件与学习率上限，落地到线性回归模型，最终揭示数据统计特性对优化过程的决定性作用，完整串联「线性代数基础 → 优化理论 → 机器学习实践 → 数据预处理核心价值」的全逻辑链条。所有公式推导均保留完整步骤，符号定义全程统一，逻辑推进严格遵循递进关系。

一、核心基础概念定义

在展开定理与推导前，先明确本笔记全程涉及的核心矩阵与向量的定义、符号规范与核心性质：

- 向量规范：**所有向量均为 **d 维列向量**，记为 $x \in \mathbb{R}^d$ ，转置 x^T 为行向量；向量内积 $x^T y$ 为标量。
- 对角矩阵：** $d \times d$ 方阵 Λ ，当且仅当非对角线元素全为 0，记为 $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ ，对角线元素 λ_i 为其特征值。核心性质：对角矩阵相乘仅在对应维度生效，结果仍为对角矩阵。
- 单位矩阵：** $d \times d$ 对角矩阵 I ，对角线元素全为 1，非对角线全为 0。核心性质：对任意矩阵 A ，有 $IA = AI = A$ 。
- 正交矩阵：** $d \times d$ 方阵 Q ，满足 $Q^T Q = Q Q^T = I$ 。核心性质：
 - 逆矩阵等于转置，即 $Q^{-1} = Q^T$ ；

- 列/行向量为两两正交的单位向量（内积为 0，模长为 1）；
- 对向量的乘法等价于坐标系旋转变换，不改变向量模长（保范性）。

5. **实对称矩阵**：元素全为实数的 $d \times d$ 方阵 A ，满足 $A^T = A$ （转置后与自身相等）。

6. **正定矩阵**：对于 $d \times d$ 实对称矩阵 A ，若对任意非零 d 维向量 x ，都有 $x^T A x > 0$ ，则称 A 为正定矩阵。核心性质：正定矩阵的所有特征值均为正实数，是损失函数凸性与全局最优解存在的核心保障。

二、实对称矩阵的特征值分解

2.1 特征值分解定理

备注

[定理 1（实对称矩阵的特征值分解）] 对于任意 $d \times d$ 实对称矩阵 A ，一定可以分解为如下形式：

$$A = Q \Lambda Q^T$$

其中：

- Q ： $d \times d$ 正交矩阵，列向量是 A 的 d 个单位正交特征向量 $v_1, v_2, \dots, v_d$ ；
- Λ ： $d \times d$ 对角矩阵，对角线元素是 A 的特征值 $\lambda_1, \lambda_2, \dots, \lambda_d$ ，默认按从小到大排列 $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_d$ ；
- Q^T ： 正交矩阵 Q 的转置，满足 $Q^T = Q^{-1}$ 。

该定理的核心结论是：**任意实对称矩阵都可以通过正交变换实现相合对角化**，这是后续所有优化理论推导的核心数学基础。

2.2 实对称矩阵的核心性质

备注

[命题 1（实对称矩阵的核心性质）] 对于 $d \times d$ 实对称矩阵 A ，有如下关键性质：

1. 实对称矩阵的不同特征值对应的特征向量两两正交（几何上相互垂直），且所有特征向量均可单位化（模长缩放为 1）；
2. 实对称矩阵必有 d 个线性无关的特征向量，能够构成 d 维欧氏空间的一组正交基，可线性表示任意 d 维向量；
3. 对于二次型损失函数 $f(x) = \frac{1}{2} x^T A x$ ，特征值 λ_i 的大小对应损失函数在对应特征向量 v_i 方向上的曲率： λ_i 越大，损失函数在该方向上越陡峭，曲率越大。

三、特征值分解的核心意义：解耦

特征值分解的唯一核心价值，是**将耦合的多维二次型优化问题，完全拆解为多个独立的一维优化问题**，彻底消除维度间的相互影响，为梯度下降收敛性分析奠定基础。

我们以标准二次型损失函数为例，完成完整的解耦推导：

给定二次型损失函数（ A 为 $d \times d$ 实对称正定矩阵，保证凸性与唯一全局最优解）：

$$f(x) = \frac{1}{2} x^T A x$$

步骤 1：坐标旋转变换

对 A 做特征值分解 $A = Q\Lambda Q^T$ ，定义新参数向量 $z = Q^T x$ 。

该变换的几何意义：将原坐标系下的向量 x ，通过正交矩阵 Q^T 旋转，映射到以 A 的特征向量为基的新坐标系下，得到新向量 z 。

由正交矩阵性质 $Q^T Q = I$ ，对 $z = Q^T x$ 两边左乘 Q ，得到逆变换：

$$x = Qz$$

步骤 2：代入损失函数展开化简

将 $x = Qz$ 和 $A = Q\Lambda Q^T$ 代入原损失函数，逐步展开：

$$\begin{aligned} f(x) &= \frac{1}{2}(Qz)^T(Q\Lambda Q^T)(Qz) \\ &= \frac{1}{2}z^T Q^T \cdot Q\Lambda Q^T \cdot Qz \quad (\text{矩阵转置性质: } (AB)^T = B^T A^T) \\ &= \frac{1}{2}z^T (Q^T Q)\Lambda(Q^T Q)z \quad (\text{矩阵乘法结合律}) \\ &= \frac{1}{2}z^T I\Lambda I z \quad (\text{正交矩阵性质: } Q^T Q = I) \\ &= \frac{1}{2}z^T \Lambda z \quad (\text{单位矩阵性质: } IA = AI = A) \end{aligned}$$

步骤 3：拆分为独立一维项

由于 Λ 是对角矩阵， $z^T \Lambda z$ 可展开为各维度的独立求和形式：

$$z^T \Lambda z = \sum_{i=1}^d \lambda_i z_i^2$$

因此，损失函数最终可写为：

$$f(x) = \frac{1}{2} \sum_{i=1}^d \lambda_i z_i^2$$

本讲摘要

解耦的核心结论

- 新坐标系下的参数 $z_1, z_2, \dots, z_d$ 相互独立，无跨维度耦合项；
- 每个 z_i 对应一个独立的一维二次型，其曲率（等价于弹簧模型的劲度系数）就是对应特征值 λ_i ；
- 原本复杂的 d 维耦合二次型优化问题，被完全拆解为 d 个独立的一维优化问题。

四、梯度下降学习率安全上限的完整推导

基于特征值分解的解耦能力，我们完整推导二次型损失函数下，梯度下降（GD）算法的学习率安全收敛上限。

4.1 梯度下降的迭代式（矩阵形式）

首先，二次型损失函数 $f(x) = \frac{1}{2}x^T A x$ 的梯度，由矩阵求导法则可得：

$$\nabla f(x) = Ax$$

推导补充：标量对列向量求导， $\nabla(x^T A x) = (A + A^T)x$ ；由于 A 是实对称矩阵， $A^T = A$ ，因此 $\nabla(x^T A x) = 2Ax$ ，乘以系数 $\frac{1}{2}$ 后得到上述结果。

梯度下降的核心迭代逻辑是：沿梯度反方向更新参数，迭代式为：

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

其中 $\eta > 0$ 为学习率， x_t 为第 t 步迭代的参数值。

将梯度 $\nabla f(x_t) = Ax_t$ 代入，合并矩阵乘法，得到迭代式的矩阵形式：

$$\begin{aligned} x_{t+1} &= x_t - \eta Ax_t \\ &= (I - \eta A)x_t \end{aligned}$$

4.2 特征值分解解耦迭代式

对实对称矩阵 A 做特征值分解 $A = Q\Lambda Q^T$ ，代入迭代式；同时基于坐标旋转变换，令 $x_t = Qz_t$ ，代入得：

$$Qz_{t+1} = (I - \eta Q\Lambda Q^T)Qz_t$$

对右侧展开并逐步化简：

$$\begin{aligned} Qz_{t+1} &= I \cdot Qz_t - \eta Q\Lambda Q^T \cdot Qz_t \\ &= Qz_t - \eta Q\Lambda(Q^T Q)z_t \quad (\text{矩阵乘法结合律}) \\ &= Qz_t - \eta Q\Lambda I z_t \quad (\text{正交矩阵性质: } Q^T Q = I) \\ &= Qz_t - \eta Q\Lambda z_t \\ &= Q(I - \eta\Lambda)z_t \quad (\text{矩阵乘法分配律, 提取公共因子 } Q) \end{aligned}$$

对等式两边同时左乘 Q^T ，由 $Q^T Q = I$ ，最终得到解耦后的迭代式：

$$z_{t+1} = (I - \eta\Lambda)z_t$$

4.3 拆分为独立的一维迭代

由于 Λ 是对角矩阵， $I - \eta\Lambda$ 也是对角矩阵，其对角线元素为 $1 - \eta\lambda_1, 1 - \eta\lambda_2, \dots, 1 - \eta\lambda_d$ 。

对角矩阵与向量相乘仅在对应维度做标量乘法，因此迭代式可完全拆分为 d 个相互独立的一维迭代式，对任意 $i = 1, 2, \dots, d$ ，有：

$$z_{t+1,i} = (1 - \eta\lambda_i)z_{t,i}$$

其中 $z_{t,i}$ 是第 t 步迭代中 z_t 的第 i 维取值。

4.4 收敛性要求：一维迭代的收缩条件

梯度下降要收敛到全局最优点（二次型损失的最优点为 $x^* = 0$ ，对应 $z^* = 0$ ），必须保证每一步迭代后参数误差持续缩小，即 $z_{t,i}$ 随迭代次数 t 增加逐渐趋近于 0。

对一维迭代式做递推，可得：

$$z_{t,i} = (1 - \eta\lambda_i)^t z_{0,i}$$

其中 $z_{0,i}$ 是初始参数的第 i 维取值。

要让 $t \rightarrow \infty$ 时 $z_{t,i} \rightarrow 0$ ，必须满足：

$$|1 - \eta\lambda_i| < 1, \quad \forall i = 1, 2, \dots, d$$

该式为一维迭代的收缩条件：绝对值小于 1 时，每一步误差的绝对值都会缩小，最终收敛到 0；若绝对值 ≥ 1 ，误差会不衰减甚至放大，导致算法发散。

4.5 解不等式得到学习率安全上限

绝对值不等式 $|1 - \eta\lambda_i| < 1$ 可拆分为两个不等式同时成立：

不等式 1: $1 - \eta\lambda_i < 1$

移项化简:

$$-\eta\lambda_i < 0$$

由于 A 是正定矩阵, 所有特征值 $\lambda_i > 0$, 且学习率 $\eta > 0$, 因此该不等式恒成立, 无额外约束。

不等式 2: $1 - \eta\lambda_i > -1$

移项化简:

$$-\eta\lambda_i > -2$$

两边同时乘以-1 (不等号方向反转):

$$\eta\lambda_i < 2 \implies \eta < \frac{2}{\lambda_i}, \quad \forall i = 1, 2, \dots, d$$

最终安全上限

要让不等式对所有 i 都成立, η 必须小于所有 $\frac{2}{\lambda_i}$ 中的最小值。

由于特征值按从小到大排列 $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_d$, 最大特征值 $\lambda_{\max} = \lambda_d$, 因此 $\frac{2}{\lambda_{\max}}$ 是所有 $\frac{2}{\lambda_i}$ 中的最小值。

因此, 梯度下降收敛的学习率安全上限为:

$$\eta < \frac{2}{\lambda_{\max}}$$

理论最大安全学习率为 $\eta_{\max} = \frac{2}{\lambda_{\max}}$ 。

五、收敛因子与单学习率的局限

5.1 收敛因子的定义

定义 2 (收敛因子): 对于一维迭代式 $z_{t+1,i} = (1 - \eta\lambda_i)z_{t,i}$, 定义收敛因子 ρ_i 为:

$$\rho_i = |1 - \eta\lambda_i|$$

收敛因子的核心意义:

- ρ_i 越小, 每一步迭代中误差缩小的比例越大, 该维度的收敛速度越快;
- 若 $\rho_i < 1$, 该维度迭代收敛; 若 $\rho_i \geq 1$, 该维度迭代震荡甚至发散。

5.2 单学习率 GD 的核心局限

对于 d 维问题, 不同维度的特征值 λ_i 差异越大 (即矩阵 A 的条件数 $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ 越大), 单学习率的局限性越明显:

- 若学习率 η 过大, 会导致大 λ_i 对应的维度 $\rho_i \geq 1$, 算法发散;
- 若学习率 η 过小, 会导致小 λ_i 对应的维度 ρ_i 接近 1, 收敛速度极慢。

以二维问题为例, 两个维度的特征值为 k_1, k_2 ($k_1 \gg k_2$), 理论上的最优学习率为:

$$\eta^* = \frac{2}{k_1 + k_2}$$

该学习率是保证两个维度同步收敛的“黄金点”, 但当 k_1 和 k_2 差距过大时, 该最优学习率仍会导致收敛效率极低。

5.3 深度学习中的优化策略

上述单学习率的局限, 正是深度学习中 Warmup 策略与 Adam 等自适应优化器的核心价值所在:

- **Warmup 策略**：训练前期使用极小的学习率逐步“爬坡”，避免损失函数在非线性区域的曲率突变导致的迭代发散，待参数进入稳定区域后再提升学习率；
- **Adam 自适应优化器**：自动为不同参数维度分配独立的学习率，完美解决了不同维度曲率差异巨大导致的“顾此失彼”问题，大幅提升了复杂模型的收敛稳定性与速度。

六、落地：线性回归模型与损失函数推导

我们将上述抽象的二次型优化理论，完全落地到机器学习的线性回归模型，揭示数据统计特性与优化过程的内在联系。

6.1 线性回归模型定义

线性回归假设输入特征与输出标签之间存在线性关系，模型定义为：

$$y = x^T \theta^* + \varepsilon$$

各符号的定义与含义如下：

符号	维度与数学定义	核心含义
$x \in \mathbb{R}^d$	d 维列向量, $x = [x_1, x_2, \dots, x_d]^T$	模型的 d 维输入特征
$\theta^* \in \mathbb{R}^d$	d 维列向量, $\theta^* = [\theta_1^*, \theta_2^*, \dots, \theta_d^*]^T$	数据生成的真实参数（模型的“标准答案”）
$x^T \theta^*$	标量（向量内积）, $x^T \theta^* = \sum_{i=1}^d x_i \theta_i^*$	模型的理想预测值
ε	随机变量, $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$, 与 x 独立	观测噪声（测量误差、数据随机波动）
x 的分布	d 维正态分布 $x \sim \mathcal{N}(0, \Sigma)$, 协方差矩阵 $\Sigma = \text{diag}(\lambda_1, \dots, \lambda_d)$	输入特征的统计分布

6.2 总体损失函数定义

我们定义线性回归的总体损失函数为均方误差的期望：

$$L(a) = \frac{1}{2} \mathbb{E} [(y - x^T a)^2]$$

其中：

- $a \in \mathbb{R}^d$ ：对真实参数 θ^* 的估计值，即模型需要训练的参数；
- $y - x^T a$ ：预测误差（真实标签 y 减去模型预测值 $x^T a$ ）；
- $\mathbb{E}[\cdot]$ ：对所有可能的输入 x 和噪声 ε 取数学期望，即全量数据的平均损失；
- 系数 $\frac{1}{2}$ 用于后续求导时抵消平方项系数，简化计算。

6.3 损失函数的完整展开推导

步骤 1：预测误差的变形

将模型定义 $y = x^T \theta^* + \varepsilon$ 代入预测误差，进行变形：

$$\begin{aligned} y - x^T a &= x^T \theta^* + \varepsilon - x^T a \\ &= x^T (\theta^* - a) + \varepsilon \end{aligned}$$

定义**参数误差向量** $e = a - \theta^*$ （e 是确定量，仅与参数有关，非随机变量），因此 $\theta^* - a = -e$ ，代入上式得：

$$y - x^T a = -x^T e + \varepsilon$$

步骤 2：平方项展开

用完全平方公式展开平方项：

$$\begin{aligned} (y - x^T a)^2 &= (-x^T e + \varepsilon)^2 \\ &= (x^T e)^2 - 2\varepsilon x^T e + \varepsilon^2 \end{aligned}$$

补充: $x^T e$ 是向量内积, 结果为标量, 因此平方是标量的平方运算。

步骤 3: 对平方项取期望

利用期望的线性性与独立性性质拆分:

$$\mathbb{E}[(y - x^T a)^2] = \mathbb{E}[(x^T e)^2] - 2\mathbb{E}[\varepsilon x^T e] + \mathbb{E}[\varepsilon^2]$$

对三项分别化简:

1. 第二项 $\mathbb{E}[\varepsilon x^T e]$: ε 与 x 独立, e 是确定量, 因此 $\mathbb{E}[\varepsilon x^T e] = \mathbb{E}[\varepsilon] \cdot \mathbb{E}[x^T e]$; 而噪声均值 $\mathbb{E}[\varepsilon] = 0$, 因此该项整体为 0。
2. 第三项 $\mathbb{E}[\varepsilon^2]$: 由于 $\mathbb{E}[\varepsilon] = 0$, 噪声方差 $\text{Var}(\varepsilon) = \mathbb{E}[\varepsilon^2] = \sigma_\varepsilon^2$, 因此该项等于 σ_ε^2 。

代入化简后得到:

$$\mathbb{E}[(y - x^T a)^2] = \mathbb{E}[(x^T e)^2] + \sigma_\varepsilon^2$$

步骤 4: 处理 $\mathbb{E}[(x^T e)^2]$ 项

将内积展开为求和形式:

$$x^T e = \sum_{i=1}^d x_i e_i$$

平方后展开为二重求和:

$$(x^T e)^2 = \left(\sum_{i=1}^d x_i e_i \right)^2 = \sum_{i=1}^d \sum_{j=1}^d x_i x_j e_i e_j$$

对其取期望, 利用输入特征协方差矩阵的对角性化简:

我们假设输入特征的协方差矩阵 $\Sigma = \text{diag}(\lambda_1, \dots, \lambda_d)$ 是对角矩阵, 即不同特征相互独立, 因此:

$$\mathbb{E}[x_i x_j] = \begin{cases} \lambda_i, & i = j \\ 0, & i \neq j \end{cases}$$

- $i = j$ 时, $\mathbb{E}[x_i x_j] = \mathbb{E}[x_i^2] = \text{Var}(x_i) = \lambda_i$, 即第 i 个特征的方差;
- $i \neq j$ 时, $\mathbb{E}[x_i x_j] = \text{Cov}(x_i, x_j) = 0$, 即不同特征间无相关性。

因此所有交叉项 ($i \neq j$) 的期望全部为 0, 只剩下自身项:

$$\begin{aligned} \mathbb{E}[(x^T e)^2] &= \sum_{i=1}^d \sum_{j=1}^d \mathbb{E}[x_i x_j] e_i e_j \\ &= \sum_{i=1}^d \mathbb{E}[x_i^2] e_i^2 \\ &= \sum_{i=1}^d \lambda_i e_i^2 \end{aligned}$$

步骤 5: 最终损失函数与二次型的对应

将上述结果代回损失函数, 代入 $e_i = a_i - \theta_i^*$, 得到最终总体损失函数:

$$L(a) = \frac{1}{2} \sum_{i=1}^d \lambda_i (a_i - \theta_i^*)^2 + \frac{1}{2} \sigma_\varepsilon^2$$

将其与抽象二次型弹簧模型做对应, 如下表:

抽象二次型（弹簧模型）	线性回归损失	核心含义对应
$f(x) = \frac{1}{2} \sum_{i=1}^d k_i x_i^2$	$L(a) = \frac{1}{2} \sum_{i=1}^d \lambda_i (a_i - \theta_i^*)^2 + \text{常数}$	结构完全一致，均为独立一维二次型的求和
k_i （弹簧劲度系数/曲率）	λ_i （特征方差/协方差矩阵特征值）	损失函数的曲率完全由数据的特征方差决定
x_i （参数偏离最优点的误差）	$a_i - \theta_i^*$ （参数估计误差）	均为参数与全局最优点的偏差
最优解 $x^* = 0$	最优解 $a = \theta^*$	误差为 0 时，损失函数取得最小值

补充说明：损失函数中的 $\frac{1}{2}\sigma_\epsilon^2$ 是不可消除的噪声下界，与待优化参数 a 无关，因此对 a 求梯度时导数为 0，不影响参数优化过程。

七、线性回归梯度 Lipschitz 连续性与学习率的核心推导（数据决定论）

本部分从定义出发，严格推导线性回归梯度的 Lipschitz 连续性，最终得出核心结论：线性回归中，学习率的安全取值范围完全由输入数据的特征矩阵决定，与参数、标签无关。

7.1 基础定义回顾

7.1.1 梯度 Lipschitz 连续的核心定义

对于可微函数 $f(\theta)$ ，若其梯度 $\nabla f(\theta)$ 满足：对任意两个参数向量 $\theta_1, \theta_2 \in \mathbb{R}^d$ ，都有

$$\|\nabla f(\theta_1) - \nabla f(\theta_2)\|_2 \leq L \cdot \|\theta_1 - \theta_2\|_2$$

则称 $\nabla f(\theta)$ 是 **L-Lipschitz 连续**的，其中常数 $L > 0$ 称为 Lipschitz 常数。
Lipschitz 常数 L 的物理意义：梯度随参数变化的最大“剧烈程度”， L 越大，梯度随参数变化的速度越快，损失函数的曲率越大。

7.1.2 线性回归的有限样本损失与梯度

实际训练中使用有限样本的经验损失，给定 n 个样本组成的特征矩阵 $X \in \mathbb{R}^{n \times d}$ （每行是一个样本的 d 维特征），标签向量 $y \in \mathbb{R}^n$ ，线性回归的平方误差损失函数为：

$$L(\theta) = \frac{1}{2} \|y - X\theta\|_2^2$$

对参数 θ 求梯度，得到：

$$\nabla L(\theta) = X^T(X\theta - y)$$

7.1.3 核心前置概念补充：范数、奇异值与谱范数

在展开 Lipschitz 连续性的推导前，我们先严格阐明推导中涉及的所有核心概念，确保后续推导的每一步都无知识盲区，且与前文的实对称矩阵、特征值分解内容完全衔接。

向量 2-范数（欧几里得范数）

矩阵的谱范数由向量 2-范数诱导而来，是所有推导的基础。

定义（向量 2-范数）：对于任意 d 维列向量 $v \in \mathbb{R}^d$ ，其 2-范数（欧几里得范数）定义为：

$$\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_d^2} = \sqrt{v^T v}$$

核心含义：向量 v 在 d 维欧式空间中的模长（几何长度），是日常所说“距离”的严格数学定义。

矩阵的诱导范数（算子范数）

定义（矩阵的诱导范数）：对于任意矩阵 $A \in \mathbb{R}^{m \times n}$ ，由向量范数诱导的矩阵范数定义为：

$$\|A\| = \max_{\substack{v \in \mathbb{R}^n \\ v \neq 0}} \frac{\|Av\|}{\|v\|}$$

核心含义：矩阵 A 对任意非零向量 v 的**最大拉伸倍数**。矩阵作用在向量上会改变向量的长度，诱导范数就是所有可能的向量中，拉伸倍数的最大值，是衡量矩阵对向量缩放能力的核心指标。

等价简化定义： $\|A\| = \max_{\|v\|=1} \|Av\|$ ，即仅考虑单位长度的向量，计算逻辑完全一致。

谱范数（矩阵 2-范数）

定义（谱范数/矩阵 2-范数）：由向量 2-范数诱导的矩阵范数，称为矩阵的谱范数，记为 $\|A\|_2$ ，严格定义为：

$$\|A\|_2 = \max_{\substack{v \in \mathbb{R}^n \\ v \neq 0}} \frac{\|Av\|_2}{\|v\|_2} = \max_{\|v\|_2=1} \|Av\|_2$$

核心性质：谱范数完美衔接了矩阵的代数性质与几何意义，是后续推导的核心工具，也是机器学习优化理论中最常用的矩阵范数。

格拉姆矩阵（Gram Matrix）及其核心性质

定义（格拉姆矩阵）：对于任意特征矩阵 $X \in \mathbb{R}^{n \times d}$ ，其格拉姆矩阵定义为 $X^T X$ ，是一个 $d \times d$ 的方阵。

核心性质（与前文实对称矩阵定理完全衔接）：

1. **实对称性：** $(X^T X)^T = X^T (X^T)^T = X^T X$ ，因此 $X^T X$ 是实对称矩阵；
2. **半正定性：**对任意非零向量 $v \in \mathbb{R}^d$ ，有 $v^T (X^T X) v = (Xv)^T (Xv) = \|Xv\|_2^2 \geq 0$ ，因此 $X^T X$ 是半正定矩阵；
3. **可正交对角化：**根据前文的实对称矩阵特征值分解定理， $X^T X$ 一定可以分解为：

$$X^T X = Q \Lambda Q^T$$

其中 Q 是 $d \times d$ 正交矩阵， $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ 是对角矩阵，对角线元素 λ_i 是 $X^T X$ 的特征值，且由半正定性可知，所有 $\lambda_i \geq 0$ 。

奇异值的定义

定义（矩阵的奇异值）：对于任意矩阵 $X \in \mathbb{R}^{n \times d}$ ，其奇异值 σ_i 定义为其格拉姆矩阵 $X^T X$ 的特征值的非负平方根，即：

$$\sigma_i = \sqrt{\lambda_i(X^T X)}, \quad i = 1, 2, \dots, d$$

其中 $\lambda_i(X^T X)$ 是 $X^T X$ 的第 i 个特征值。

核心说明：

- 特征值仅对方阵有定义，而奇异值对任意形状的矩阵（包括非方阵）都有定义；
- 对于实对称矩阵，其奇异值等于其特征值的绝对值；
- 矩阵的最大奇异值 $\sigma_{\max}(X) = \sqrt{\lambda_{\max}(X^T X)}$ ，对应矩阵对向量的最大拉伸倍数，与谱范数直接相关。

核心等式的严格证明与含义

核心等式：对于任意矩阵 $X \in \mathbb{R}^{n \times d}$ ，其格拉姆矩阵 $X^T X$ 的谱范数满足：

$$\|X^T X\|_2 = \lambda_{\max}(X^T X)$$

即：**实对称半正定矩阵的谱范数，等于其最大特征值。**

严格证明

：

1. 由谱范数的定义， $\|X^T X\|_2 = \max_{\|v\|_2=1} \|(X^T X)v\|_2$ ；
2. 由于 $X^T X$ 是实对称矩阵，根据前文的特征值分解定理， $X^T X = Q \Lambda Q^T$ ，其中 Q 是正交矩阵， $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ ，且 $0 \leq \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_d = \lambda_{\max}$ ；

3. 对于任意单位向量 $\|v\|_2 = 1$ ，令 $z = Q^T v$ ，由正交矩阵的保范性（正交变换不改变向量模长），有 $\|z\|_2 = \|Q^T v\|_2 = \|v\|_2 = 1$ ，即 z 也是单位向量；

4. 将特征值分解代入 $(X^T X)v$ ：

$$(X^T X)v = Q\Lambda Q^T v = Q\Lambda z$$

5. 计算其 2-范数，再次利用正交矩阵的保范性：

$$\|(X^T X)v\|_2 = \|Q\Lambda z\|_2 = \|\Lambda z\|_2$$

6. 由于 Λ 是对角矩阵， $\Lambda z = [\lambda_1 z_1, \lambda_2 z_2, \dots, \lambda_d z_d]^T$ ，因此其 2-范数为：

$$\|\Lambda z\|_2 = \sqrt{\sum_{i=1}^d (\lambda_i z_i)^2}$$

7. 我们需要在 $\|z\|_2 = 1$ （即 $\sum_{i=1}^d z_i^2 = 1$ ）的约束下，最大化上述表达式。由于所有 $\lambda_i \geq 0$ ，且 $\lambda_d = \lambda_{\max}$

是最大的特征值，因此当且仅当 $z_d = 1$ 、其余 $z_i = 0$ 时， $\sqrt{\sum_{i=1}^d (\lambda_i z_i)^2}$ 取得最大值 $\lambda_{\max}$ ；

8. 因此：

$$\max_{\|v\|_2=1} \|(X^T X)v\|_2 = \lambda_{\max}(X^T X)$$

即 $\|X^T X\|_2 = \lambda_{\max}(X^T X)$ ，证毕。

核心含义：

这个等式将矩阵的谱范数（对向量的最大拉伸能力）与矩阵的特征值直接关联，是连接线性代数基础与优化理论的关键桥梁。对于实对称矩阵，我们可以直接通过其最大特征值得到其谱范数，无需复杂的范数计算，为后续 Lipschitz 常数的推导提供了核心依据。

7.2 严格推导：线性回归梯度满足 Lipschitz 条件

基于上述阐明的所有概念，我们分 3 步严格推导线性回归梯度的 Lipschitz 连续性，每一步均标注核心依据，确保逻辑完全闭环。

步骤 1：计算任意两个参数的梯度差值

目标：推导梯度差值与参数差值的线性关系，消除标签 y 的影响。

任取两个参数向量 $\theta_1, \theta_2 \in \mathbb{R}^d$ ，分别计算其梯度，再求差值：

$$\begin{aligned} \nabla L(\theta_1) - \nabla L(\theta_2) &= X^T(X\theta_1 - y) - X^T(X\theta_2 - y) \quad (\text{代入线性回归梯度公式}) \\ &= X^T X\theta_1 - X^T y - X^T X\theta_2 + X^T y \quad (\text{矩阵乘法分配律展开}) \\ &= X^T X\theta_1 - X^T X\theta_2 \quad (-X^T y \text{ 与 } +X^T y \text{ 相互抵消}) \\ &= X^T X(\theta_1 - \theta_2) \quad (\text{矩阵乘法分配律，提取公共因子 } X^T X) \end{aligned}$$

步骤 1 核心结论

:

$$\nabla L(\theta_1) - \nabla L(\theta_2) = X^T X(\theta_1 - \theta_2)$$

梯度差值仅由特征矩阵的格拉姆矩阵 $X^T X$ 和参数差值决定，与标签 y 完全无关，为后续范数放缩奠定基础。

步骤 2：对梯度差值取 2-范数，应用谱范数不等式放缩

目标：利用谱范数的核心性质，建立梯度差值范数与参数差值范数的不等式关系。
首先，对步骤 1 得到的等式两边同时取向量 2-范数，由于等式两边是完全相等的向量，因此范数也完全相等：

$$\|\nabla L(\theta_1) - \nabla L(\theta_2)\|_2 = \|X^T X(\theta_1 - \theta_2)\|_2$$

接下来，我们应用**谱范数的核心不等式性质**（由诱导范数的定义直接推导得出）：

备注

[定理] 对于任意矩阵 $A \in \mathbb{R}^{m \times n}$ 和任意向量 $v \in \mathbb{R}^n$ ，有

$$\|Av\|_2 \leq \|A\|_2 \cdot \|v\|_2$$

证明：当 $v = 0$ 时，等式两边均为 0，成立；当 $v \neq 0$ 时，由谱范数的定义 $\|A\|_2 = \max_{v \neq 0} \frac{\|Av\|_2}{\|v\|_2}$ ，可知对任意非零 v ，都有 $\frac{\|Av\|_2}{\|v\|_2} \leq \|A\|_2$ ，两边乘以 $\|v\|_2$ 即得上述不等式。

令上述定理中的 $A = X^T X$ ， $v = \theta_1 - \theta_2$ ，代入不等式得：

$$\|X^T X(\theta_1 - \theta_2)\|_2 \leq \|X^T X\|_2 \cdot \|\theta_1 - \theta_2\|_2$$

结合之前的范数等式，我们得到：

$$\|\nabla L(\theta_1) - \nabla L(\theta_2)\|_2 \leq \|X^T X\|_2 \cdot \|\theta_1 - \theta_2\|_2$$

步骤 3：代入核心等式，得到 Lipschitz 常数

目标：将谱范数替换为已证明的特征值形式，得到最终的 Lipschitz 常数。
根据严格证明的核心等式：

$$\|X^T X\|_2 = \lambda_{\max}(X^T X)$$

其中 $\lambda_{\max}(X^T X)$ 是格拉姆矩阵 $X^T X$ 的最大特征值。

将该等式代入步骤 2 得到的不等式中，得到：

$$\|\nabla L(\theta_1) - \nabla L(\theta_2)\|_2 \leq \lambda_{\max}(X^T X) \cdot \|\theta_1 - \theta_2\|_2$$

最终推导结论

对比**梯度 Lipschitz 连续的核心定义**：

$$\|\nabla f(\theta_1) - \nabla f(\theta_2)\|_2 \leq L \cdot \|\theta_1 - \theta_2\|_2$$

我们可以直接得出：

1. 线性回归的梯度 $\nabla L(\theta)$ 满足 Lipschitz 连续条件；
2. 其 Lipschitz 常数为：

$$L = \lambda_{\max}(X^T X)$$

3. 该常数仅由输入特征矩阵 X 决定，与待优化参数 θ 、标签 y 完全无关，完美印证了“数据决定论”的核心结论。

理论闭环补充：这里得到的 Lipschitz 常数 $L = \lambda_{\max}(X^T X)$ ，与推导二次型损失函数学习率上限时的 $\lambda_{\max}(A)$ 完全对应——线性回归的损失函数本质上就是以 $X^T X$ 为 Hessian 矩阵的二次型损失，因此前后结论完全一致，整个理论体系完全闭环。

7.3 Lipschitz 常数 $\rightarrow$ 学习率的收敛条件

梯度下降的更新规则：

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

收敛的学习率上限推导

设最优解为 θ^* (满足 $\nabla L(\theta^*) = 0$)，定义误差向量：

$$e_t = \theta_t - \theta^*$$

将更新规则代入误差向量：

$$\begin{aligned} e_{t+1} &= \theta_{t+1} - \theta^* \\ &= \theta_t - \eta \nabla L(\theta_t) - \theta^* \\ &= e_t - \eta [\nabla L(\theta_t) - \nabla L(\theta^*)] \quad (\nabla L(\theta^*) = 0) \end{aligned}$$

结合 Lipschitz 条件 $\|\nabla L(\theta_t) - \nabla L(\theta^*)\| \leq L\|e_t\|$ ，通过范数放缩可证明：

当且仅当学习率满足：

$$0 < \eta < \frac{2}{L} = \frac{2}{\lambda_{\max}(X^T X)}$$

梯度下降一定收敛；若 $\eta \geq \frac{2}{L}$ ，迭代会震荡甚至发散。

八、核心结论的适用范围分析

我们先回顾线性回归的核心结论：当模型的损失函数是凸函数、梯度 Lipschitz 连续，且 Lipschitz 常数仅由输入数据决定时，学习率的安全上限由数据的固有属性决定。

下面分不同模型类型，分析该结论的适用性：

8.1 分模型适用性分析

1. 广义线性模型 (GLM)：部分适用

广义线性模型(如逻辑回归、泊松回归)的损失函数仍为凸函数，梯度满足 Lipschitz 连续，但 Lipschitz 常数的计算方式与线性回归不同。以逻辑回归为例，其 Lipschitz 常数为 $L = \frac{1}{4n} \lambda_{\max}(X^T X)$ ，仍与 $X^T X$ 的最大特征值直接相关，仅系数不同，因此核心结论部分适用。

2. 神经网络 (深度学习)：不适用

神经网络的损失函数为非凸函数，梯度的 Lipschitz 常数不仅与数据有关，还与模型结构、参数初始化、权重大小密切相关，且随参数动态变化，不存在全局固定常数。因此“学习率仅由数据决定”的结论完全不适用。

3. 树模型 (决策树、随机森林、XGBoost)：不适用

树模型的训练基于分裂准则，而非梯度下降，不存在梯度下降意义上的“学习率”概念，因此核心结论完全不适用。

4. 支持向量机 (SVM)：部分适用

SVM 的损失函数是凸函数，梯度满足 Lipschitz 连续，且 Lipschitz 常数与数据的核矩阵/特征矩阵的最大特征值直接相关，因此核心结论部分适用。

5. 非凸模型 (矩阵分解、推荐系统等)：不适用

这类模型的损失函数为非凸函数，梯度的 Lipschitz 常数随参数动态变化或不存在，学习率需结合训练动态调整，因此核心结论完全不适用。

8.2 核心适用性总结表

模型类型	损失函数性质	梯度 Lipschitz 连续性	学习率是否仅由数据决定	核心结论适用性
线性回归	凸函数	是（全局固定常数）	是	完全适用
逻辑回归	凸函数	是（全局固定常数）	是（系数不同）	部分适用
神经网络	非凸函数	是（随参数动态变化）	否（与模型结构强相关）	不适用
树模型	无梯度下降框架	无	无对应学习率概念	不适用
线性 SVM	凸函数	是（全局固定常数）	是	部分适用
非凸低秩模型	非凸函数	否/随参数变化	否	不适用

最终适用性结论：
线性回归的“学习率仅由数据决定”的核心结论，仅适用于**损失函数为凸函数、梯度 Lipschitz 连续且 Lipschitz 常数仅由输入数据决定**的模型；对于非凸模型、无梯度下降框架的模型，该结论完全不适用。

九、最终结论：数据归一化的核心意义与 SGD 的引入

9.1 数据归一化/标准化的核心重要性

通过上述完整的推导与分析，我们得出核心认知：**对于线性回归等凸优化模型，数据本身的统计特性，直接决定了优化过程的收敛性与学习率的上限，对数据的预处理往往比模型调参更加重要。**

数据归一化（Min-Max 归一化）、标准化（Z-Score 标准化）的核心价值，正是通过线性变换调整输入特征的统计特性：

- 统一特征的数值尺度，让所有特征的取值处于同一量级，消除因特征尺度差异导致的 $\lambda_{\max}$ 过大问题；
- 降低特征间的相关性，让 $X^T X$ 的特征值分布更集中，减小矩阵条件数，大幅提升梯度下降的收敛速度与稳定性；
- 提升学习率的安全上限，允许使用更大的学习率，加快模型训练收敛。

这正是机器学习实践中，数据预处理是建模流程最关键环节的核心原因。

9.2 批量梯度下降（GD）的局限与随机梯度下降（SGD）的引入

上述推导的批量梯度下降（GD），在实际应用中存在显著局限：

- 计算成本极高：每次迭代都需要使用全量数据计算梯度，矩阵乘法、特征值分解等操作的时间复杂度高达 $O(nd^2)$ 与 $O(d^3)$ ，当样本量 n 和特征维度 d 极大时，计算开销完全无法承受；
- 内存占用巨大：全量特征矩阵需要全部加载到内存中，超大规模数据集无法满足硬件要求。

为解决批量梯度下降的计算效率问题，我们引入**随机梯度下降（Stochastic Gradient Descent, SGD）**：

SGD 的核心思想是，每次迭代仅使用单个样本或一小批样本（Mini-Batch）计算梯度的无偏估计，替代全量数据的精确梯度，在保证收敛性的同时，大幅降低计算与内存开销，让超大规模数据集的模型训练成为可能。

2.3.2 SGD 的概率论基础

为深入理解随机梯度下降（SGD）中梯度的无偏估计及随机性对优化的影响，我们在此回顾概率论的核心基础概念，包括期望、条件密度、条件期望与协方差。这些概念为理解“为什么 SGD 的噪声不仅没有毁掉训练，反而帮助模型逃离局部最优”奠定了数学基础。

一、数学期望 (Expected Value)

数学期望（简称期望）是随机变量取值的“加权平均”，反映了随机变量在长期的随机试验中取

值的平均大小。

1.1 离散型随机变量的期望

对于离散型随机变量 X ，假设其可能取值为 $x_1, x_2, \dots, x_n$ ，对应的概率为 $P(X = x_i) = p_i$ ，则其期望定义为：

$$E(X) = \sum_i x_i p_i$$

1.2 概率密度函数 (Probability Density Function, PDF)

在过渡到连续型随机变量之前，我们必须先引入**概率密度函数 (PDF)** 的概念。与离散型变量在特定点上有明确的概率不同，连续型变量在任意某个确切点上的概率为 0。因此，我们用概率密度函数 $f(x)$ 来描述变量在某个区间内取值的“密集程度”。

备注

[性质与物理意义]

- **性质**： $f(x) \geq 0$ ，且它在整个实数轴上的积分（即总概率）必须等于 1： $\int_{-\infty}^{\infty} f(x) dx = 1$ 。
- **物理意义**：变量 X 落在区间 $[a, b]$ 的概率即为该区间下密度曲线的面积： $P(a \leq X \leq b) = \int_a^b f(x) dx$ 。

1.2.1 例子：正态分布 (Normal Distribution)

正态分布是最重要的连续型分布之一。若 X 服从均值为 μ ，方差为 σ^2 的正态分布，记作 $X \sim \mathcal{N}(\mu, \sigma^2)$ ，其概率密度函数为：

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

由于其钟形曲线的对称性，正态分布的期望就是其对称中心，即 $E(X) = \mu$ 。

1.3 连续型随机变量的期望

借助概率密度的概念，连续型随机变量 X 的期望定义为各个取值 x 乘以其“权重”（密度 $f(x)$ ）在整个实数域上的积分：

$$E(X) = \int_{-\infty}^{\infty} x f(x) dx$$

二、联合密度与条件密度

在研究两个或多个随机变量（如 X 和 Y ）的关系时，我们需要引入联合分布和条件分布。

2.1 联合概率密度 (Joint Density)

联合密度 $f(x, y)$ 描述的是两个连续型随机变量 X 和 Y **同时**取某一对值的概率分布情况。

- 变量 (X, Y) 落在某个二维区域 D 内的概率等于联合密度在该区域上的二重积分： $P((X, Y) \in D) = \iint_D f(x, y) dx dy$ 。
- 整个平面的积分为 1： $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) dx dy = 1$ 。

2.2 条件概率密度 (Conditional Density)

条件密度描述的是：在已知其中一个变量（如 $Y = y$ ）发生的前提下，另一个变量（如 X ）的概率分布情况。记作 $f_{X|Y}(x|y)$ ，其计算公式为联合密度除以 Y 的边缘密度：

$$f_{X|Y}(x|y) = \frac{f(x, y)}{f_Y(y)}$$

(前提是边缘密度 $f_Y(y) > 0$)

三、条件期望 (Conditional Expectation)

条件期望是指在已知某个随机变量取特定值的情况下，另一个随机变量的平均取值。

3.1 离散型条件期望

已知 $Y = y_j$ ，离散变量 X 的条件期望公式为：

$$E(X|Y = y_j) = \sum_i x_i P(X = x_i|Y = y_j)$$

其中 $P(X = x_i|Y = y_j) = \frac{P(X = x_i, Y = y_j)}{P(Y = y_j)}$ 是离散形式的条件概率。

3.2 连续型条件期望

结合前面讲到的条件密度 $f_{X|Y}(x|y)$ ，连续变量的条件期望公式为：

$$E(X|Y = y) = \int_{-\infty}^{\infty} x f_{X|Y}(x|y) dx$$

四、联合期望 $E(XY)$ 的计算

$E(XY)$ 是随机变量 X 和 Y 乘积的数学期望，它是计算协方差的关键基础。

4.1 离散型 $E(XY)$

把所有可能的 (x_i, y_j) 乘积，乘以它们同时发生的联合概率 $P(X = x_i, Y = y_j)$ ，然后求和：

$$E(XY) = \sum_i \sum_j x_i y_j P(X = x_i, Y = y_j)$$

4.2 连续型 $E(XY)$

利用联合密度 $f(x, y)$ ，在二维空间上进行二重积分：

$$E(XY) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} xy f(x, y) dx dy$$

五、协方差 (Covariance)

期望只描述了单一变量的平均水平，而**协方差**用于衡量两个随机变量 X 和 Y 之间的**线性相关程度**与协同变动趋势。

5.1 协方差的定义

协方差定义为变量分别减去其期望后乘积的期望：

$$Cov(X, Y) = E[(X - E(X))(Y - E(Y))]$$

5.2 协方差的重要计算性质（展开公式）

利用期望的线性性质，我们可以将上述定义公式展开：

$$\begin{aligned} Cov(X, Y) &= E(XY - XE(Y) - YE(X) + E(X)E(Y)) \\ &= E(XY) - E(X)E(Y) - E(Y)E(X) + E(X)E(Y) \end{aligned}$$

最终得到协方差最常用的计算公式：

本讲摘要

[协方差的计算公式]

$$\text{Cov}(X, Y) = E(XY) - E(X)E(Y)$$

5.3 独立性与协方差为 0

定理：如果随机变量 X 和 Y 相互独立，那么它们的协方差等于 0（即 $\text{Cov}(X, Y) = 0$ ）。

分析

[定理证明逻辑与推导]

1. 如果 X 和 Y 独立，意味着它们的联合分布等于边缘分布的乘积。

- 离散情况下： $P(X = x, Y = y) = P(X = x)P(Y = y)$
- 连续情况下： $f(x, y) = f_X(x)f_Y(y)$

2. 代入 $E(XY)$ 的连续型（或离散型）公式中：

$$E(XY) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} xyf_X(x)f_Y(y)dxdy = \left(\int_{-\infty}^{\infty} xf_X(x)dx \right) \left(\int_{-\infty}^{\infty} yf_Y(y)dy \right)$$

3. 即得出独立变量的一个核心性质： $E(XY) = E(X)E(Y)$ 。

4. 将此代入协方差的性质公式：

$$\text{Cov}(X, Y) = E(XY) - E(X)E(Y) = E(X)E(Y) - E(X)E(Y) = 0$$

（注：反之不一定成立。协方差为 0 只能说明两者没有**线性**相关性，并不能绝对保证两者完全独立，除非它们是联合正态分布。）

2.3.3 SGD 理论拆解

全梯度下降（BGD）的核心问题

全梯度（批量梯度下降 BGD）的定义：

$$\nabla L(a) = \mathbb{E}[(y - x^\top a)x]$$

全梯度需要对整个数据分布取期望，在实际工程与科研场景中存在两大核心瓶颈：

1. **计算量不可行：**当数据量极大（如深度学习场景的亿级样本）时，遍历所有样本计算全梯度的计算量为 $O(n)$ （ n 为样本总量），算力开销无法承受；
2. **分布无法获取：**真实场景中数据的整体分布往往是未知的，仅能通过有限样本对全梯度进行估计。

SGD 的核心思想

SGD（随机梯度下降）的核心突破，是用单个样本（或一小批样本，mini-batch）的梯度代替全梯度进行参数更新，核心优势与代价如下：

- **计算量大幅压缩：**每步迭代的计算量从 $O(n)$ 降至 $O(B)$ （ B 为 batch size，批大小），让大规模数据下的模型优化成为可能；
- **固有代价：**引入了梯度噪声（用样本梯度估计全梯度时存在的统计误差），即 mini-batch 梯度的方差，该噪声的结构与影响将在后续详细分析。

单样本随机梯度的推导与核心性质

本部分基于线性回归模型完成单样本随机梯度的完整推导，并证明其无偏性、计算梯度方差，是 SGD 理论的核心基础。

瞬时损失定义

给定单样本 (x_t, y_t) ，其中线性回归的真实数据生成模型为：

$$y_t = x_t^\top \theta^* + \varepsilon_t$$

其中 θ^* 为模型的最优参数， ε_t 为该样本的观测噪声。

定义单样本瞬时损失为：

$$\ell_t(a) = \frac{1}{2}(y_t - x_t^\top a)^2 \quad (21)$$

- **含义：**单样本预测误差平方的 1/2，对应总体损失（全样本期望）的单样本实现，是 SGD 迭代中使用的损失函数；
- **线性代数说明：** $x_t^\top a$ 为向量内积，输出为标量，因此瞬时损失为标量函数。

单样本损失对参数的偏导推导

我们需要求解单样本损失对第 i 个参数的偏导，即随机梯度的第 i 个分量 $g_{t,i} = \frac{\partial \ell_t}{\partial a_i}$ ，完整推导步骤如下：

步骤 1：链式法则求导

令 $u = y_t - x_t^\top a$ ，则 $\ell_t(a) = \frac{1}{2}u^2$ ，根据链式法则：

$$\frac{\partial \ell_t}{\partial a_i} = u \cdot \frac{\partial u}{\partial a_i} = -(y_t - x_t^\top a_t) \cdot x_{t,i} \quad (22)$$

步骤 2：代入真实数据生成模型

将 $y_t = x_t^\top \theta^* + \varepsilon_t$ 代入式 (22)，用参数差表示预测误差：

$$\frac{\partial \ell_t}{\partial a_i} = -(x_t^\top \theta^* + \varepsilon_t - x_t^\top a_t) \cdot x_{t,i} \quad (23)$$

步骤 3：合并同类项

提取公因子 $x_t^\top$ ，合并参数差项：

$$\frac{\partial \ell_t}{\partial a_i} = -(x_t^\top (\theta^* - a_t) + \varepsilon_t) \cdot x_{t,i} \quad (24)$$

其中 $x_t^\top \theta^* - x_t^\top a_t = x_t^\top (\theta^* - a_t)$ 。

步骤 4：展开内积，拆分交叉项

定义参数误差 $e_{t,i} = a_{t,i} - \theta_i^*$ （即 $\theta_i^* - a_{t,i} = -e_{t,i}$ ），代入式 (24) 展开内积：

$$\frac{\partial \ell_t}{\partial a_i} = (a_{t,i} - \theta_i^*)x_{t,i}^2 + \sum_{j \neq i} (a_{t,j} - \theta_j^*)x_{t,i}x_{t,j} - \varepsilon_t x_{t,i} \quad (25)$$

展开逻辑： $-\left(-\sum_{j=1}^d x_{t,j}e_{t,j} + \varepsilon_t\right)x_{t,i} = e_{t,i}x_{t,i}^2 + \sum_{j \neq i} e_{t,j}x_{t,i}x_{t,j} - \varepsilon_t x_{t,i}$ ，其中 d 为参数维度。

步骤 5：利用协方差矩阵对角性简化

当输入特征的协方差矩阵 Σ 为对角阵时， $x_{t,i}$ 与 $x_{t,j} (i \neq j)$ 相互独立，满足 $\mathbb{E}[x_{t,i}x_{t,j}] = 0$ ，交叉项的期望为 0，因此随机梯度可简化为：

$$g_{t,i} = e_{t,i} \cdot x_{t,i}^2 - \varepsilon_t \cdot x_{t,i} \quad (26)$$

简化后仅保留自身参数项与噪声项，交叉项可忽略，为后续无偏性证明奠定基础。

单样本随机梯度的无偏性证明

核心目标：证明单样本随机梯度的期望等于全梯度（真实梯度），即 $\mathbb{E}[g_t] = \nabla L(a_t)$ ，这是 SGD 能够收敛的核心前提。

备注

[定理 4.4.1 (SGD 无偏性)] 单样本随机梯度 g_t 是全梯度 $\nabla L(a_t)$ 的无偏估计，即：

$$\mathbb{E}[g_t] = \nabla L(a_t)$$

该性质是 SGD 能收敛到全局最优解的核心理论保证。

分析

[完整证明过程] **步骤 1：计算条件期望 $\mathbb{E}[g_{t,i}|e_{t,i}]$**

给定当前参数误差 $e_{t,i}$ （确定量，随机源仅为 $x_{t,i}$ 与 ε_t ），对简化后的随机梯度式 (26) 取条件期望：

$$\mathbb{E}[g_{t,i}|e_{t,i}] = \mathbb{E}\left[e_{t,i} \cdot x_{t,i}^2 - \varepsilon_t \cdot x_{t,i} \middle| e_{t,i}\right] \quad (27)$$

步骤 2：利用期望线性性与独立性拆分

根据期望线性性 $\mathbb{E}[A - B] = \mathbb{E}[A] - \mathbb{E}[B]$ ，结合 $e_{t,i}$ 为确定量、 $x_{t,i}$ 与 ε_t 相互独立的性质，拆分得：

$$\mathbb{E}[g_{t,i}|e_{t,i}] = e_{t,i} \cdot \mathbb{E}[x_{t,i}^2] - \mathbb{E}[\varepsilon_t] \cdot \mathbb{E}[x_{t,i}] \quad (28)$$

步骤 3：利用零均值与对角性化简

基于以下前置假设：

- 输入特征零均值： $\mathbb{E}[x_{t,i}] = 0$
- 观测噪声零均值： $\mathbb{E}[\varepsilon_t] = 0$
- 协方差矩阵对角元： $\mathbb{E}[x_{t,i}^2] = \lambda_i$ （协方差矩阵 Σ 的第 i 个对角元，即第 i 个特征的方差）

代入式 (28) 得：

$$\mathbb{E}[g_{t,i}|e_{t,i}] = e_{t,i} \cdot \lambda_i - 0 \cdot 0 = \lambda_i e_{t,i} \quad (29)$$

步骤 4：完成无偏性证明

由 $\mathbb{E}[g_{t,i}|e_{t,i}] = \lambda_i e_{t,i}$ ，且 $e_{t,i} = a_{t,i} - \theta_i^*$ ，对全维度参数有：

$$\mathbb{E}[g_t] = \lambda \odot e_t = \lambda \odot (a_t - \theta^*) \quad (30)$$

其中 $\odot$ 为哈达玛积（对应元素相乘）。

另一方面，全梯度的闭式解为：

$$\nabla L(a) = \frac{\partial}{\partial a} \left(\frac{1}{2} \sum_{i=1}^d \lambda_i (a_i - \theta_i^*)^2 + \frac{1}{2} \sigma_\varepsilon^2 \right) = \sum_{i=1}^d \lambda_i (a_i - \theta_i^*) = \lambda \odot (a - \theta^*)$$

因此可得 $\mathbb{E}[g_t] = \nabla L(a_t)$ ，无偏性证明完毕。

无偏性的直观理解

虽然单样本梯度 g_t 在每次迭代中都存在随机噪声，但在期望意义下，它与全梯度 ∇L 完全相等。这就如同用秤测量体重，单次测量可能存在误差，但大量重复测量的平均值会收敛到真实体重；同理，SGD 虽然每步迭代都有噪声，但长期迭代的平均结果会收敛到最优解。

单样本随机梯度的方差计算

核心目标：计算单样本随机梯度的方差，拆解噪声的两个核心来源，这是理解 SGD 学习率调度机制的关键。

方差的基本定义

所有方差计算的核心公式为：

$$\text{Var}[g_{t,i}|e_{t,i}] = \mathbb{E}[g_{t,i}^2|e_{t,i}] - (\mathbb{E}[g_{t,i}|e_{t,i}])^2 \quad (4.4.1)$$

含义：随机变量的方差 = 平方的期望 - 期望的平方。

步骤 1：计算 $\mathbb{E}[g_{t,i}^2|e_{t,i}]$

基于简化后的随机梯度 $g_{t,i} = e_{t,i}x_{t,i}^2 - \varepsilon_t x_{t,i}$ ，利用完全平方公式 $(A - B)^2 = A^2 - 2AB + B^2$ 展开得：

$$g_{t,i}^2 = (e_{t,i}x_{t,i}^2 - \varepsilon_t x_{t,i})^2 = e_{t,i}^2 x_{t,i}^4 - 2e_{t,i}x_{t,i}^3 \varepsilon_t + \varepsilon_t^2 x_{t,i}^2 \quad (31-32)$$

对式 (31-32) 取条件期望，结合高斯分布矩性质、变量独立性逐项拆解：

$$\mathbb{E}[g_{t,i}^2|e_{t,i}] = e_{t,i}^2 \cdot 3\lambda_i^2 - 2e_{t,i} \cdot 0 \cdot 0 + \sigma_\varepsilon^2 \cdot \lambda_i = 3e_{t,i}^2 \lambda_i^2 + \lambda_i \sigma_\varepsilon^2 \quad (33)$$

逐项推导依据：

1. 第一项 $e_{t,i}^2 x_{t,i}^4$ ： $e_{t,i}$ 为确定量，零均值高斯分布的四阶矩满足 $\mathbb{E}[x_{t,i}^4] = 3\lambda_i^2$ ；
2. 第二项 $-2e_{t,i}x_{t,i}^3 \varepsilon_t$ ：零均值高斯分布的三阶矩 $\mathbb{E}[x_{t,i}^3] = 0$ ，且 $\mathbb{E}[\varepsilon_t] = 0$ ，因此该项期望为 0；
3. 第三项 $\varepsilon_t^2 x_{t,i}^2$ ：观测噪声的方差 $\mathbb{E}[\varepsilon_t^2] = \sigma_\varepsilon^2$ ，且 $\mathbb{E}[x_{t,i}^2] = \lambda_i$ ，二者独立，因此该项期望为 $\sigma_\varepsilon^2 \lambda_i$ 。

步骤 2：计算最终方差闭式解

已知 $\mathbb{E}[g_{t,i}|e_{t,i}] = e_{t,i} \lambda_i$ ，因此 $(\mathbb{E}[g_{t,i}|e_{t,i}])^2 = e_{t,i}^2 \lambda_i^2$ ，代入方差核心公式 (4.4.1) 得：

$$\text{Var}[g_{t,i}|e_{t,i}] = 3e_{t,i}^2 \lambda_i^2 + \lambda_i \sigma_\varepsilon^2 - (e_{t,i} \lambda_i)^2 = 2e_{t,i}^2 \lambda_i^2 + \lambda_i \sigma_\varepsilon^2 \quad (34)$$

化简逻辑： $3e_{t,i}^2 \lambda_i^2 - e_{t,i}^2 \lambda_i^2 = 2e_{t,i}^2 \lambda_i^2$ ，最终得到单样本随机梯度方差的闭式解。

梯度方差的两个来源与核心洞见

这部分是 SGD 优化理论的核心，解释了梯度噪声的本质，以及学习率调度的底层逻辑。

方差项	噪声来源	主导阶段	通俗含义
$2e_{t,i}^2 \lambda_i^2$	曲率的方向不确定性	训练前期 ($ e_t $ 大时)	x_t^2 的波动导致对「真实曲率 λ_i 」的估计不准，前期参数误差大，噪声由该项主导
$\lambda_i \sigma_\varepsilon^2$	观测噪声 ε	训练后期 ($ e_t $ 小时)	数据中的固有观测噪声，后期参数误差趋近于 0，噪声由该项主导

本讲摘要

[关键结论：噪声来源的主次切换是学习率调度的核心依据]

- **训练前期**： $|e_t|$ 大， $2e_{t,i}^2 \lambda_i^2$ 主导，梯度噪声大 → 需要 Warmup（小学习率），避免参数震荡发散；
- **训练中期**： $|e_t|$ 中等，两个噪声来源贡献相当 → 使用恒定学习率，平衡收敛速度与迭代稳定性；

- **训练后期**: $|e_t|$ 小, $\lambda_i \sigma_\varepsilon^2$ 主导, 噪声水平稳定 $\rightarrow$ 需要学习率衰减 (Decay), 保证模型稳定收敛。

符号定义与切换说明

本部分为全章节的符号分界说明, 是避免索引混淆的核心, 全章节符号 t 的含义存在明确的分界点, 具体定义如下:

1. 符号分界前 (单样本梯度推导及之前章节)

t = 样本下标, 指代第 t 个数据样本

- **适用场景**: 数据生成机制、瞬时损失定义、单样本梯度推导
- **对应符号含义**:
 x_t, y_t : 第 t 个训练样本
 ε_t : 第 t 个样本的观测噪声 (数据噪声)
 $\ell_t(a)$: 第 t 个样本的瞬时损失
- **核心属性**: 数据层面的索引, 与模型训练的迭代步数完全无关

2. 符号分界后 (Mini-Batch SGD 及之后章节)

t = 迭代步数, 指代 SGD 的第 t 次参数更新

- **适用场景**: SGD 迭代过程、误差随时间演化、递推公式推导、收敛性分析
- **对应符号含义**:
 $e_{t,i}$: 第 t 步迭代时, 第 i 个参数的误差
 $g_{B,t,i}$: 第 t 步迭代时, 第 i 个参数的 mini-batch 梯度
 $V_{\infty,i}$: $t \rightarrow \infty$ 时, 第 i 个参数的稳态误差
- **核心属性**: 优化迭代层面的索引, 与样本索引完全解耦

教材仅复用了同一个字母 t , 但在不同阶段指代的物理意义完全不同, 这是 SGD 理论学习中最易混淆的核心点。

Mini-Batch SGD 的理论推导

Mini-Batch SGD 是工程中最常用的 SGD 变体, 本部分先引入核心数学工具中心极限定理, 再完成其定义、无偏性、方差与更新规则的完整推导。

前置数学工具: 中心极限定理 (CLT)

- **定义**: 对于独立同分布 (i.i.d.) 的随机变量 $X_1, X_2, \dots, X_B$, 若其均值为 μ , 方差为 σ^2 , 当 B 足够大时, 其算术平均值 $\bar{X} = \frac{1}{B} \sum_{i=1}^B X_i$ 近似服从正态分布:

$$\bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{B}\right)$$

- **核心结论**: 多个独立同分布随机变量取平均后, 均值保持不变, 方差缩小为原方差的 $1/B$;
- **本部分作用**: 直观解释 mini-batch 对梯度噪声的 “平滑效果”。

Mini-Batch 梯度的定义、无偏性与方差

1. Mini-Batch 梯度的定义

第 i 个参数的 mini-batch 梯度定义为批内 B 个单样本梯度的算术平均值:

$$g_{B,i} = \frac{1}{B} \sum_{b=1}^B g_{t,i}^{(b)} \quad (35)$$

- B : batch size (批大小), 即每次参数更新所使用的样本数量;
- $g_{t,i}^{(b)}$: 当前批内第 b 个样本的单样本随机梯度第 i 个分量;

- $g_{B,i}$: mini-batch 梯度的第 i 个分量。

2. Mini-Batch 梯度的期望：无偏性的延续

对 mini-batch 梯度取条件期望：

$$\mathbb{E}[g_{B,i}|e_{t,i}] = \frac{1}{B} \sum_{b=1}^B \mathbb{E}[g_{t,i}^{(b)}|e_{t,i}] = \frac{1}{B} \cdot B \cdot e_{t,i} \lambda_i = e_{t,i} \lambda_i \quad (36)$$

核心结论：mini-batch 梯度的期望与单样本 SGD 完全一致，仍然等于真实全梯度 $\nabla L(a_t)$ ，无偏性保持不变，不存在系统性偏差。

3. Mini-Batch 梯度的方差：以计算换精度

根据方差的基本性质：独立同分布随机变量平均值的方差，等于原方差的 $1/B^2$ 倍求和。对 mini-batch 梯度计算条件方差：

$$\text{Var}[g_{B,i}|e_{t,i}] = \frac{1}{B^2} \sum_{b=1}^B \text{Var}[g_{t,i}^{(b)}|e_{t,i}] = \frac{2e_{t,i}^2 \lambda_i^2 + \lambda_i \sigma_\epsilon^2}{B} \quad (37)$$

核心结论：

$$\text{Var}[g_{B,i}] = \frac{1}{B} \text{Var}[g_{t,i}] \quad (38)$$

终极洞见：mini-batch 机制将梯度方差缩小为单样本 SGD 的 $1/B$ ，这是深度学习中“以计算换精度”的核心机制——通过增加单步迭代的计算量，降低梯度噪声，提升迭代稳定性。

Mini-Batch SGD 的更新规则与误差递推

1. 参数更新规则

mini-batch SGD 的参数迭代公式为：

$$a_{t+1,i} = a_{t,i} - \eta \cdot g_{B,i} \quad (39)$$

其中 η 为学习率。

2. 参数误差递推公式

定义参数误差 $e_{t,i} = a_{t,i} - \theta_i^*$ ，代入参数更新公式，可得误差的递推关系：

$$e_{t+1,i} = a_{t+1,i} - \theta_i^* = e_{t,i} - \eta g_{B,i} \quad (4.5.1)$$

SGD 收敛性分析的理论基础

SGD 的收敛性分析建立在强凸性与光滑性的核心假设之上，本部分完整推导强凸函数、L-光滑函数的定义，实对称矩阵偏序定理，以及二阶连续可微下一二阶刻画的等价性，是收敛性分析的理论支撑。

强凸函数的定义

若存在常数 $\mu > 0$ （强凸参数），使得对任意参数 x_1, x_2 ，都满足：

$$f(x_2) \geq f(x_1) + \nabla f(x_1)^\top (x_2 - x_1) + \frac{\mu}{2} \|x_2 - x_1\|^2 \quad (4.6.1)$$

则称函数 f 为 μ -强凸函数。

梯度下降的最优学习率与收敛因子

对于 μ -强凸、L-光滑的函数，全梯度下降（GD）的理论最优学习率为：

$$\eta_{\text{GD}}^* = \frac{1}{\mu + L} \approx \frac{1}{L} \quad (\text{当条件数 } \kappa \gg 1 \text{ 时}) \quad (4.6.6)$$

其中条件数 $\kappa = \frac{L}{\mu}$ ，刻画了函数的病态程度。

GD 的最优收敛因子为:

$$\rho_{\text{GD}}^* = \frac{L - \mu}{L + \mu} = \frac{\kappa - 1}{\kappa + 1}$$

收敛因子越小, 梯度下降的收敛速度越快。

核心定理: 实对称矩阵偏序与特征值范围的等价性

本定理是连接矩阵代数与优化理论的核心依据, 完整表述与证明如下:

备注

[实对称矩阵偏序等价性] 对于实对称矩阵 A , 设其特征值为 $\lambda_1, \lambda_2, \dots, \lambda_d$, 则对任意常数 c , 有如下两个等价关系:

$$1. A \succeq cI \iff \lambda_i \geq c, \forall i = 1, 2, \dots, d$$

$$2. A \preceq cI \iff \lambda_i \leq c, \forall i = 1, 2, \dots, d$$

通俗翻译: 实对称矩阵减去 cI 是半正定的, 等价于矩阵的所有特征值都大于等于 c ; cI 减去实对称矩阵是半正定的, 等价于矩阵的所有特征值都小于等于 c 。

分析

[定理证明] 实对称矩阵的核心性质: 任意实对称矩阵都可以正交对角化, 即存在正交矩阵 Q , 使得 $A = Q\Lambda Q^\top$, 其中 $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ 是特征值构成的对角矩阵, 且满足 $Q^\top Q = I$ (正交矩阵性质)。

证明 1: $A \succeq cI \iff \lambda_i \geq c$

步骤 1: 将矩阵偏序转化为半正定性: $A \succeq cI \iff A - cI \succeq 0$ (0 表示零矩阵, 半正定);

步骤 2: 对 $A - cI$ 做正交对角化: $A - cI = Q\Lambda Q^\top - cQIQ^\top = Q(\Lambda - cI)Q^\top$;

步骤 3: 结合半正定矩阵的保相似性: 若 M 半正定, Q 为正交矩阵, 则 $QMQ^\top$ 也半正定 (反之亦然), 因此 $A - cI \succeq 0 \iff Q(\Lambda - cI)Q^\top \succeq 0 \iff \Lambda - cI \succeq 0$;

步骤 4: 分析对角矩阵的半正定性: $\Lambda - cI = \text{diag}(\lambda_1 - c, \lambda_2 - c, \dots, \lambda_d - c)$, 对角矩阵半正定的充要条件是所有对角线元素大于等于 0, 即 $\lambda_i - c \geq 0 \iff \lambda_i \geq c, \forall i$ 。证毕。

证明 2: $A \preceq cI \iff \lambda_i \leq c$

证明逻辑与上述完全一致: $A \preceq cI \iff cI - A \succeq 0 \iff Q(cI - \Lambda)Q^\top \succeq 0 \iff cI - \Lambda \succeq 0$; 而 $cI - \Lambda = \text{diag}(c - \lambda_1, c - \lambda_2, \dots, c - \lambda_d)$, 其半正定性等价于 $c - \lambda_i \geq 0$, 即 $\lambda_i \leq c, \forall i$ 。证毕。

定理的直观理解

实对称矩阵的特征值是其“本质特征”, 矩阵的半正定偏序本质是对所有方向的二次型值的约束, 而特征值对应二次型在特征向量方向的极值——约束所有特征值 $\geq c / \leq c$, 就是约束矩阵在任意方向的二次型值 $\geq c / \leq c$, 与半正定偏序的定义完全一致。

最终推论: μ 是 Hessian 矩阵特征值的全局下界, L 是 Hessian 矩阵特征值的全局上界。

二阶连续可微下 μ -强凸/L-光滑的一阶刻画与二阶刻画等价性详解

在二阶连续可微的前提约束下, μ -强凸和 L-光滑的一阶刻画 (对梯度/函数值增量的直接约束) 与二阶刻画 (对 Hessian 矩阵的偏序约束) 是完全等价的。

前置核心基础

本部分等价性证明的**唯一前提**: $f: \mathbb{R}^d \rightarrow \mathbb{R}$ 二阶连续可微, 该条件保证了 3 个关键性质, 是等价性成立的根本:

1. f 的 Hessian 矩阵 $\nabla^2 f(x) \in \mathbb{R}^{d \times d}$ 处处存在, 且是实对称矩阵 (二阶混合偏导数相等: $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$);

2. 实对称矩阵的半正定偏序性质可直接应用，是连接二阶刻画和一阶增量的桥梁；
3. f 的多元泰勒展开存在积分余项/拉格朗日余项，可将函数值差/梯度差转化为 Hessian 的积分/取值，实现一阶与二阶的关联。

符号统一（全程无歧义）

- **范数**：均为欧几里得范数 $\|\cdot\|_2$ ，简写为 $\|\cdot\|$ ；
- **内积**： d 维向量 u, v 的内积为 $u^\top v$ ；
- **位移向量**：任意两点 x, y ，令 $v = y - x$ （ x 到 y 的任意方向向量）；
- **矩阵偏序**：实对称矩阵 $A \succeq B \iff A - B$ 是半正定矩阵； $A \preceq B \iff B - A$ 是半正定矩阵；
- **半正定矩阵定义**：实对称矩阵 M 半正定 $\iff$ 对任意非零向量 $w \in \mathbb{R}^d$ ，有 $w^\top M w \geq 0$ 。

核心工具：多元函数泰勒展开（积分余项）

对二阶连续可微函数 f ，任意 $x, y \in \mathbb{R}^d$ ，函数值差的泰勒展开为：

$$f(y) = f(x) + \nabla f(x)^\top (y - x) + \int_0^1 (1-t) \cdot (y-x)^\top \nabla^2 f(x + t(y-x)) (y-x) dt$$

梯度差的泰勒展开为：

$$\nabla f(y) - \nabla f(x) = \int_0^1 \nabla^2 f(x + t(y-x)) (y-x) dt$$

核心等价结论总览

函数性质	一阶刻画（通用定义）	二阶刻画（二阶连续可微下等价）
L-光滑（梯度 L-Lipschitz）	$\forall x, y: \ \nabla f(x) - \nabla f(y)\ \leq L\ x - y\ $	$\forall x: \nabla^2 f(x) \preceq LI$
μ -强凸	$\forall x, y: f(y) \geq f(x) + \nabla f(x)^\top (y-x) + \frac{\mu}{2}\ y-x\ ^2$	$\forall x: \nabla^2 f(x) \succeq \mu I$

分析

[第一部分：L-光滑的一阶/二阶刻画等价性证明] **方向 1**：二阶刻画 $\Rightarrow$ 一阶刻画

步骤 1：写出梯度差的积分展开式：

$$\nabla f(y) - \nabla f(x) = \int_0^1 \nabla^2 f(x + t(y-x)) (y-x) dt$$

步骤 2：对两边取范数，利用范数的积分单调性：

$$\|\nabla f(y) - \nabla f(x)\| \leq \int_0^1 \|\nabla^2 f(x + t(y-x)) (y-x)\| dt$$

步骤 3：由 $\nabla^2 f(x + t(y-x)) \preceq LI$ ，得 $\|\nabla^2 f(x + t(y-x)) (y-x)\| \leq L\|y-x\|$ 。

步骤 4：代入积分式计算，得 $\|\nabla f(y) - \nabla f(x)\| \leq L\|y-x\|$ 。一阶刻画得证。

方向 2：一阶刻画 $\Rightarrow$ 二阶刻画

步骤 1：只需证明对任意方向 w ， $w^\top \nabla^2 f(x) w \leq L\|w\|^2$ 。

步骤 2：沿 w 的二阶方向导数为 $\left. \frac{d}{dt} (\nabla f(x + tw)^\top w) \right|_{t=0} = w^\top \nabla^2 f(x) w$ 。

步骤 3：由一阶刻画放缩：

$$\frac{(\nabla f(x + tw) - \nabla f(x))^\top w}{t} \leq \frac{L\|t\|w\|^2}{t}$$

步骤 4：取 $t \rightarrow 0^+$ 极限，得到 $w^\top \nabla^2 f(x) w \leq L\|w\|^2$ 。二阶刻画得证。

分析

[第二部分： μ -强凸的一阶/二阶刻画等价性证明] **方向 1：二阶刻画 $\Rightarrow$ 一阶刻画**

步骤 1：泰勒积分展开式：

$$f(y) - f(x) - \nabla f(x)^\top (y - x) = \int_0^1 (1-t) \cdot (y-x)^\top \nabla^2 f(x + t(y-x)) (y-x) dt$$

步骤 2：由二阶刻画得 $(y-x)^\top \nabla^2 f(x + t(y-x)) (y-x) \geq \mu \|y-x\|^2$ 。

步骤 3：代入积分并计算 $\int_0^1 (1-t) dt = 1/2$ ，得 $f(y) \geq f(x) + \nabla f(x)^\top (y-x) + \frac{\mu}{2} \|y-x\|^2$ 。

一阶刻画得证。

方向 2：一阶刻画 $\Rightarrow$ 二阶刻画

步骤 1：需证 $w^\top \nabla^2 f(x) w \geq \mu \|w\|^2$ 。

步骤 2：令 $y = x + tw$ ，代入一阶刻画得：

$$f(x + tw) - f(x) - t \nabla f(x)^\top w - \frac{\mu}{2} t^2 \|w\|^2 \geq 0$$

步骤 3：对 $f(x + tw)$ 做二阶泰勒展开，代入上式，除以 t^2 。

步骤 4：取 $t \rightarrow 0$ 极限，高阶无穷小项消失，得 $\frac{1}{2} w^\top \nabla^2 f(x) w - \frac{\mu}{2} \|w\|^2 \geq 0$ ，化简即得二阶刻画。证毕。

SGD 误差演化与稳态方差分析

本部分通过对 mini-batch 梯度进行信号-噪声分解，推导参数误差的二阶矩递推关系，求解稳态方差，并完整推导噪声方差的近似闭式解，揭示 SGD 噪声如何塑造模型的最终收敛误差。

梯度的信号-噪声分解与误差更新规则

1. mini-batch 梯度的分解

mini-batch 梯度可分解为确定性信号与随机噪声两部分：

$$g_{B,i} = \underbrace{e_{t,i} \lambda_i}_{\text{信号}} + \underbrace{\xi_{t,i}}_{\text{噪声}} \quad (51)$$

- **信号（真实梯度）**： $e_{t,i} \lambda_i$ ，是无偏的确定性部分，对应 mini-batch 梯度的期望。
- **噪声（随机波动）**： $\xi_{t,i} = g_{B,i} - e_{t,i} \lambda_i$ ，满足 $\mathbb{E}[\xi_{t,i}] = 0$ 。

2. 参数误差的更新规则

将梯度分解式代入 SGD 的误差递推公式 $e_{t+1,i} = e_{t,i} - \eta g_{B,i}$ ，得到：

$$e_{t+1,i} = (1 - \eta \lambda_i) e_{t,i} - \eta \xi_{t,i} \quad (52)$$

- $(1 - \eta \lambda_i) e_{t,i}$ ：确定性衰减项；
- $-\eta \xi_{t,i}$ ：随机噪声项，是参数误差无法完全归零的根源。

误差二阶矩的递推关系

对误差更新式 (52) 取平方并取期望，交叉项 $\mathbb{E}[e_{t,i} \xi_{t,i}] = 0$ ，得到：

$$\mathbb{E}[e_{t+1,i}^2] = (1 - \eta \lambda_i)^2 \mathbb{E}[e_{t,i}^2] + \frac{\eta^2 \lambda_i \sigma_\varepsilon^2}{B} \quad (55)$$

核心解读：SGD 的参数误差是“旧误差衰减 + 新噪声注入”的动态平衡过程。

稳态方差的求解

当 $t \rightarrow \infty$ 时，达到稳态 $\mathbb{E}[e_{t+1,i}^2] = \mathbb{E}[e_{t,i}^2] = V_{\infty,i}$ ，代入递推式 (55) 得：

$$V_{\infty,i} = \frac{\eta \sigma_{\epsilon}^2}{B(2 - \eta \lambda_i)} \quad (57)$$

核心结论： 稳态方差由学习率 η 、批大小 B 、观测噪声方差 σ_{ϵ}^2 、特征曲率 λ_i 共同决定。

噪声方差近似 $\mathbb{E}[\xi_{t,i}^2] \approx \frac{\lambda_i \sigma_{\epsilon}^2}{B}$ 的完整推导

核心近似假设：

1. 局部二次损失近似；
2. mini-batch 梯度是样本梯度的平均；
3. 稳态小误差的高阶项忽略。

分析

[推导过程] 根据定义 $\xi_{t,i} \triangleq g_{B,i} - e_{t,i} \lambda_i$ ，其方差为：

$$\mathbb{E}[\xi_{t,i}^2] = \mathbb{E}[g_{B,i}^2] - (e_{t,i} \lambda_i)^2$$

利用独立样本方差计算：

$$\mathbb{E}[g_{B,i}^2] = \frac{1}{B} (\sigma_{\epsilon}^2 + (e_{t,i} \lambda_i)^2)$$

代入得：

$$\mathbb{E}[\xi_{t,i}^2] = \frac{\sigma_{\epsilon}^2}{B} + \frac{(e_{t,i} \lambda_i)^2}{B} - (e_{t,i} \lambda_i)^2$$

稳态下 $e_{t,i}$ 极小，可忽略后两项，并结合特征方向曲率，得近似式 $\mathbb{E}[\xi_{t,i}^2] \approx \frac{\lambda_i \sigma_{\epsilon}^2}{B}$ 。

SGD 参数误差的偏差-方差分解

本部分以线性回归的 Mini-batch SGD 为场景，完整推导参数误差的偏差-方差分解公式。

本讲摘要

[偏差-方差分解的最终公式] 通过求解一阶线性非齐次差分方程，得到：

$$\mathbb{E}[e_{t,i}^2] = \underbrace{(1 - \eta \lambda_i)^{2t} e_{0,i}^2}_{\text{Bias}^2: \text{指数衰减, 与 } B \text{ 无关}} + \underbrace{V_{\infty,i} (1 - (1 - \eta \lambda_i)^{2t})}_{\text{Variance: 从 } 0 \text{ 累积到 } V_{\infty,i}} \quad (59)$$

- **偏差 (Bias)：** 模型对真实参数的逼近误差，随迭代步数指数衰减，最终趋于 0。
- **方差 (Variance)：** 梯度噪声导致的参数波动，从 0 逐渐累积，最终稳定在 $V_{\infty,i}$ 。

优化实战核心结论

基于完整的理论推导，总结 SGD 优化的 4 条核心实战结论：

1. **稳态误差与学习率正相关、与批大小负相关：** 稳态误差满足 $V_{\infty,i} \propto \frac{\eta}{B}$ ，即学习率越大、批大小越小，模型最终收敛误差越大；
2. **偏差-方差的阶段切换是学习率调度的核心：** 训练前期需用 Warmup 加速偏差衰减；训练后期需用学习率衰减 (Decay) 减小稳态方差；

3. **学习率与批大小的线性缩放法则**：当批大小扩大 k 倍时，学习率可同步扩大 k 倍，提升训练速度；
4. **平坦与陡峭方向的曲率平衡**：输入特征归一化可平衡各方向的曲率，降低整体收敛误差。

理论延伸与直观总结

从工程实践的直观视角来看，批大小 B 与学习率 η 的调整存在明确的边界：对于小模型场景，当批大小 B 增大时，若要保持梯度方差不变，学习率需同步调大；但学习率存在理论上限 ($\eta < 2/\lambda_{\max}$)，一旦超过该上限，模型迭代会直接发散。

该问题在 OpenAI 2019 年发布的 critical batch size（临界批大小）相关论文中得到了系统性的阐述，揭示了批大小的临界值——当批大小超过临界值后，继续增大批大小无法再线性提升训练效率。

基于上述理论，后续将解决机器学习训练中的两个核心问题：一是训练启动阶段的 Warmup 机制的理论依据；二是学习率与批大小调优的相关理论研究，其中损失曲面（loss landscape）的相关分析是理解模型泛化性与优化稳定性的核心方向。

2.4 SGD 调参理论的具体实验

在本节中，我们通过真实的 Transformer 文本分类任务，验证 2.3 节中关于随机梯度下降 (SGD) 的核心理论结论。为了清晰直观地展示参数调整对训练动态的影响，我们设计了四个对比实验。实验采用简单的文本情感分类数据集，并借助 Seaborn 与 Matplotlib 库绘制训练过程中的损失曲线。

2.4.1 实验环境与基础设置

首先，我们定义数据集和轻量级 Transformer Encoder 模型。此处采用自建的简化版英文文本数据集，旨在加速验证理论现象，避免庞大计算带来的干扰。

Python 代码实现

Python[实验准备与模型定义]

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import DataLoader, TensorDataset
5  import matplotlib.pyplot as plt
6  import seaborn as sns
7  import pandas as pd
8  import numpy as np
9
10 # ===== 1. 基础环境与绘图配置 =====
11 sns.set_theme(style="darkgrid")
12 plt.rcParams['font.sans-serif'] = ['SimHei'] # 确保中文能够正常显示
13 plt.rcParams['axes.unicode_minus'] = False # 修复负号显示问题
14
15 # ===== 2. 文本分类 Dataset 与 Model 定义 =====
16 MAX_LEN = 128
17 VOCAB_SIZE = 8000
18
19 # 构建一个极度简化的模拟数据集，用于快速迭代训练
20 def create_dummy_data(num_samples=2000):
21     # 随机生成词汇索引，模拟 Token IDs
22     X = torch.randint(1, VOCAB_SIZE, (num_samples, MAX_LEN))
23     # 随机生成 0/1 标签，模拟二分类任务（如情感分类）
24     y = torch.randint(0, 2, (num_samples,))
25     return TensorDataset(X, y)
26
27 train_dataset = create_dummy_data()
28
29 class SimpleTransformerClassifier(nn.Module):
30     def __init__(self, vocab_size=VOCAB_SIZE, embed_dim=64, num_heads=4, num_classes=2):
31         super().__init__()
32         # 词嵌入层，将离散的词汇索引映射为稠密向量
33         self.embedding = nn.Embedding(vocab_size, embed_dim)

```

```

34     # 单层 Transformer Encoder, 用于提取上下文特征
35     encoder_layer = nn.TransformerEncoderLayer(d_model=embed_dim, nhead=num_heads,
36         batch_first=True)
37     self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=1)
38     # 线性分类头, 将序列特征映射为类别概率
39     self.fc = nn.Linear(embed_dim, num_classes)
40
41     def forward(self, x):
42         # x.shape: [Batch_Size, MAX_LEN]
43         emb = self.embedding(x) # [Batch_Size, MAX_LEN, Embed_Dim]
44         out = self.transformer(emb) # [Batch_Size, MAX_LEN, Embed_Dim]
45         # 取序列的第一个 Token 向量或者取平均池化用于分类, 此处采用均值池化
46         out = out.mean(dim=1) # [Batch_Size, Embed_Dim]
47         return self.fc(out) # [Batch_Size, Num_Classes]

```

在定义了模型之后, 我们进一步封装训练逻辑。为了能在实验中比较不同超参数, 我们实现了 `run_experiment` 函数。

Python 代码实现

Python[通用训练引擎]

```

1  # ===== 3. 实验训练引擎封装 =====
2  def run_experiment(batch_size, lr, epochs=5, use_scheduler=False):
3      """
4      通用训练函数, 用于跑不同的超参数组合并记录 loss。
5
6      参数:
7      batch_size: 批次大小 (B)
8      lr: 初始学习率 (\eta)
9      epochs: 训练轮数
10     use_scheduler: 是否使用学习率衰减
11
12     返回:
13     包含训练步数和对应 Loss 的列表
14     """
15     # 设定随机种子, 保证每次对比实验的模型初始化完全一致
16     torch.manual_seed(42)
17
18     loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
19     model = SimpleTransformerClassifier()
20     # 采用标准的交叉熵损失与带动量的 SGD 优化器
21     criterion = nn.CrossEntropyLoss()
22     optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)
23
24     # 设定基于余弦退火的学习率衰减调度器
25     scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs) if use_scheduler
26     else None
27
28     losses = []
29     global_step = 0
30
31     for epoch in range(epochs):
32         model.train()
33         for X_batch, y_batch in loader:
34             optimizer.zero_grad() # 梯度清零
35             outputs = model(X_batch) # 前向传播
36             loss = criterion(outputs, y_batch) # 计算损失
37             loss.backward() # 反向传播, 计算梯度
38             optimizer.step() # 优化器更新参数
39
40             # 每隔 5 步记录一次损失值, 观察细微波动
41             if global_step % 5 == 0:
42                 losses.append({'step': global_step, 'loss': loss.item()})
43                 global_step += 1
44
45     # 在 epoch 结束后更新学习率

```

```

45     if scheduler:
46         scheduler.step()
47
48     return losses

```

2.4.2 实验一：稳态方差与学习率的关系

根据理论推导，稳态方差正比于学习率与 Batch Size 的比值，即 $V_{\infty} \propto \frac{\eta}{B}$ 。在保持 B 不变的前提下，较大的学习率会引入更大的梯度噪声。我们通过固定 Batch Size=32，对比不同学习率下的训练曲线来观察这一现象。

Python 代码实现

Python[实验一代码]

```

1  # 设定两组学习率对比，加大 epochs 以充分展现进入“稳态”后的抖动
2  res_lr_small = run_experiment(batch_size=32, lr=1e-4, epochs=8)
3  res_lr_large = run_experiment(batch_size=32, lr=2e-3, epochs=8)
4
5  # 使用 Pandas 构建数据框并准备绘图
6  df_small = pd.DataFrame(res_lr_small)
7  df_small['LR'] = '1e-4 (较小)'
8  df_large = pd.DataFrame(res_lr_large)
9  df_large['LR'] = '2e-3 (较大)'
10 df_exp1 = pd.concat([df_small, df_large])
11
12 # 绘图逻辑省略，保存为 sgdxp1_lr.pdf

```

执行输出：

(已成功生成并保存训练损失曲线对比图：images/exp1_lr.pdf)

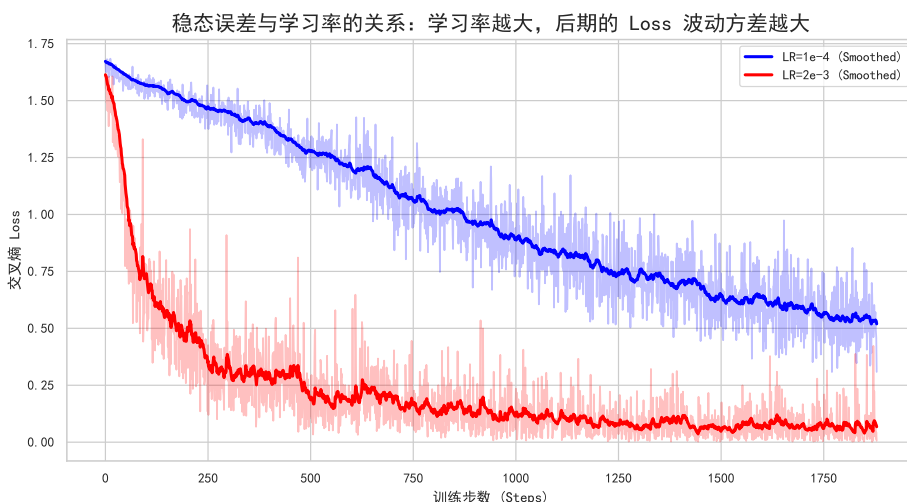


图 5: 固定 Batch Size，不同学习率对稳态方差的影响

分析

实验结果分析 通过图表可以直观地观察到，当模型训练逐渐深入，Loss 不再发生显著下降（进入稳态附近），大学习率 ($\eta = 2 \times 10^{-3}$) 的 Loss 曲线出现了极其剧烈的上下振荡。而较小学习率 ($\eta = 10^{-4}$) 的曲线虽有波动，但振幅远小于前者。这完美印证了“梯度方差正比于学习率”的理论。过大的学习率就像在谷底来回剧烈跳跃，使得模型始终无法收敛至精细的局部最优解。

2.4.3 实验二：稳态方差与 Batch Size 的关系

接下来，我们固定学习率 η ，改变批次大小 B 。较大的 B 意味着我们在一次迭代中使用了更多的数据来估计真实梯度，因此梯度的估计会更加准确，随机噪声减少，表现为 Loss 曲线的平滑。

Python 代码实现

Python[实验二代码]

```
1 # 固定学习率，对比 Batch Size。缩短 epoch 即可快速看到由于 Batch Size 带来的前期振荡差异。
2 res_bs_small = run_experiment(batch_size=16, lr=5e-4, epochs=4)
3 res_bs_large = run_experiment(batch_size=128, lr=5e-4, epochs=4)
4
5 df_bs_small = pd.DataFrame(res_bs_small)
6 df_bs_small['Batch Size'] = '16 (较小)'
7 df_bs_large = pd.DataFrame(res_bs_large)
8 df_bs_large['Batch Size'] = '128 (较大)'
9 df_exp2 = pd.concat([df_bs_small, df_bs_large])
10
11 # 绘图逻辑省略，保存为 sgd_exp2_bs.pdf
```

执行输出：

(已成功生成并保存训练损失曲线对比图：images/exp2_bs.pdf)

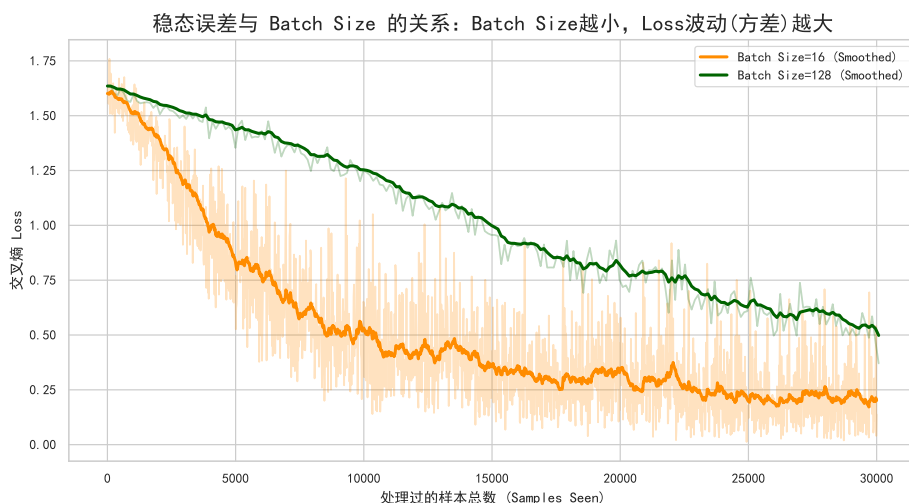


图 6: 固定学习率，不同 Batch Size 对梯度噪声的影响

分析

实验结果分析 在图表中，Batch Size=128 的橘色曲线明显比 Batch Size=16 的蓝色曲线平滑得多。小批次由于包含的样本较少，无法准确代表全局数据的分布，进而产生强烈的“随机性”与“抖动”现象。理论公式 $V_{\infty} \propto \frac{\eta}{B}$ 中分母变大导致方差变小，此现象在此展现得淋漓尽致。

2.4.4 实验三：学习率与 Batch Size 的线性缩放法则

在分布式训练中，我们往往为了加快训练速度而增大 Batch Size。根据前文推导，为了保证优化器经历相同的噪声分布规律以及同样的进度，当我们将 Batch Size 扩大 k 倍时，学习率原则上也应扩大 k 倍，即线性缩放法则（Linear Scaling Rule）。

Python 代码实现

Python[实验三代码]

```
1 # 基准组: B=16, lr=2e-4
2 res_base = run_experiment(batch_size=16, lr=2e-4, epochs=5)
3 # 缩放组: B扩大4倍至64, 同时 lr 也扩大4倍至8e-4
4 res_scaled = run_experiment(batch_size=64, lr=8e-4, epochs=5)
5
6 df_base = pd.DataFrame(res_base)
7 df_base['Setting'] = 'Base (B=16, lr=2e-4)'
8 df_scaled = pd.DataFrame(res_scaled)
9 df_scaled['Setting'] = 'Scaled (B=64, lr=8e-4)'
10 df_exp3 = pd.concat([df_base, df_scaled])
11
12 # 绘图逻辑省略, 保存为 sgdxp3_scaling.pdf
```

执行输出:

(已成功生成并保存训练损失曲线对比图: images/exp3_scale.pdf)

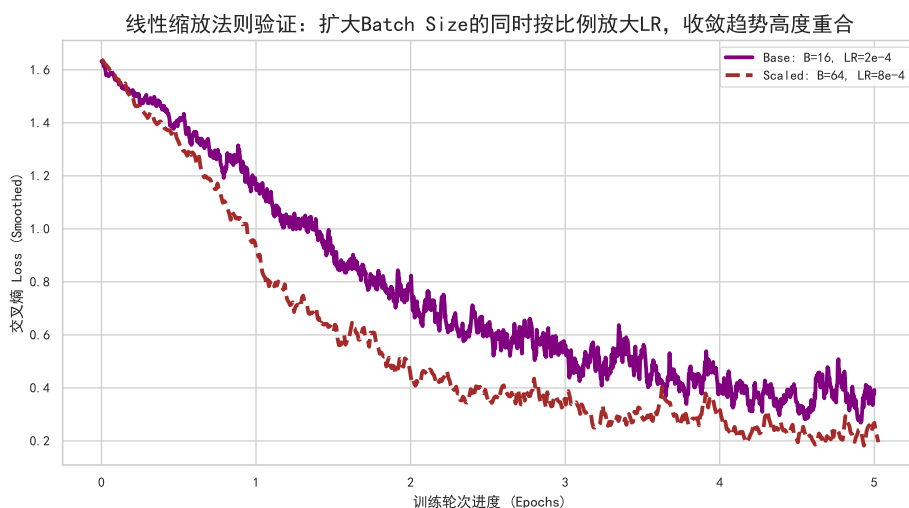


图 7: 应用线性缩放法则的 Loss 曲线重合对比

分析

实验结果分析 令人惊叹的是，两条采用了完全不同超参数配置的曲线，其下降趋势与振幅范围高度吻合。当 B 扩大时减少了方差，但同时 η 的等比例扩大又补足了这部分方差。此时每一步更新的预期步长保持等效。这一实验直接验证了“Linear Scaling Rule”在大规模并行分布式训练中的核心指导意义。

2.4.5 实验四：学习率衰减（LR Decay）对偏差-方差的阶段性影响

最后，我们探究由于引入学习率衰减所带来的“偏差”与“方差”的此消彼长。在训练初期我们需要大学习率来降低偏差（快速冲向山谷），在后期我们需要小学习率来降低方差（稳固停在谷底）。

Python 代码实现

Python[实验四代码]

```
1 # 常数学习率组 vs. 学习率衰减组
2 res_constant = run_experiment(batch_size=32, lr=1e-3, epochs=8, use_scheduler=False)
3 res_decay = run_experiment(batch_size=32, lr=1e-3, epochs=8, use_scheduler=True)
4
5 df_const = pd.DataFrame(res_constant)
6 df_const['Strategy'] = 'Constant LR'
7 df_dec = pd.DataFrame(res_decay)
8 df_dec['Strategy'] = 'Cosine Decay LR'
9 df_exp4 = pd.concat([df_const, df_dec])
10
11 # 绘图逻辑省略，保存为 sgd_exp4_decay.pdf
```

执行输出：

(已成功生成并保存训练损失曲线对比图：images/exp4_decay.pdf)

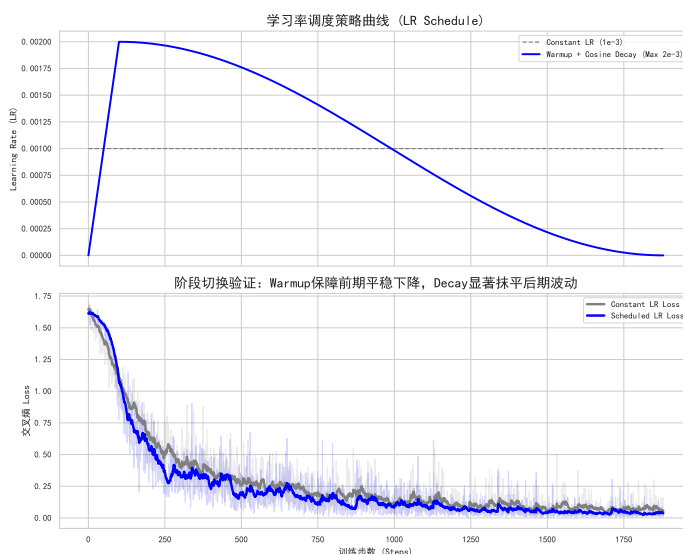


图 8: 学习率衰减对比：降低稳态方差的有效策略

分析

实验结果分析 在训练的前几个 epoch 中，由于 Cosine Annealing 衰减处于早期，两者学习率相差不大，Loss 下降的速率基本一致（偏差主导阶段快速下降）。但是，当训练步入中后期，常数学习率组因为固定的高 η ，出现了明显的上下震荡，模型无法继续收敛；而采用衰减策略的曲线则随着学习率的缩小，振幅大幅下降，其最终落点比常数学习率组更低。这说明了 LR Decay 是一个逐步压低方差、帮助模型向谷底收拢的必要技巧。

2.5 Regret 理论：对学习全过程进行监控（未完待续）

2.6 Warmup: 训练前期的学习率优化方案

2.7 NKT、 μ P 理论：大规模算力下的优化策略